



DAYS 01-20 · REAL-MACHINE LAB

bash-mastery-linux

Twenty days of Linux operations — the host, the network, security hardening, and configuration management — on real machines you build yourself.

20

DAYS

125

SCRIPTS

216

CHECKS

3

MACHINES

systemd

LVM

journald

network namespaces

BIND / unbound

nftables

SELinux

Ansible

WireGuard

nginx + TLS

auditd

restic

libvirt / KVM

bash-mastery-linux

Contents

FRONT MATTER

Repository overview	3
Curriculum — twenty days	7
The lab	12

PHASE 1 — THE HOST (DAYS 01–05)

01	systemd and the boot path	15
02	Users, sudo, permissions and ACLs	20
03	Processes, signals, cgroups v2 and limits	25
04	Storage: LVM, filesystems and mount units	30
05	Logs and time: journald, logrotate and chrony	36

PHASE 2 — THE NETWORK (DAYS 06–10)

06	Interfaces, routing and building the namespace lab	43
07	The DNS resolution path	48
08	Running DNS: authoritative and recursive	53
09	Packet-level debugging	58
10	TLS on the wire and a private CA	62

PHASE 3 — HARDENING AND CONFIGURATION MANAGEMENT (DAYS 11–15)

11	firewalld, and the nftables underneath it	67
12	SSH hardening, bastions and fail2ban	72
13	SELinux: contexts, booleans and denial triage	77
14	Ansible fundamentals	82
15	Ansible roles: your hardening baseline	87

PHASE 4 — PRODUCTION OPERATIONS (DAYS 16–20)

16	WireGuard: a private network between hosts	92
17	Reverse proxy and TLS termination	96
18	Bridges, VLANs and link aggregation	101
19	Intrusion detection and audit alerting	105
20	Backup, restore and the restore drill	111

REFERENCE

Contributing	120
Engineering handoff	124
License	168

Repository overview

README.md

Twenty days of Linux operations — the host, the network, security hardening, and configuration management — on real machines you build yourself.

Nothing here is simulated. There is no offline mode, no fake output, and no capstone application. You get three virtual machines and a kernel-level network lab, and you operate them until they behave.

Get started

```
./lab/lab.sh check      # can this machine run the lab? start here
./lab/lab.sh image      # Rocky 9 base image, ~1 GB, downloaded once
./lab/lab.sh up control # a real VM, about a minute
./lab/lab.sh ssh control # you are in
```

`check` tells you exactly what your distribution is missing and prints the install command for it. On Ubuntu that is usually:

```
sudo apt-get install -y qemu-system-x86 libvirt-daemon-system libvirt-clients libvirt-daemon-config-
network virtinst acl
sudo systemctl enable --now libvirtd
sudo usermod -aG kvm,libvirt "$USER" # then log out and back in
```

The `usermod` step is the one people skip. Without it `/dev/kvm` is not writable and every VM creation fails on permissions.

Not ready to install a hypervisor? Start at Day 06 instead. Days 06–10 and 18 need only `iproute2` and root:

```
sudo ./lab/lab.sh netns-up      # builds the whole network, 0 MB
sudo ./lab/lab.sh netns-status # shows it and ping-tests it
```

The path

Phase 1 — The host · Days 01–05 One machine, understood properly: boot, identity, processes, storage, logs.

Phase 2 — The network · Days 06–10 Five days of real kernel networking that cost no memory at all.

Phase 3 — Hardening and configuration management · Days 11–15 Lock a host down by hand, then make it repeatable.

Phase 4 — Production operations · Days 16–20 The things that turn a configured host into one you can rely on.

DAY	TITLE	RUNS ON	VERIFIED BY
01	systemd and the boot path	VM: control	lint + your lab
02	Users, sudo, permissions and ACLs	VM: node1	lint + your lab
03	Processes, signals, cgroups v2 and limits	VM: node1	lint + your lab
04	Storage: LVM, filesystems and mount units	VM: node1 + extra disk	lint + CI
05	Logs and time: journald, logrotate and chrony	VM: node1	lint + your lab
06	Interfaces, routing and building the namespace lab	Host: network namespaces	lint + CI
07	The DNS resolution path	Host: network namespaces	lint + CI
08	Running DNS: authoritative and recursive	Host: network namespaces	lint + CI
09	Packet-level debugging	Host: network namespaces	lint + CI
10	TLS on the wire and a private CA	Host: network namespaces	lint + CI
11	firewalld, and the nftables underneath it	VM: node1	lint + your lab
12	SSH hardening, bastions and fail2ban	VM: control + node1	lint + your lab
13	SELinux: contexts, booleans and denial triage	VM: node1	lint + your lab
14	Ansible fundamentals	control -> node1	lint + your lab
15	Ansible roles: your hardening baseline	control -> node1 + node2	lint + your lab
16	WireGuard: a private network between hosts	VM: control + node1	lint + your lab
17	Reverse proxy and TLS termination	VM: node1	lint + your lab
18	Bridges, VLANs and link aggregation	Host: network namespaces	lint + CI
19	Intrusion detection and audit alerting	VM: node1	lint + your lab
20	Backup, restore and the restore drill	VM: control + node1	lint + your lab

Full detail, with the reasoning for each day, is in [docs/curriculum.md](#). The complete state of the project — design decisions, what is verified, what is not, and what is left to do — is in [docs/HANDOFF.md](#).



Memory budget

Designed for an 8 GB laptop, measured rather than hoped.

DAYS	NEEDS	RAM
01	one VM (control)	~1 GB
02–05, 11–13, 17, 19	one VM (node1)	~2 GB
06–10, 18	no VM at all	0 MB
14, 16, 20	two VMs	~3.5 GB
15	three VMs	~5.5 GB

VM	RAM	ROLE
<code>control</code>	1536 MB	Where you sit. Ansible runs from here
<code>node1</code>	2048 MB	The machine you configure and break
<code>node2</code>	2048 MB	Starts clean. Only Day 15 needs it

Disks are thin qcow2 overlays on one shared base image, so three VMs cost barely more than one until you install packages. Budget about 12 GB of disk. Peak memory happens on Day 15 only.

No VM is given less than 1536 MB, which is the minimum Rocky 9 recommends and the number `virt-install` warns about. Smaller VMs boot happily and then get `dnf` killed by the kernel mid-transaction, which reads as `killed` and looks nothing like a memory problem. If Day 15's three VMs are too much for your machine, close the browser first - or bring up `control` and `node2`, apply the role, and start `node1` afterwards.

What a green tick means

Verification is split into three tiers, because pretending CI can prove everything is how you end up trusting a badge that proves nothing.

TIER	WHAT IT CHECKS	WHERE IT RUNS	DAYS
lint	<code>bash -n</code> , shellcheck	GitHub Actions	all 20
CI	the day executed for real	GitHub Actions	04, 06, 07, 08, 09, 10, 18
lab	the day executed for real	your VMs	01, 02, 03, 05, 11, 12, 13, 14, 15, 16, 17, 19, 20

GitHub runners are full Ubuntu VMs with `sudo`, so namespaces, DNS servers, `tcpdump`, `openssl`, VLANs and even LVM on a loopback file are genuinely real there — that is 7 of the 20 days.

The other 13 cannot be faked on a runner. Ubuntu has no SELinux, no firewall, and no second host to reach over SSH. Those days are verified by a script you run on your own lab:

```
./days/day13/verify.sh
```

```
Day 13 – SELinux: contexts, booleans and denial triage
-----
PASS  SELinux is enforcing
PASS  the web root carries a web content label
FAIL  a custom policy module is loaded
YOU   you fixed a denial by relabelling, not by disabling SELinux

6 passed, 1 failed, 2 for you to judge
```

Every day has one. `PASS / FAIL` come from real commands against real state; `YOU` items are judgement calls that no script can test and that never affect the exit status. Run it on the wrong machine and it reports `SKIP` with the reason, rather than lying in either direction.

The lab

One script owns the whole environment.

```

./lab/lab.sh check           # prerequisites, with install hints
./lab/lab.sh image          # download the Rocky 9 base image
./lab/lab.sh up [vm...]     # create VMs (default: control node1)
./lab/lab.sh status         # VMs, their IPs, and namespaces
./lab/lab.sh ssh <vm>       # Log in
./lab/lab.sh push <vm> [path] # copy days/ and lab/ to ~/lab on a VM
./lab/lab.sh add-disk <vm> [GB] # attach a blank disk (Day 04)
./lab/lab.sh down [vm...]   # delete VMs and their disks
sudo ./lab/lab.sh netns-up   # build the Days 06-10 network
sudo ./lab/lab.sh netns-status # show it and ping-test it
sudo ./lab/lab.sh netns-down # tear it down
./lab/lab.sh diagnose <vm>  # every clue about a VM that will not boot
./lab/lab.sh console <vm>   # attach to its console (--restart to watch it boot)
./lab/lab.sh destroy        # everything

```

Looking at a VM

Every VM gets a VNC screen bound to 127.0.0.1:

```
virsh vncdisplay control # e.g. 127.0.0.1:0 - open it in any VNC viewer
```

That is deliberate. A headless guest that boots but writes nothing to its serial port is impossible to diagnose, and on some hardware the absence of a display device stops these cloud images booting at all. Set `LAB_GRAPHICS=none` if you want strictly headless VMs.

The namespace network

```

client 10.10.0.2 ---- 10.10.0.1 [router] 10.10.1.1 ---- 10.10.1.2 resolver
                                10.10.2.1 ---- 10.10.2.2 auth

```

Real interfaces, real routing tables, real packets, real `tcpdump` captures — this is the same kernel machinery containers are built from. It is not a simulation, and it costs no memory.

Why Rocky Linux guests on an Ubuntu host

Because of Day 13. SELinux is RHEL-family; Ubuntu ships AppArmor instead. It also makes Day 11 `firewalld` -first and makes Ansible feel native. Your host distribution barely matters — KVM is in the Linux kernel already, so Ubuntu, Fedora, Debian and Arch hosts are all fine and `check` prints the right package names for each.

Layout

README.md	you are here
docs/curriculum.md	all 20 days, with the reasoning
docs/HANDOFF.md	complete project state, for a cold start
CONTRIBUTING.md	how to change this without breaking it
LICENSE	MIT
tests/cli.sh	repo self-test: no VM, no root, no network
.pre-commit-config.yaml	shellcheck, whitespace, large-file guard
lab/lab.sh	the whole environment, one script
lab/verify-lib.sh	assertion helpers for verify.sh
lab/ci-day.sh	runs one day in CI, honestly
lab/README.md	hardware, install, manual fallbacks
days/dayNN/README.md	the day
days/dayNN/verify.sh	did you finish it
days/dayNN/scripts/	your work goes here
.github/workflows/ci.yml	lint everything, execute what it can

License and contributing

MIT - see [LICENSE](#).

Fixes are welcome, especially anything that does not work on your distribution or your hardware. Read [CONTRIBUTING.md](#) first - it covers how a day is put together and what has to stay in step with what.

docs/curriculum.md

Twenty days, four phases of five. Each phase ends somewhere you could stop and still have gained something whole.

At a glance

DAY	TITLE	RUNS ON	MEMORY	VERIFIED BY
01	systemd and the boot path	VM: control	~1 GB	lint + your lab
02	Users, sudo, permissions and ACLs	VM: node1	~2 GB	lint + your lab
03	Processes, signals, cgroups v2 and limits	VM: node1	~2 GB	lint + your lab
04	Storage: LVM, filesystems and mount units	VM: node1 + extra disk	~2 GB	lint + CI
05	Logs and time: journald, logrotate and chrony	VM: node1	~2 GB	lint + your lab
06	Interfaces, routing and building the namespace lab	Host: network namespaces	0 MB	lint + CI
07	The DNS resolution path	Host: network namespaces	0 MB	lint + CI
08	Running DNS: authoritative and recursive	Host: network namespaces	0 MB	lint + CI
09	Packet-level debugging	Host: network namespaces	0 MB	lint + CI
10	TLS on the wire and a private CA	Host: network namespaces	0 MB	lint + CI
11	firewalld, and the nftables underneath it	VM: node1	~2 GB	lint + your lab
12	SSH hardening, bastions and fail2ban	VM: control + node1	~3.5 GB	lint + your lab
13	SELinux: contexts, booleans and denial triage	VM: node1	~2 GB	lint + your lab
14	Ansible fundamentals	control -> node1	~3.5 GB	lint + your lab
15	Ansible roles: your hardening baseline	control -> node1 + node2	~5.5 GB	lint + your lab
16	WireGuard: a private network between hosts	VM: control + node1	~3.5 GB	lint + your lab
17	Reverse proxy and TLS termination	VM: node1	~2 GB	lint + your lab
18	Bridges, VLANs and link aggregation	Host: network namespaces	0 MB	lint + CI
19	Intrusion detection and audit alerting	VM: node1	~2 GB	lint + your lab
20	Backup, restore and the restore drill	VM: control + node1	~3.5 GB	lint + your lab

Phase 1 — The host (Days 01–05)

One machine, understood properly: boot, identity, processes, storage, logs.

Day 01 — systemd and the boot path

Follow a Linux machine from power-on to a running service, and write a unit that survives a reboot.

Runs on VM: control. Every later day ends with "make it persist", and on this family of distros that always means a unit file. Start where the machine starts.

Day 02 — Users, sudo, permissions and ACLs

Give a service account exactly the access it needs and nothing else.

Runs on VM: node1. "It works when I run it as root" is where most security incidents begin. Least privilege is a habit you build with your hands.

Day 03 — Processes, signals, cgroups v2 and limits

Constrain a process so it cannot take the machine down with it.

Runs on VM: node1. Containers are cgroups plus namespaces. Meet the primitives directly and container behaviour stops being magic.

Day 04 — Storage: LVM, filesystems and mount units

Grow a filesystem while it is mounted, and mount it persistently.

Runs on VM: node1 + extra disk. Disk full at 3am is the most common page in operations. Growing storage without downtime is the fix, and it should be boring.

Day 05 — Logs and time: journald, logrotate and chrony

Make logs persistent, bounded, and correctly timestamped.

Runs on VM: node1. Every diagnosis in Days 6-20 is log reading. Unbounded logs fill the disk; wrong clocks make correlation across hosts impossible.

Phase 2 — The network (Days 06–10)

Five days of real kernel networking that cost no memory at all.

Day 06 — Interfaces, routing and building the namespace lab

Build a four-node routed network inside your kernel and prove packets cross it.

Runs on Host: network namespaces. This is real kernel networking, not a simulation: real interfaces, real routing tables, real packets. It is also the environment Days 7-10 and 18 run in, at zero memory cost.

Day 07 — The DNS resolution path

Trace one name lookup through every layer that can answer it.

Runs on Host: network namespaces. "It is always DNS" is a joke because it is usually true. Knowing which layer answered is the difference between a five minute fix and an afternoon.

Day 08 — Running DNS: authoritative and recursive

Serve your own zone, then resolve it recursively from another namespace.

Runs on Host: network namespaces. Reading DNS is one skill; owning a zone is another. Days 10, 12 and 17 all need names that resolve to your own machines.

Day 09 — Packet-level debugging

Prove where a packet stops, instead of guessing.

Runs on Host: network namespaces. When two hosts disagree about whether traffic arrived, only a capture settles it. This day is the one you will reuse most.

Day 10 — TLS on the wire and a private CA

Run your own certificate authority and understand what a client actually verifies.

Runs on Host: network namespaces. Certificate errors are the most common self-inflicted outage. Issuing certificates yourself makes the trust chain concrete, and Day 17 needs this CA.

Phase 3 — Hardening and configuration management (Days 11–15)

Lock a host down by hand, then make it repeatable.

Day 11 — firewalld, and the nftables underneath it

Close everything, open only what you need, and see the kernel rules your commands produced.

Runs on VM: node1. Most people learn the front end and never look underneath, so they cannot debug it when the front end lies. Look at both.

Day 12 — SSH hardening, bastions and fail2ban

Reach node1 only through control, with keys only, and ban brute force.

Runs on VM: control + node1. SSH is the door. It is also the service most often left with defaults that a scanner finds in minutes.

Day 13 — SELinux: contexts, booleans and denial triage

Serve content from a non-default path with SELinux still enforcing.

Runs on VM: node1. This is the day that separates operators from people who type `setenforce 0`. Denial triage is a procedure, and once you know it, SELinux stops being an obstacle.

Day 14 — Ansible fundamentals

Replace one day of manual work with a playbook that is safe to run twice.

Runs on control -> node1. You have now configured hosts by hand for thirteen days. This is the day that work becomes repeatable, and idempotency stops being a buzzword.

Day 15 — Ansible roles: your hardening baseline

Turn Days 11-13 into one role and apply it to a host that has never been touched.

Runs on control -> node1 + node2. A baseline you can apply to a fresh machine in one command is the whole point of configuration management. node2 is the proof, because it starts clean.

Phase 4 — Production operations (Days 16–20)

The things that turn a configured host into one you can rely on.

Day 16 — WireGuard: a private network between hosts

Build an encrypted tunnel between two machines and make it come back after reboot.

Runs on VM: control + node1. Every real environment has a management network you are not supposed to expose. WireGuard is small enough to understand completely, which is rare in VPNs.

Day 17 — Reverse proxy and TLS termination

Put nginx in front of a plain HTTP service and serve it over TLS with your own CA.

Runs on VM: node1. This is how almost every internal service is actually exposed. It also ties Day 10 to something running, and Day 13 will fight you here.

Day 18 — Bridges, VLANs and link aggregation

Segment one wire into isolated networks, then prove the isolation.

Runs on Host: network namespaces. Every hypervisor and every switch does this. It is also how you explain to yourself why two machines on the same cable cannot see each other.

Day 19 — Intrusion detection and audit alerting

Notice an attack and a file change without watching the screen.

Runs on VM: node1. Hardening stops the easy attempts. Detection is what tells you about the rest, and auditd is the tool that answers who changed this file.

Day 20 — Backup, restore and the restore drill

Prove you can get the data back, on a schedule, and know how long it takes.

Runs on VM: control + node1. An untested backup is a rumour. This is the last day because it is the one that decides whether everything before it was worth configuring.

Why there is no capstone

A capstone is a thing you build. Operations is not a building discipline — it is a diagnostic one. The exam for a developer is whether the thing works; the exam for an operator is whether you can find out why it does not.

So instead of a final project, the lab persists. Day 06 builds a network and Days 07–10 and 18 operate inside it. Days 11–13 harden a host by hand and Day 15 turns that work into an Ansible role applied to a machine that has never been touched. Day 20 restores the data the earlier days created. Nothing is thrown away, and nothing is rebuilt from a template.

lab/README.md

What you need

HOST OS LINUX (KVM LIVES IN YOUR KERNEL; NOTHING TO INSTALL FOR THE HYPERVISOR ITSELF)

CPU virtualization enabled in BIOS/UEFI — `vmx` for Intel, `svm` for AMD

RAM 8 GB works. 2 GB for node1, about 3.8 GB for all three

Disk ~12 GB: a 1 GB base image plus thin overlays

Packages `libvirt`, `virt-install`, `qemu` system emulation for x86

Run the check first. It reports exactly what is missing and the command to fix it for your distribution:

```
./lab/lab.sh check
```

One-time setup

```
# Fedora / Rocky / Alma
sudo dnf install -y libvirt virt-install qemu-kvm libvirt-daemon-config-network

# Debian / Ubuntu
sudo apt-get install -y qemu-system-x86 libvirt-daemon-system libvirt-clients libvirt-daemon-config-network virtinst acl

# There is no qemu-kvm package on Debian or Ubuntu any more, and plain
# qemu-system installs every architecture. qemu-system-x86 is the right one.

sudo systemctl enable --now libvirtd
sudo usermod -aG kvm,libvirt "$USER"      # then Log out and back in
```

One more step the first time, and only once:

```
sudo install -d -o "$(id -un)" -g "$(id -gn)" /var/lib/libvirt/images/bash-mastery-linux
```

The lab keeps its base image and VM disks there rather than under your home directory. That is not a style preference. Under `qemu:///system` the VM runs as the `libvirt-qemu` user, and on Ubuntu AppArmor confines that process to a fixed list of paths which does not include home directories, so a disk under `$HOME` is refused even when the file permissions are correct. `lab.sh` uses this directory automatically as soon as it exists and is writable by you.

That last step matters. Without it `/dev/kvm` is not writable and every VM creation fails with a permission error.

Daily use

```
./lab/lab.sh up control node1      # bring up what the day needs
./lab/lab.sh status                 # VMs, addresses, namespaces
./lab/lab.sh push node1             # copy days/ and lab/ into ~/Lab on the VM
./lab/lab.sh ssh node1
./lab/lab.sh add-disk node1 2       # Day 4 wants a spare disk
./lab/lab.sh down node1             # delete it; recreate in about a minute
./lab/lab.sh diagnose node1        # when a VM will not boot or get an address
```

Each VM has a VNC screen on `127.0.0.1`; `virsh vncdisplay node1` prints the display to open in a viewer. Guests created with no display device have been seen to spin forever on this hardware without writing anything to the serial console, so the screen is on by default. `LAB_GRAPHICS=none` turns it off.

VMs are disposable on purpose. If Day 12 locks you out of `sshd` or Day 13 leaves the filesystem mislabelled, delete and recreate rather than repair. The recovery skill is worth practising once, deliberately — not by accident at midnight.

The namespace topology

```
client 10.10.0.2 ---- 10.10.0.1 router 10.10.1.1 ---- 10.10.1.2 resolver
                                router 10.10.2.1 ---- 10.10.2.2 auth
```

```
sudo ./lab/lab.sh netns-up
sudo ./lab/lab.sh netns-status
sudo ip netns exec router tcpdump -ni veth-rau
sudo ./lab/lab.sh netns-down
```

These are real network stacks in your kernel. Real routing decisions, real packets, real captures. They are not a simulation, and they cost no memory — which is why Days 6–10 are the cheapest days in the curriculum.

If the script fails

It is a convenience wrapper, not a dependency. Everything it does is plain `libvirt`, and doing it by hand is a reasonable Day 0 exercise:

```
qemu-img create -f qcow2 -F qcow2 \
  -b ~/.local/share/bash-mastery-linux/images/Rocky-9-GenericCloud-Base.latest.x86_64.qcow2 \
  ~/.local/share/bash-mastery-linux/disks/node1.qcow2 10G

virt-install --name node1 --memory 2048 --vcpus 1 \
  --disk path=~/.local/share/bash-mastery-linux/disks/node1.qcow2,format=qcow2 --import \
  --os-variant rocky9 --network network=default --noautoconsole \
  --graphics vnc,listen=127.0.0.1 \
  --disk path=/path/to/seed.iso,device=cdrom,readonly=on --boot hd

virsh list --all
virsh domifaddr node1
virsh console node1      # escape with ctrl-]
```

If `--os-variant rocky9` is rejected, your `osinfo-db` is older than Rocky 9. Use `rhe19.0`, or `generic` as a last resort.

Known limits

- The `default` `libvirt` network is NAT. VMs reach the internet and each other, but nothing on your LAN reaches them without extra work.
- Namespaces share your host kernel, so they cannot host Days 1–5 or Day 13: no separate init, no separate disks, no separate SELinux state.
- `lab.sh` has not been executed against real hardware. Treat `check` as the first thing to run and the first thing to trust.

docs/curriculum.md#phase-1

PHASE 1

The host

Days 01–05

01	systemd and the boot path	15
02	Users, sudo, permissions and ACLs	20
03	Processes, signals, cgroups v2 and limits	25
04	Storage: LVM, filesystems and mount units	30
05	Logs and time: journald, logrotate and chrony	36

DAY 01 Day 01 — systemd and the boot path

days/day01/README.md

This day is provable, not descriptive: **4 automatic checks** and **1 judgement calls** in `verify.sh`.

Follow a Linux machine from power-on to a running service, and write a unit that survives a reboot.

PHASE	THE HOST
Runs on	VM: control
Memory	~1 GB (one VM)
Verified by	lint + your lab

Why this day exists

Every later day ends with "make it persist", and on this family of distros that always means a unit file. Start where the machine starts.

What you will work with

- `systemctl list-units --failed` - every unit that tried to start and did not. On a healthy machine this prints nothing, which is why it is the first thing to run when something is wrong.
- `systemctl cat sshd.service` - shows the real unit file systemd is using, including any drop-in overrides, so you read what is in effect rather than what you think you installed.
- `systemd-analyze blame` - each unit and how long it took to start, slowest first. Answers "why is this boot slow".
- `systemd-analyze critical-chain` - the dependency path that actually determined boot time. Slower than `blame` at first glance but more honest: a unit can be slow and still not hold anything up.
- `journalctl -b -p err` - errors only, from this boot only. The fastest way to see what the kernel and services complained about since power-on.
- `/etc/systemd/system/` - where your own units and overrides live. Anything here wins over the packaged units in `/usr/lib/systemd/system/`, which is the whole reason you edit here and never there.

Verify

Checked automatically:

- ☐ a custom unit is installed and enabled
- ☐ that unit is running
- ☐ the machine boots with no failed units
- ☐ the unit restarts itself after being killed

Only you can confirm:

- ☐ you can read `systemd-analyze critical-chain` and name the slowest unit

Run the automatic checks with:

```
./days/day01/verify.sh
```

Run it on control, not on your laptop. Nothing inside `verify.sh` checks which machine you are on, so on your laptop it runs anyway and reports `FAIL` for `lab-demo.service` — a service that was never meant to exist there. Red on the wrong machine means "wrong machine", not "wrong work". Watch the third check too: `the machine boots with no failed units` will report `PASS` about *your laptop's* boot, which proves nothing about this day. The only results that mean anything are the ones from a shell on the VM.

CI can only lint this day. Nothing on a GitHub runner has SELinux, firewalld, systemd units you control, or a second host to reach over SSH — so the checks below are proven by running `verify.sh` on your own lab, and nowhere else.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>lab-demo.sh</code>	The service payload. Long-running, prints to the journal, traps SIGTERM.	no
<code>setup.sh</code>	Installs the payload and writes, enables and starts <code>lab-demo.service</code> . Idempotent.	yes
<code>explore-boot.sh</code>	Read-only guided tour: boot timing, blame, critical chain, failed units, cgroup.	no
<code>break-and-fix.sh</code>	Kills the service to watch <code>Restart=always</code> work. <code>--hard</code> also breaks the unit file, then repairs it.	yes
<code>teardown.sh</code>	Removes everything <code>setup.sh</code> installed, and proves it is gone.	yes

Read them before you run them. They are commented as teaching material rather than production code — the comments are half the day.

Run it on the lab

1. On your laptop, bring up one VM

Run these one at a time, from the top of the repository. **Do not paste all four at once:** `check` is a gate, and the three after it are pointless until it is clean.

```
./lab/lab.sh check          # first time only: does this machine have KVM?
```

On a fresh Ubuntu machine `check` will report missing tools, because `qemu`, `libvirt` and `virt-install` are not installed by default. It prints the exact commands; they are also here:

```
sudo apt-get install -y qemu-system-x86 libvirt-daemon-system libvirt-clients libvirt-daemon-config-network virtinst acl
```

Then, once, so the hypervisor can reach the lab's disks:

```
sudo install -d -o "$(id -un)" -g "$(id -gn)" /var/lib/libvirt/images/bash-mastery-linux
```

- VM disks cannot live under your home directory on Ubuntu. AppArmor confines the `libvirt-qemu` process to a list of paths that excludes `/home`, so `qemu` is denied the disk even when its permissions are right. `lab.sh` picks this directory up automatically once it exists.

Two things about that line, because both bite people:

- **There is no `qemu-kvm` package on Debian or Ubuntu any more.** It is a virtual package with no installation candidate, so `apt` refuses and installs *nothing at all* - which is why a follow-up `systemctl`

`enable --now libvirtd` then says the unit does not exist. That is not a second fault, it is the same one. The real package is `qemu-system-x86`.

- **Do not use plain `qemu-system`**. The QEMU website suggests it, but that metapackage installs emulators for every CPU architecture - hundreds of megabytes you will never boot. KVM only runs the host architecture.

Now start the daemon. Recent libvirt splits `libvirtd` into modular daemons and may not ship `libvirtd.service` at all, so use whichever exists:

```
sudo systemctl enable --now libvirtd 2>/dev/null \  
|| sudo systemctl enable --now virtqemud.socket virtnetworkd.socket  
sudo usermod -aG kvm,libvirt "$USER"      # Log out and back in after this
```

If `check` then still says the `default` network is not defined:

```
sudo virsh net-start default && sudo virsh net-autostart default
```

Log out and back in, then run `check` again.

Only once `check` ends in `ready` do you continue:

```
./lab/lab.sh image          # first time only: ~900 MB download, cached  
./lab/lab.sh up control     # ~1 GB of RAM, about a minute  
./lab/lab.sh status        # wait until control has an IP address
```

The VM has a screen as well as a serial console. `virsh vncdisplay control` prints something like `127.0.0.1:0`; open that in any VNC viewer to watch it boot. If it never gets an address, `./lab/lab.sh diagnose control` collects everything worth knowing in one go.

2. Copy the repo onto the VM

Everything from here happens **on the VM**. The day installs a unit, kills processes and breaks a service on purpose; none of that belongs on your own machine. `setup.sh` and `break-and-fix.sh` now refuse to run anywhere that is not a lab VM, and your shell prompt showing `[lab@control ~]$` is the confirmation to look for.

```
./lab/lab.sh push control
```

That lands `days/` and `lab/` in `~/lab` on the VM. Both are needed: every `verify.sh` sources `lab/verify-lib.sh`, so pushing `days/` alone gives you a day that cannot check itself.

Re-run `push` whenever you edit anything on the host. It overwrites, so it is safe to run as often as you like.

3. Work through the day on the VM

```
./lab/lab.sh ssh control  
cd ~/lab/days/day01
```

Then, in this order:

```
less scripts/setup.sh      # 1. read it BEFORE running it  
sudo ./scripts/setup.sh    # 2. install the service
```

```
systemctl status lab-demo          # 3. Look at your own work
journalctl -u lab-demo -f          #   ctrl-c when you have seen enough
systemctl cat lab-demo
```

```
./scripts/explore-boot.sh          # 4. the tour. no root needed
```

Do not skim this one. It prints the command it is about to run before each block, and the reason you are looking at the output. Stop at anything you cannot explain and go read `man systemd.service` or `man journalctl` on the VM - both are installed.

```
sudo ./scripts/break-and-fix.sh    # 5. crash it, watch it come back
sudo ./scripts/break-and-fix.sh --hard # 6. now break the unit file itself
```

Step 6 is the important one. A crashed process and a broken unit file are two different failures with two different signatures, and the whole skill is telling them apart from `systemctl status` alone.

4. Check yourself

```
sudo ./verify.sh
echo "exit: $?"
```

Expected on a healthy Day 01: **4 PASS, 1 YOU, exit 0**. The `you` line is the critical-chain question - nothing can grade that but you.

Root is needed because the checks read `/etc/systemd/system/` and query unit state. Without it the run exits 0 with everything skipped, which is not a pass.

IF YOU SEE	IT MEANS	DO THIS
FAIL a custom unit is installed and enabled	<code>setup.sh</code> did not finish, or you ran it in the wrong directory	<code>cd ~/lab/days/day01 && sudo ./scripts/setup.sh</code>
FAIL that unit is running	it started and then died	<code>journalctl -u lab-demo -n 30</code>
FAIL the machine boots with no failed units	something unrelated is broken on the VM - a genuine finding, not a bug in the day	<code>systemctl list-units --failed</code> , then fix it and write down what it was
SKIP missing root	you forgot <code>sudo</code>	run it again with <code>sudo</code>

5. Prove it survives a reboot

This is the actual objective of the day, and no script can do it for you. **Reboot the VM, not your laptop**. Run this while logged in over `./lab/lab.sh ssh control`, so the shell you type it into belongs to the VM:

```
hostname          # must print: control
sudo reboot       # your ssh session will drop - that is the point
```

If `hostname` prints your laptop name instead, you are in the wrong shell and `sudo reboot` would restart your own machine. On a desktop it will usually refuse with `Operation inhibited by ... user session inhibited`, which is a useful accident: it is `systemd-inhibit` protecting a logged-in graphical session, and it is the clearest sign you are on the wrong host.

You can also power-cycle it from the laptop without logging in:

```
virsh reboot control
```

Then from the host, once it is back (about 20 seconds):

```
./lab/lab.sh ssh control
systemctl is-active lab-demo      # active
systemctl is-enabled lab-demo     # enabled
systemd-analyze critical-chain    # does your service appear? why not?
```

If `is-enabled` says `enabled` but `is-active` says `inactive`, you have found the difference between the two words the hard way - which is the best way.

6. Optional cleanup

```
sudo ./scripts/teardown.sh
```

You do not have to. Nothing in Days 02-20 conflicts with `lab-demo`, and leaving it running gives you a harmless service to practise on later. Run the teardown if you want to do the whole install-inspect-remove loop again from scratch.

To give the RAM back to your laptop when you are done for the day:

```
./lab/lab.sh down control      # deletes the VM and its disk
```

The base image stays cached, so `up` next time takes a minute, not a download.

Notes

Keep your own notes here. What broke, what the error actually said, and what fixed it — that is the part you will come back for.

Next up: **Day 02 — Users, sudo, permissions and ACLs.**

DAY 02 Day 02 — Users, sudo, permissions and ACLs

days/day02/README.md

This day is provable, not descriptive: **5 automatic checks** and **1 judgement calls** in `verify.sh`.

Give a service account exactly the access it needs and nothing else.

PHASE	THE HOST
Runs on	VM: node1
Memory	~2 GB (one VM)
Verified by	lint + your lab

Why this day exists

"It works when I run it as root" is where most security incidents begin. Least privilege is a habit you build with your hands.

Today builds one small, complete example of it: a service account with no shell, one directory it shares with humans, one command it may run as root, and a running service that breaks the moment any of that is wrong. Then you break each piece on purpose and read the failure.

What you will work with

- `useradd --system --shell /sbin/nologin` - a service account. The system UID is a convention that tells a human "not a person"; the `nologin` shell is the part that actually stops a login, and it is why `su - appsvc` fails while `sudo -u appsvc <cmd>` works.
- `visudo -cf <file>` - parses a sudoers file without installing it. A syntax error under `/etc/sudoers.d/` makes `sudo` refuse to run *at all*, on a machine where `sudo` is how you become root. Build, check, install - never edit in place.
- `sudo -l -U <user>` - what sudo will really allow, aliases expanded. This is the authority, not the file you think you wrote. Needs root to ask about someone else.
- **setgid directories** (`chmod 2770`) - files created inside inherit the *directory's* group instead of the creating user's primary group. Without it, two members of the same group create files each other cannot edit, which is the most common cause of "but we are both in the group". Nothing fails on the day you lose the bit; it fails later.
- `setfacl` / `getfacl` - Unix modes have exactly three slots: owner, group, everyone. There is no fourth slot for "and also this one account". An ACL is that fourth slot, scoped to the object rather than granted machine-wide like a group. `setfacl -d` sets a *default* ACL, which is not an access rule but a template inherited by new files.
- `umask` - the permission bits removed from anything you create. Files start at 666 and directories at 777, which is why a 022 umask gives 644 and 755.
- `ls -ld` versus `getfacl` - a `+` on the end of the mode is the *only* hint `ls` gives that an ACL exists. On a machine that uses ACLs, `ls -l` is not a complete answer and never says so.

Verify

Checked automatically:

- ☐ a system account `appsvc` exists with no login shell
- ☐ `appsvc` may restart one service and nothing else
- ☐ `appsvc` cannot become root
- ☐ the shared directory is `setgid`
- ☐ an ACL grants `appsvc` access without changing the owner

Only you can confirm:

- ☐ you can explain every line of `sudo -l -U appsvc`

Run the automatic checks with:

```
sudo ./days/day02/verify.sh
```

Root is needed: asking `sudo` about another user's privileges is itself privileged, and the ACL check reads a `2770` directory. Without it every check reports `SKIP`, which is not a pass.

Run it on node1, not on your laptop. Nothing inside `verify.sh` checks which machine you are on. Without `sudo` you get the honest `SKIP` list and the line `This day runs elsewhere`, but that is only because this day needs root — run it *with* `sudo` on your laptop and it will report `FAIL` for an `appsvc` account that was never meant to exist there. Red on the wrong machine means "wrong machine", not "wrong work". The only results that mean anything are the ones from a shell on the VM.

CI can only lint this day. Nothing on a GitHub runner has SELinux, `firewalld`, `systemd` units you control, or a second host to reach over SSH — so the checks below are proven by running `verify.sh` on your own lab, and nowhere else.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>lab-app.sh</code>	The service payload. Runs as <code>appsvc</code> and appends to <code>/srv/shared/lab-app.log</code> , so a permission mistake makes it die rather than lie.	no
<code>setup.sh</code>	Creates the <code>appdata</code> group, the <code>appsvc</code> account, <code>/srv/shared</code> (<code>setgid</code> + ACLs), the one-command <code>sudo</code> rule, and <code>lab-app.service</code> . Idempotent.	yes
<code>explore-perms.sh</code>	Read-only guided tour: identity, the account, <code>sudo -l</code> , the three mechanisms stacked on one directory, <code>setgid</code> , <code>umask</code> .	no
<code>break-and-fix.sh</code>	Three permission failures, each repaired: a refused <code>sudo</code> command, a removed ACL, a removed <code>setgid</code> bit. <code>--hard</code> also breaks <code>sudo</code> itself and recovers.	yes
<code>teardown.sh</code>	Removes the user, group, directory, <code>sudo</code> rule and unit, then proves each is gone.	yes

Read them before you run them. They are commented as teaching material rather than production code — the comments are half the day.

Run it on the lab

Day 02 runs on `node1`, not `control`. Day 01 was `control`; from here on the day page names the machine and you should check `hostname` before typing anything that changes state. Every script that changes state refuses to run anywhere that is not a lab VM, but it cannot tell you that you are on the *wrong* lab VM.

1. On your laptop, bring up node1

From the top of the repository:

```
./lab/lab.sh status           # what is already running?
./lab/lab.sh up node1        # 2 GB, about a minute
./lab/lab.sh status          # wait until node1 has an IP address
```

If you still have `control` up from Day 01 and your laptop is tight on memory, give it back first — nothing today needs it:

```
./lab/lab.sh down control
```

If `node1` never gets an address, `./lab/lab.sh diagnose node1` collects everything worth knowing in one go, and `virsh vncdisplay node1` gives you a screen to watch it boot.

2. Copy the repo onto the VM

```
./lab/lab.sh push node1
```

That lands `days/` and `lab/` in `~/lab` on the VM. Both are needed: `verify.sh` sources `lab/verify-lib.sh` and the day scripts source `lab/on-lab-vm.sh`, so pushing `days/` alone gives you a day that can neither guard nor check itself.

Re-run `push` whenever you edit anything on the host. It overwrites, so run it as often as you like.

3. Work through the day on the VM

```
./lab/lab.sh ssh node1
hostname                     # must print: node1
cd ~/lab/days/day02
```

Then, in this order:

```
less scripts/setup.sh       # 1. read it BEFORE running it
sudo ./scripts/setup.sh     # 2. build the least-privilege setup
```

It prints each thing it creates and refuses to finish if the service did not manage to write its log — because a service that starts and then cannot write is exactly the failure this day is about.

```
sudo -l -U appsvc           # 3. Look at your own work
getfacl -p /srv/shared      # note the trailing '+'
ls -ld /srv/shared          #
journalctl -u lab-app -f    # ctrl-c when you have seen enough
```

```
sudo ./scripts/explore-perms.sh # 4. the tour
```

It works without root but two of its commands need it, so run it with `sudo` the first time and read every block. Stop at anything you cannot explain and read `man 5 sudoers`, `man setfacl` or `man 7 acl` on the VM — all three are installed.

```
sudo ./scripts/break-and-fix.sh # 5. three failures, three fixes
sudo ./scripts/break-and-fix.sh --hard # 6. now break sudo itself
```

Step 5 is the one to slow down on. Removing the ACL produces a service that fails with `Permission denied` while `ls -l` shows a directory whose owner, group and mode are all correct — the failure with no visible cause. Step 6 is the one that stops you locking yourself out of a real machine one day: it shows a *valid* sudoers file being silently ignored because its mode is `0644` instead of `0440`.

4. Check yourself

```
sudo ./verify.sh
echo "exit: $?"
```

Expected on a healthy Day 02: **5 PASS, 1 YOU, exit 0**. The **YOU** line is the `sudo -l -U appsvc` question — read the output line by line and say what each permits. If you cannot, you have a rule you do not understand on a machine you administer.

IF YOU SEE	IT MEANS	DO THIS
SKIP on everything	no root, or <code>acl</code> is not installed	<code>sudo ./verify.sh</code> ; if it still skips, <code>sudo dnf install -y acl</code>
FAIL a system account <code>appsvc</code> exists...	<code>setup.sh</code> did not finish	<code>cd ~/lab/days/day02 && sudo ./scripts/setup.sh</code>
FAIL <code>appsvc</code> may restart one service...	the sudo rule is missing or being ignored	<code>visudo -cf /etc/sudoers.d/appsvc</code> , then <code>ls -l</code> it — it must be <code>0440 root:root</code>
FAIL <code>appsvc</code> cannot become root	a rule somewhere grants <code>(ALL)</code> <code>ALL</code>	<code>sudo -l -U appsvc</code> , then find which file: <code>grep -r appsvc /etc/sudoers /etc/sudoers.d/</code>
FAIL the shared directory is <code>setgid</code>	you ran <code>break-and-fix.sh</code> and it was interrupted before the repair	<code>sudo chmod 2770 /srv/shared</code>
FAIL an ACL grants <code>appsvc</code> access...	same, at the ACL step	<code>sudo setfacl -m u:appsvc:rwX /srv/shared</code>

5. Prove it holds across a reboot

Users, groups, modes and ACLs live on disk, so they survive by nature — but the *service* running as an unprivileged user is the part that tends to break, because a unit that works when you start it by hand can still fail at boot.

```
hostname           # must print: node1
sudo reboot        # your ssh session will drop - that is the point
```

If `hostname` prints your laptop's name, you are in the wrong shell and this would restart your own machine. You can also power-cycle it from the laptop: `virsh reboot node1`.

Then, once it is back:

```
./lab/lab.sh ssh node1
systemctl is-active lab-app      # active
tail -3 /srv/shared/lab-app.log  # timestamps from after the reboot
ls -l /srv/shared                # group is still appdata
getfacl -p /srv/shared           # the ACL is still there
id -Gn                           # NOW your shell has appdata
```

That last one is the point worth having: `setup.sh` added your account to `appdata` , but supplementary groups are read at login, so the shell you ran it in never had it. A reboot is one way to notice. `newgrp appdata` is the

other.

6. Optional cleanup

```
sudo ./scripts/teardown.sh
```

You do not have to. Nothing in Days 03-20 conflicts with `appsvc`, `/srv/shared` or `lab-app`, and leaving them gives you a real service account to practise on — Day 03 constrains a process, and an unprivileged one is a better subject than root.

To give the RAM back to your laptop when you are done for the day:

```
./lab/lab.sh down node1          # deletes the VM and its disk
```

The base image stays cached, so `up` next time takes a minute, not a download.

Notes

Keep your own notes here. What broke, what the error actually said, and what fixed it — that is the part you will come back for.

Next up: **Day 03 — Processes, signals, cgroups v2 and limits.**

DAY 03 Day 03 — Processes, signals, cgroups v2 and limits

days/day03/README.md

This day is provable, not descriptive: **4 automatic checks** and **2 judgement calls** in `verify.sh`.

Constrain a process so it cannot take the machine down with it.

PHASE	THE HOST
Runs on	VM: node1
Memory	~2 GB (one VM)
Verified by	lint + your lab

Why this day exists

Containers are cgroups plus namespaces. Meet the primitives directly and container behaviour stops being magic.

Today builds one service that misbehaves on request — it eats memory as fast as bash can allocate it, or burns a whole core — and then holds it to 100 MB and a fifth of one core. Same program, two very different neighbours. Then you hit each cap on purpose and read what the machine says, which is much less than you would expect.

What you will work with

- `ps -eo pid,ppid,stat,cmd` - the process table with the columns that matter. `STAT` is the one people skip and then guess about: `S` sleeping, `R` running, `D` uninterruptible sleep, `Z` zombie, `T` stopped. A pile of `D` means storage or network is not answering, and no amount of `kill` will help — `D` cannot be interrupted, which is what the name means.
- `kill -TERM` versus `kill -KILL`, and `trap` - `kill` sends a signal, it does not kill. `TERM` asks a process to stop and can be trapped, so the process closes its files and releases its locks first. `KILL` (9) and `STOP` (19) are handled by the kernel and the process is never told, so there is nothing to trap and no cleanup. That is why `kill -9` is not a fix, and why an OOM kill leaves a log ending mid-sentence.
- `/sys/fs/cgroup/` - the unified hierarchy, as files. `memory.max` and `cpu.max` are the caps as the *kernel* stores them, `memory.current` is live usage, and `memory.events` counts `oom_kill`. This is the authority: `systemctl show` tells you what systemd asked for, and these files tell you what is being enforced. When the two disagree, the setting was accepted and silently dropped.
- `systemd-run --scope -p MemoryMax=` - a throwaway cgroup around a command, with no unit file and no `daemon-reload`. The fastest way to find out how much memory something actually needs before you write a cap into a unit, and the fastest way to see a cap kill something you started yourself.
- `systemctl set-property` - changes a cap on a **running** service, immediately, by writing the kernel file directly. The trap is where it writes the persistent copy: `/etc/systemd/system.control/`, not your unit file. After using it, reading the unit file alone will mislead you; `systemctl cat` shows both.
- `ulimit` and `/etc/security/limits.d/` - the *other* limit system, and the one constantly confused with the first. These are per **login session**, applied by `pam_limits` when a user logs in, and inherited by whatever that shell starts. Systemd does not log anybody in to start a service, so `limits.d` has no effect on

daemons at all — a unit takes its own `LimitNOFILE=`. That single fact is the most expensive misunderstanding in this day.

Verify

Checked automatically:

- ☐ `cgroups v2` is the unified hierarchy
- ☐ a service is capped with `MemoryMax`
- ☐ the same service is capped with `CPUQuota`
- ☐ a `nofile` limit is raised for one account only

Only you can confirm:

- ☐ you triggered the memory cap and found the kill in the journal
- ☐ you can explain why `SIGKILL` cannot be trapped

Run the automatic checks with:

```
./days/day03/verify.sh
```

No root needed today, unlike Day 02: every check reads state that is world-readable — a mount table, two `systemctl show` properties, and a file in `/etc/security/limits.d/`.

Run it on node1, not on your laptop. Nothing inside `verify.sh` checks which machine you are on, so on your laptop it runs anyway and reports `FAIL` for `lab-cap.service` — a service that was never meant to exist there. Red on the wrong machine means "wrong machine", not "wrong work". Worse, `cgroups v2 is the unified hierarchy` reads nothing but the mount table, so it will report `PASS` on any modern Linux: a green line for work you never did. The only results that mean anything are the ones from a shell on the VM.

CI can only lint this day. Nothing on a GitHub runner has SELinux, `firewalld`, `systemd` units you control, or a second host to reach over SSH — so the checks below are proven by running `verify.sh` on your own lab, and nowhere else.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>lab-cap.sh</code>	The service payload. Ticks quietly into the journal and traps <code>SIGTERM</code> , or on request eats 10 MB a second, or burns a core.	no
<code>setup.sh</code>	Installs the payload and <code>lab-cap.service</code> with <code>MemoryMax=100M</code> and <code>CPUQuota=20%</code> , then the <code>labworker</code> account and its own <code>nofile</code> limit. Idempotent.	yes
<code>explore-procs.sh</code>	Read-only guided tour: process states, the tree, signals, the unified hierarchy, this day's cgroup from both sides, and both limit systems. Runs without root; two blocks show more with it and say so.	no
<code>break-and-fix.sh</code>	Three resource failures, each repaired: an OOM kill, silent CPU throttling, and <code>SIGKILL</code> against a trap. <code>--hard</code> adds a limit that is set correctly and ignored, and a cap so low the service cannot start.	yes
<code>teardown.sh</code>	Removes the unit, both override directories, the payload, the limit file and the account, then proves each is gone.	yes

Read them before you run them. They are commented as teaching material rather than production code — the comments are half the day.

Run it on the lab

1. On your laptop, bring up node1

Day 03 runs on `node1`, the same VM as Day 02. Nothing today conflicts with Day 02, so you can leave `appsvc` and `/srv/shared` in place.

```
./lab/lab.sh status          # what is already running?
./lab/lab.sh up node1        # 2 GB, about a minute
```

If `node1` is already up from yesterday, `status` will say so and `up` is a no-op.

2. Copy the repo onto the VM

```
./lab/lab.sh push node1      # carries both days/ and lab/
./lab/lab.sh ssh node1
```

Both directories have to travel: the day scripts source `lab/on-lab-vm.sh`, and without it every one of them refuses to run.

3. Work through the day on the VM

As always the day page names the machine. Check it before typing anything that changes state:

```
hostname                    # must print: node1
cd ~/lab/days/day03
```

In this order:

```
less scripts/setup.sh      # 1. read it BEFORE running it
sudo ./scripts/setup.sh    # 2. build the capped service
```

It refuses to continue on a machine that is not on the unified hierarchy, and it prints the caps twice — once as systemd reports them, once as the kernel stores them. Those two lists must agree. When they do not, systemd accepted a setting the kernel is not enforcing, and reading both is the habit worth taking from today.

```
systemctl cat lab-cap      # 3. Look at your own work
systemctl show lab-cap -p MemoryMax -p CPUQuotaPerSecUSec
cat /sys/fs/cgroup/system.slice/lab-cap.service/cpu.max
journalctl -u lab-cap -f    # ctrl-c when you have seen enough
```

`cpu.max` prints `20000 100000`: 20 ms of CPU per 100 ms, which is the 20% you asked for, expressed the way the kernel stores it.

```
sudo ./scripts/explore-procs.sh # 4. the tour
```

It works without root, but two blocks need it, so run it with `sudo` the first time and read every section. Stop at anything you cannot explain and read `man 5 systemd.resource-control`, `man 7 signal` OR `man 5 limits.conf` on the VM — all three are installed.

```
sudo ./scripts/break-and-fix.sh # 5. three failures, three fixes
```

This one takes about a minute of waiting, on purpose. It watches `memory.current` climb second by second until the kernel kills the process, then shows you that the journal ends mid-sentence with no shutdown line. It

cannot print one — an OOM kill inside a cgroup is a `SIGKILL`, so the `trap` in `lab-cap.sh` is never reached. The receipt is the `oom_kill` counter in `memory.events`, not the log.

The second failure is the one worth slowing down for. A throttled service reports `active (running)`, logs nothing, fails nothing, and does a fifth of the work. The only place that is visible is `cpu.stat`.

```
sudo ./scripts/break-and-fix.sh --hard # 6. two harder ones
```

The `--hard` pair are the mistakes that cost the most time:

- a `nofile` limit set correctly in `/etc/security/limits.d/`, verified with `su - labworker; ulimit -n`, and completely ignored by the service — because `pam_limits` runs at **login** and `systemd` never logs anybody in. The setting that applies to a daemon is the unit's `LimitNOFILE=`.
- a cap so low the service cannot start at all, where the error says *start request repeated too quickly* and never mentions memory.

Every override the script makes goes into a drop-in and is removed again, so your unit file is left exactly as `setup.sh` wrote it. Confirm that yourself:

```
systemctl cat lab-cap # unit only, no drop-ins
ls /etc/systemd/system/lab-cap.service.d/ 2>/dev/null || echo none
ls /etc/systemd/system.control/ 2>/dev/null || echo none
```

4. Check yourself

```
./verify.sh
echo "exit: $?"
```

Expected on a healthy Day 03: **4 PASS, 2 YOU, exit 0**. The two `you` lines are what this day is actually for — you have seen an OOM kill you caused, and you can say why no amount of trapping would have caught it.

Two things to look at before you move on, because neither is on any dashboard:

```
systemctl show lab-cap -p NRestarts
cat /sys/fs/cgroup/system.slice/lab-cap.service/cpu.stat
```

`Restart=on-failure` makes a service that is being killed every ten seconds read as `active (running)`.

`NRestarts` is where the truth is, and `throttled_usec` is where a latency complaint with no error in it comes from.

5. Prove it holds across a reboot

This matters more today than on either of the previous two days, because Day 03 sets limits through three different mechanisms and they do not all persist the same way. One is a unit file, one is a `limits.d` file that is only read at login, and one — if you used `systemctl set-property` — is a drop-in under a directory you never wrote to by hand.

Reboot the VM, not your laptop. Run this while logged in over `./lab/lab.sh ssh node1`, so the shell belongs to the VM:

```
hostname # must print: node1
sudo reboot # your ssh session will drop - that is the point
```

If `hostname` prints your laptop's name, you are in the wrong shell and this would restart your own machine. You can also power-cycle it from the laptop: `virsh reboot node1`.

Then, once it is back:

```
./lab/lab.sh ssh node1
systemctl is-enabled lab-cap           # enabled
systemctl is-active lab-cap           # active
systemctl show lab-cap -p MemoryMax -p CPUQuotaPerSecUSec # still capped
cat /sys/fs/cgroup/system.slice/lab-cap.service/memory.max # kernel agrees
su - labworker -c 'ulimit -n'         # still raised
systemctl show lab-cap -p NRestarts   # back to 0
```

Three things worth noticing in that output:

- The cgroup at `/sys/fs/cgroup/.../memory.max` is a **new** one. Cgroups are kernel state and did not survive the reboot — systemd recreated it from the unit file. That is the difference between a limit that is *configured* and a limit that is *in effect*, and a reboot is the cleanest way to see it.
- `NRestarts` is back to `0`. If you were using it to spot a service being killed in a loop, a reboot erases your evidence. The journal does not — `journalctl -u lab-cap -b -1` reads the *previous* boot.
- `ulimit -n` for `labworker` still shows the raised value, because `limits.d` is on disk and `su -` is a login. The service's own limit is still `LimitNOFILE` from the unit, and still a different number. Same reboot, same file, two answers — which is the `--hard` lesson again, from the other side.

6. Optional cleanup

```
sudo ./scripts/teardown.sh
```

You do not have to. Nothing in Days 04-20 conflicts with `lab-cap` or `labworker`. Run it once anyway, so that "create, inspect, remove, prove it is gone" is a complete loop — and because two of the things it removes are easy to leave behind: a drop-in that survives deleting the unit file, and a file in `limits.d` that applies to every future login of that account.

To give the RAM back to your laptop when you are done for the day:

```
./lab/lab.sh down node1           # deletes the VM and its disk
```

The base image stays cached, so `up` next time takes a minute, not a download.

Notes

Keep your own notes here. What broke, what the error actually said, and what fixed it — that is the part you will come back for.

Next up: **Day 04 — Storage: LVM, filesystems and mount units.**

Day 04 — Storage: LVM, filesystems and mount units

days/day04/README.md

This day is provable, not descriptive: **5 automatic checks** and **2 judgement calls** in `verify.sh`.

Grow a filesystem while it is mounted, and mount it persistently.

PHASE	THE HOST
Runs on	VM: node1 + extra disk
Memory	~2 GB (one VM)
Verified by	lint + CI

Why this day exists

Disk full at 3am is the most common page in operations.

The fix is almost never "delete something". It is either a volume that can be grown without downtime, or a file that is already deleted and still costing you every block it ever used. Today you build the first, and then create the second on purpose so you recognise it the next time `df` and `du` disagree by 200 MB and nobody can find the files.

The volume starts at 512 MB deliberately. Storage you cannot grow while it is in use is storage that will eventually cost you an outage window.

What you will work with

- `lsblk` and `blkid` - the two questions you ask a machine you have never seen before. `lsblk` draws the tree: which disks exist, which are partitioned, which carry logical volumes, what is mounted where. `blkid` reads the signature at the very start of a device and tells you what filesystem is there. A device with no `blkid` output is blank, and that is the only safe thing to hand to LVM — every destructive storage mistake starts with being wrong about which device was empty.
- `pvccreate`, `vgcreate`, `lvcreate` - the three LVM layers, in order, and people blur them constantly. A **physical volume** is a disk given to LVM. A **volume group** is one or more PVs pooled together. A **logical volume** is a slice of that pool, and it is the only one of the three you ever put a filesystem on. `pvs`, `vgs` and `lvs` report on each layer, and `vgs` has the number that matters most: `vfree`, the headroom you actually have.
- `lvextend` plus `resize2fs` / `xfs_growfs` - **two commands, always**. `lvextend` makes the container bigger; the filesystem inside it does not notice and will not find out on its own. Run only the first and `df` reports the old size forever, which reads convincingly as "lvextend did nothing". Note the difference in what they take: `resize2fs` wants the **device**, `xfs_growfs` wants the **mount point**.
- `mkfs.ext4` versus `mkfs.xfs` - both grow while mounted, which is the property that matters today. Only ext4 can shrink, and only while unmounted; xfs can never shrink at all. Rocky defaults to xfs, so on a real RHEL-family machine "make it smaller" means a new volume and a copy. This day uses ext4 so you can watch a shrink be refused for a reason worth understanding.

- `/etc/fstab` versus `.mount` units - the same job, two contracts. `fstab` is the old one, and `systemd` silently generates units from it at boot anyway (`systemd-fstab-generator`). Writing the unit yourself gets you `After=`, `Requires=` and `WantedBy=`. The catch is that a `.mount` unit's **filename is derived from the path** and is not a free choice: `/srv/data` must be `srv-data.mount`. Name it anything else and `systemd` ignores the file completely, with no error naming the cause.
- `df -h` versus `du -sh` - they answer different questions and are only equal by coincidence. `df` asks the filesystem how many blocks are spent. `du` walks the directory tree and adds up files that still have a name. When they disagree, `df` is right and `du` is not lying — there are blocks in use belonging to files with no name left.
- `ls -l +L1` - the tool that finds those files. `+L1` means "link count below 1": deleted, but still open by a running process, so the kernel cannot release the blocks. The fix is never `rm`, because there is nothing left to remove — you restart or signal whatever holds the descriptor. This is why the advice for a log filling a disk is `: > file` rather than `rm file`.

Verify

Checked automatically:

- ☐ the volume group `labvg` exists
- ☐ a logical volume `labdata` exists in it
- ☐ it is mounted at `/srv/data`
- ☐ the mount is persistent
- ☐ the filesystem fills the logical volume after extending

Only you can confirm:

- ☐ you extended it live, with no unmount, and watched `df` change
- ☐ you found a deleted-but-still-open file with `ls -l +L1`

Run the automatic checks with:

```
sudo ./days/day04/verify.sh
```

Root is required today, as it was on Day 02. `vgs` and `lvs` refuse to report on volume groups for an unprivileged user, so without root the first two checks fail for want of permission while the two mount checks pass — `/proc/mounts` is world-readable. That combination reads convincingly as "my LVM is broken" when nothing is broken at all, so `verify.sh` requires root and prints `SKIP` for everything instead. `SKIP` is not a pass.

Run it on node1, not on your laptop. Nothing inside `verify.sh` checks which machine you are on, so on your laptop it runs anyway and reports `FAIL` for a volume group that was never meant to exist there. Red on the wrong machine means "wrong machine", not "wrong work". Be careful with the last check in particular: `the filesystem fills the logical volume after extending` only asks whether `df` prints a number for `/srv/data`, so if you happen to have any `/srv/data` at all it will report `PASS` without ever looking at LVM. It is the weakest check in this day, and it is listed as a known weakness in `docs/HANDOFF.md` rather than quietly presented as proof.

CI runs this day for real. Unlike Days 01–03, a GitHub runner can build this environment: LVM, a filesystem and a mount unit need no hardware, no second host and no SELinux. So `lab/ci-day.sh 04` executes

`scripts/setup.sh` and then `verify.sh` on every push. The runner has no spare disk, so it gets the loop-device path described below — which is the same LVM, on a block device the kernel builds out of a file.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>lab-writer.sh</code>	The payload. Fills a directory with real (non-sparse) files, or deletes a file while holding it open so you can hunt it with <code>lsdf</code> . Installed as <code>/usr/local/bin/lab-writer</code> .	no
<code>setup.sh</code>	Finds a spare disk or falls back to a loop device, then builds <code>labvg</code> , a 512 MB <code>labdata</code> , an ext4 filesystem, and <code>srv-data.mount</code> . Idempotent, and never reformats an existing filesystem.	yes
<code>explore-storage.sh</code>	Read-only guided tour: block devices, the three LVM layers, <code>df</code> against <code>du</code> , the mount described three ways, and the room left to grow. Runs without root; two blocks show more with it and say so.	no
<code>break-and-fix.sh</code>	Three storage failures, each repaired: a volume grown without its filesystem, a mount unit systemd ignores, and a full disk whose files cannot be found. <code>--hard</code> adds an exhausted volume group and a refused shrink.	yes
<code>teardown.sh</code>	Unmounts, removes the unit, the LV, the VG and the PV in the order that works, detaches the loop device, and proves each is gone.	yes

Read them before you run them. They are commented as teaching material rather than production code — the comments are half the day.

Run it on the lab

1. On your laptop, bring up node1

Day 04 runs on `node1`, the same VM as Days 02 and 03, and it needs something Days 02 and 03 did not: a second disk.

```
./lab/lab.sh status           # what is already running?
./lab/lab.sh up node1        # 2 GB, about a minute
./lab/lab.sh add-disk node1 2 # a blank 2 GB disk, appears as /dev/vdb
```

`add-disk` attaches the disk persistently, so it is still there after a reboot — which step 5 depends on. Nothing today conflicts with Days 02 or 03, so `appsvc` and `lab-cap` can stay.

If you skip `add-disk`, `setup.sh` still works: it falls back to a loop device built from a sparse file, and says loudly that it did. That is real LVM on a real block device, but a loop device does not survive a reboot, so **step 5 cannot be done that way**. Use the real disk if you can.

2. Copy the repo onto the VM

```
./lab/lab.sh push node1      # carries both days/ and lab/
./lab/lab.sh ssh node1
```

Both directories have to travel: the day scripts source `lab/on-lab-vm.sh`, and without it every one of them refuses to run.

3. Work through the day on the VM

As always the day page names the machine. Check it before typing anything that changes state — and today that matters more than on any previous day, because these scripts write filesystem signatures:

```
hostname                # must print: node1
cd ~/lab/days/day04
```

The Rocky 9 cloud image is minimal and has none of today's tools, so install them first:

```
sudo dnf install -y lvm2 e2fsprogs lsof
```

`lvm2` gives you `pvs`, `vgs`, `lvs` and `lvextend`; `e2fsprogs` gives `mkfs.ext4` and `resize2fs`; `lsof` is needed for the second manual check. `setup.sh` checks for all of them before it touches a disk and stops with this same command if any are missing — it will not create a volume group it cannot then put a filesystem on.

If you recreated the VM since last time, note what `./lab/lab.sh down node1` took with it: the VM's own disk **and** the extra disk from `add-disk`. So a rebuilt `node1` has no `lvm2`, no `/dev/vdb`, no `labvg` and no `/srv/data` — `pvs`, `vgs` and `lvs` print nothing at all, and `systemctl cat srv-data.mount` reports no such unit. That is a blank machine, not a broken one. Install the packages above, run `add-disk` again from your laptop, and start this step from the top.

Find out what you are working with before you build anything:

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS # /dev/vdb, 2G, no FSTYPE
sudo blkid /dev/vdb                       # prints nothing. that is what blank means
```

In this order:

```
less scripts/setup.sh          # 1. read it BEFORE running it
sudo ./scripts/setup.sh        # 2. build the volume and mount it
```

It prints which device it chose and why. It refuses any device carrying a signature, a partition table, or a mount, so it cannot eat something you care about. It also never reformats an existing `labdata` — run it twice and the second run reports what is already there.

```
sudo pvs; sudo vgs; sudo lvs    # 3. Look at your own work, one layer each
df -h /srv/data
systemctl cat srv-data.mount
```

`vgs` shows roughly 1.5 GB of `vFree`: the volume is 512 MB out of a 2 GB group, and that free space is what you are about to grow into.

Now the day itself, by hand, with the filesystem mounted and in use the whole time:

```
sudo lvextend -L +512M labvg/labdata
df -h /srv/data                # UNCHANGED. the volume grew, the fs did not
sudo resize2fs /dev/labvg/labdata
df -h /srv/data                # now it grew, with nothing unmounted
```

That is the second manual check, and those four commands are the reason this day exists.

```
./scripts/explore-storage.sh    # 4. the tour
```

It works without root, but two blocks need it, so run it with `sudo` the first time and read every section. Stop at anything you cannot explain and read `man 8 lvextend`, `man 5 systemd.mount` Or `man 8 lsof` on the VM.

```
sudo ./scripts/break-and-fix.sh          # 5. three failures, three fixes
sudo ./scripts/break-and-fix.sh --hard  # 6. the two that destroy data
```

The third failure holds a deleted file open and shows you `df`, `du` and `ls` disagreeing about the same 200 MB. Watch all three. `--hard` ends on the sequence that actually destroys filesystems: growing is `lvextend` then `resize2fs`, shrinking is `resize2fs` then `lvreduce`, and getting the shrink order backwards cuts the end off a filesystem that still believes it owns those blocks.

To make the ghost file yourself, in two shells on the VM:

```
# shell 1 - it needs to write into /srv/data, which root owns
sudo ./scripts/lab-writer.sh ghost /srv/data 100

# shell 2 - ssh in again from your laptop: ./lab/lab.sh ssh node1
df -h /srv/data          # the space is gone
du -sh /srv/data         # du cannot see it
sudo lsof +L1            # NLINK 0 - there it is
```

`setup.sh` also installs that script as `/usr/local/bin/lab-writer`, so `sudo lab-writer ghost /srv/data 100` works too — but only if `/usr/local/bin` is on your `PATH`, and `sudo` on RHEL-family systems uses its own `secure_path` which often does not include it. Calling the script by path always works, so that is what this page does.

4. Check yourself

```
cd ~/lab
sudo ./days/day04/verify.sh
```

Expect **5 PASS, 2 YOU, exit 0**. The two `you` lines are yours to judge honestly: nothing can prove you watched `df` change, or that you found the deleted file rather than reading about it here.

5. Prove it holds across a reboot

A mount that works until the next boot is not a persistent mount. This step needs the real disk from `add-disk` — on a loop device the volume group will be gone and that is expected, not a failure.

```
hostname          # must still print: node1
sudo reboot
```

The SSH session drops. Wait about thirty seconds, then from your laptop:

```
./lab/lab.sh ssh node1
```

And back on the VM:

```
mountpoint /srv/data          # mounted, with nobody having mounted it
systemctl is-enabled srv-data.mount
df -h /srv/data              # still the size you grew it to
sudo lvs labvg/labdata
sudo journalctl -u srv-data.mount -b | tail
```

Three things are worth noticing. The mount came back because the unit was **enabled**, not because it was mounted when you rebooted — `mount` alone would not have survived. The filesystem is still the extended size, because `resize2fs` wrote that into the superblock on disk rather than into memory. And LVM found

`labvg` on its own before anything tried to mount it, because it scans devices at boot and assembles volume groups from the metadata written on the disks themselves — which is why an LVM disk can be moved to a different machine and still work.

6. Optional cleanup

```
sudo ./scripts/teardown.sh
```

You do not have to. Nothing in Days 05-20 conflicts with `labvg` or `/srv/data` — Day 20 backs up `/srv/data` and is glad to find it already there. Teardown exists so that "create, inspect, remove, prove it is gone" is a complete loop, and because storage leftovers are expensive: an enabled mount unit for a device that no longer exists can block anything ordered after it at boot.

To give the RAM back to your laptop when you are done for the day:

```
./lab/lab.sh down node1          # deletes the VM and its disk
```

The base image stays cached, so `up` next time takes a minute, not a download. Note that `down` removes the extra disk from `add-disk` as well, so `up` gives you a clean VM and you would run `add-disk` again.

Notes

Next up: **Day 05 — Logs and time: journald, logrotate and chrony.**

DAY 05 Day 05 — Logs and time: journald, logrotate and chrony

days/day05/README.md

This day is provable, not descriptive: **5 automatic checks** and **2 judgement calls** in `verify.sh`.

Make logs persistent, bounded, and correctly timestamped.

PHASE	THE HOST
Runs on	VM: node1
Memory	~2 GB (one VM)
Verified by	lint + your lab

Why this day exists

Every diagnosis in Days 06-20 is log reading. If the logs are not there, or the timestamps lie, none of those days work.

Two defaults on a fresh RHEL-family machine will cost you an incident. The journal is stored in `/run`, which is memory, so the boot that crashed is erased by the boot that recovered — `journalctl -b -1` has nothing to show you at exactly the moment you need it. And logs that nothing rotates grow until the filesystem is full, which is the Day 04 page arriving by a different route.

Today you fix both, and then you break them in the three ways that are hard to recognise: a rotated log that keeps growing while the file you are watching stays empty, a journal quietly deleting your history with no error anywhere, and a clock two hours out that makes entries invisible to the query you are typing.

What you will work with

- `journalctl` and its four selectors - `-u` for a unit, `-b` for this boot (`-b -1` for the previous one), `-p err` for priority err and worse, and `--since` which takes plain English like "2 hours ago". Those four cover most real use. The important thing to understand first is that the journal is not a text file: it is a binary, indexed store of structured fields, so you cannot `grep /var/log/journal` and `journalctl` is the only reader. Run `journalctl -n 1 -o json-pretty` once and you will stop thinking of log entries as lines.
- `Storage=` in `/etc/systemd/journald.conf` - the default is `auto`, which means "use `/var/log/journal` if that directory exists, otherwise keep everything in `/run/log/journal`". `/run` is tmpfs, so on a default machine the journal is erased at every boot. Creating the directory is genuinely all it takes to change the behaviour, which is why so many machines are non-persistent by accident rather than by decision.
- `SystemMaxUse=` - a hard cap enforced by deleting the oldest entries. Not a warning, not a refusal to log: silent deletion, while every command reports healthy. The default is 10% of the filesystem, which is a strange number to have chosen for you on a root filesystem you needed for something else. Set it deliberately and understand that you are choosing how far back you can see, not how much disk to spend.
- `logrotate` and `/etc/logrotate.d/` - the other logging system, still running on every one of these machines, for every log that is a plain file rather than a journal entry. A rule there is a **request**, not a schedule: `logrotate.timer` is what actually runs it. Debugging a rotation problem without checking `systemctl list-timers | grep logrotate` is the most common wasted hour in this area.

- `copytruncate`, and what a rename really does - logrotate's default is to rename `app.log` to `app.log.1` and create a new empty `app.log`. But a process that already has the file open holds a descriptor pointing at an **inode**, and renaming a file does not touch its inode. So a daemon that never reopens its log keeps writing into `app.log.1` forever while the file you are tailing stays at zero bytes. `copytruncate` avoids that by emptying the original file in place; the better answer, when the daemon supports it, is a `postrotate` stanza that signals it to reopen.
- `logrotate -d` - a dry run that says what it would do and changes nothing. It is the only safe way to test a rule, and it is today's first manual check. Note what it does *not* tell you: whether anything will ever call it.
- `chronyc tracking` and `chronyc sources -v` - `tracking` answers "is my clock right", where `Leap status: Normal` means synchronised and `System time` is your current error in seconds. `sources -v` answers "who is telling me", and the `^*` marker is the source actually in use. Chrony **slews** small errors — speeds the clock up or slows it down until it catches up — rather than jumping, because time going backwards breaks builds, databases and certificate checks. `chronyc makestep` forces a jump, and is the right tool exactly once: when you already know the clock is grossly wrong.
- `timedatectl` - and the distinction it makes that people miss. The timezone is a **display** setting; the journal always stores UTC internally, so changing the zone changes how `journalctl` prints entries and nothing about what was recorded. The clock is the actual number of seconds. `timedatectl` shows both, plus whether NTP is even switched on, and those lines can disagree with each other.

Verify

Checked automatically:

- ☐ the journal is persistent across reboots
- ☐ journal size is bounded
- ☐ the clock is synchronised
- ☐ the timezone is set deliberately
- ☐ a logrotate rule exists for your own log

Only you can confirm:

- ☐ `logrotate -d` showed your rule doing what you intended
- ☐ you watched a rotated log keep growing, and found the open descriptor

Run the automatic checks with:

```
./days/day05/verify.sh
```

No root today. Every check reads something a normal user can read: a directory, two config files, and `chronyc`, which answers anybody. Root would let `journalctl` show you more, and the tour asks for it for that reason, but the checks themselves do not need it and none of them will print `SKIP`. If they ever do, `SKIP` is not a pass — it means the check could not run at all.

Run it on node1, not on your laptop. Nothing inside `verify.sh` checks which machine you are on, and today that is more dangerous than usual, because your laptop will score well. `the clock is synchronised` and `the timezone is set deliberately` both **PASS** on any working desktop, and `the journal is persistent across reboots` passes on most distributions too, since they ship `/var/log/journal` already. So an off-VM run

can report four green lines and one red one, which looks like "nearly done" rather than "wrong machine". The only check that genuinely reflects your work today is `a logrotate rule exists for your own log`.

CI can only lint this day. Nothing on a GitHub runner has SELinux, firewalld, systemd units you control, or a second host to reach over SSH — so the checks below are proven by running `verify.sh` on your own lab, and nowhere else.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>lab-noisy.sh</code>	The payload. Writes one line per second to a plain file and to the journal, from the same loop. Holds its log open on fd 3 and never reopens it, exactly like a real daemon — which is what makes failure 1 possible. Installed as <code>/usr/local/bin/lab-noisy</code> .	no
<code>setup.sh</code>	Makes the journal persistent and caps it at 200 MB, sets the timezone and starts chronyd, installs the payload as <code>lab-noisy.service</code> , and writes the logrotate rule. Idempotent.	yes
<code>explore-logs.sh</code>	Read-only guided tour in ten sections: where the journal really lives, the selectors worth memorising, the plain-file log beside it, what logrotate would do, who actually runs it, and the clock. Works without root, but the journal shows you less — run it with <code>sudo</code> the first time and it says so.	no
<code>break-and-fix.sh</code>	Three failures, each repaired: a rotated log that keeps growing, a journal that deletes its own history, and a clock in the future. <code>--hard</code> adds a perfect rule that never runs and journald silently dropping messages.	yes
<code>teardown.sh</code>	Stops the writer <i>before</i> deleting its files, removes the unit, payload, log directory and rule, and proves each is gone. Deliberately leaves the journal and its settings.	yes

Read them before you run them. They are commented as teaching material rather than production code — the comments are half the day.

Run it on the lab

1. On your laptop, bring up node1

Day 05 runs on `node1`, the same VM as Days 02, 03 and 04. It needs no extra disk and nothing from those days, so whatever is already there can stay.

```
./lab/lab.sh status           # what is already running?
./lab/lab.sh up node1        # 2 GB, about a minute
```

If you tore the VM down since Day 04, remember what `down` takes with it: the whole machine, its disk, and any disk added with `add-disk`. A rebuilt `node1` is a blank Rocky image with none of the packages from previous days.

2. Copy the repo onto the VM

```
./lab/lab.sh push node1      # carries both days/ and lab/
./lab/lab.sh ssh node1
```

Both directories have to travel: the day scripts source `lab/on-lab-vm.sh`, and without it every one of them refuses to run.

3. Work through the day on the VM

Check the machine before typing anything that changes state — today's scripts edit `journald.conf` and move the system clock:

```
hostname                # must print: node1
cd ~/lab/days/day05
```

The Rocky 9 cloud image does not ship chrony, so install today's tools first:

```
sudo dnf install -y chrony logrotate
```

`chrony` gives you `chronyd` and `chronyc`; `logrotate` gives the binary and its timer. `logger` and `journalctl` are already there, from `util-linux` and `systemd`. `setup.sh` checks for all five before it changes anything and stops with this same command if any are missing.

Look at the defaults before you change them — this is the part worth seeing once:

```
ls -ld /var/log/journal /run/log/journal 2>&1 # which one exists?
journalctl --list-boots                      # how many boots are remembered?
journalctl --disk-usage
timedatectl
```

On an untouched cloud image `/var/log/journal` does not exist, `--list-boots` prints exactly one line, and that one line is the whole of the machine's memory. Everything from before this boot is already gone.

In this order:

```
less scripts/setup.sh          # 1. read it BEFORE running it
sudo ./scripts/setup.sh        # 2. persistent journal, capped, service, rule
```

It prints each step as it goes and ends by showing you the same messages arriving in both systems. If `chronyd` cannot reach an NTP source it says so rather than pretending — that is a real finding about the VM's network, and `chronyc sources -v` is where you chase it.

```
tail -3 /var/log/lab-app/app.log          # 3. the file logrotate owns
journalctl -t lab-noisy -n 3 --no-pager    # the journal journald owns
```

Same process, same second, two destinations. Almost every argument about "where do the logs go" on a modern machine comes from not noticing both systems are running.

Now the manual checks, by hand. First the dry run:

```
sudo logrotate -d /etc/logrotate.d/lab-app
```

Read its output properly and find the line that says whether the log needs rotating and why. `-d` changes nothing, which is what makes it the only safe way to test a rule. That is the first `you`.

Then force a real rotation and watch what happens to the file you are tailing:

```
sudo /usr/local/bin/lab-noisy burst 1200    # push it past the 100k threshold
sudo logrotate -f /etc/logrotate.d/lab-app
ls -l /var/log/lab-app/
sleep 5; ls -l /var/log/lab-app/             # which one is growing?
```

With `copytruncate` in the rule, `app.log` is the one that grows. `break-and-fix.sh` takes that one line out and shows you the other outcome, which is the second `you`.

```
./scripts/explore-logs.sh          # 4. the tour
sudo ./scripts/explore-logs.sh    #   again with root - the journal shows more
```

Stop at anything you cannot explain and read `man 5 journald.conf`, `man 8 logrotate` Or `man 1 chronyc` on the VM.

```
sudo ./scripts/break-and-fix.sh    # 5. three failures, three fixes
sudo ./scripts/break-and-fix.sh --hard # 6. the two nobody checks for
```

Failure 1 is the one to slow down on: it prints the two file sizes side by side, three seconds apart, and then shows you `/proc/<pid>/fd/3` still pointing at the renamed inode. That is the same technique you used to find the ghost file on Day 04, applied to a problem that looks completely different. Note that `--hard` moves the system clock forward two hours and hands it back to `chronyd` at the end; if this VM has no outbound NTP the clock may need `sudo date -s` by hand afterwards.

4. Check yourself

```
cd ~/lab
./days/day05/verify.sh
```

Expect **5 PASS, 2 YOU, exit 0**. No `sudo` needed. The two `you` lines are yours to judge honestly: nothing can prove you read the dry run, or that you watched the wrong file grow rather than taking this page's word for it.

5. Prove it holds across a reboot

This is the whole point of persistence, and it is the one thing on this page you cannot verify any other way — a journal that is persistent until the next boot is just a journal.

```
hostname          # must still print: node1
journalctl -t lab-noisy -n 1 --no-pager # note this timestamp
sudo reboot
```

The SSH session drops. Wait about thirty seconds, then from your laptop:

```
./lab/lab.sh ssh node1
```

And back on the VM:

```
journalctl --list-boots          # now MORE than one line
journalctl -b -1 -t lab-noisy | tail -3 # the entries from before the reboot
journalctl -b -1 -p err          # errors from the previous boot
systemctl is-active lab-noisy.service # started itself again
tail -2 /var/log/lab-app/app.log
timedatectl                     # clock still correct, still synchronised
```

Three things are worth noticing. `journalctl -b -1` works at all — before today it would have said the boot ID has no effect, because there was nothing older than the current boot to look at. The service came back because it was **enabled**, not because it was running when you rebooted. And the clock is right immediately on boot, which is `chronyd` starting early and correcting it, not the hardware clock being trustworthy — a VM's hardware clock drifts and nothing about `date` alone would have survived.

6. Optional cleanup

```
sudo ./scripts/teardown.sh
```

You do not have to. Nothing in Days 06-20 conflicts with `lab-noisy` or `/var/log/lab-app`. Teardown exists so that "create, inspect, remove, prove it is gone" is a complete loop — and it deliberately leaves the journal, its persistence and its cap alone, because those are a machine improvement rather than this day's mess, and every later day reads logs. Run it and then `journalctl -t lab-noisy | tail`: the service is gone and its history is still there, which is the difference between the two systems in one command.

To give the RAM back to your laptop when you are done for the day:

```
./lab/lab.sh down node1          # deletes the VM and its disk
```

The base image stays cached, so `up` next time takes a minute, not a download. Note that this deletes the persistent journal along with the disk, so if you want to see `journalctl -b -1` work, do step 5 before you do this.

Notes

Keep your own notes here. What broke, what the error actually said, and what fixed it — that is the part you will come back for.

Next up: **Day 06 — Interfaces, routing and building the namespace lab.**

docs/curriculum.md#phase-2

PHASE 2

The network

Days 06–10

06	Interfaces, routing and building the namespace lab	43
07	The DNS resolution path	48
08	Running DNS: authoritative and recursive	53
09	Packet-level debugging	58
10	TLS on the wire and a private CA	62

DAY 06 Day 06 — Interfaces, routing and building the namespace lab

days/day06/README.md

This day is provable, not descriptive: **5 automatic checks** and **2 judgement calls** in `verify.sh`.

Build a four-node routed network inside your kernel and prove packets cross it.

PHASE	THE NETWORK
Runs on	Host: network namespaces
Memory	0 MB (no VM at all)
Verified by	lint + CI

Why this day exists

This is real kernel networking, not a simulation: real interfaces, real routing tables, real packets. It is also the environment Days 07-10 and 18 run in, at zero memory cost.

Today is the first day with no VM, and that changes what you should expect. A network namespace is a separate, empty copy of the kernel's entire network stack — its own interfaces, its own routing table, its own sysctls. Nothing you do today adds an address, a route or a rule to the namespace your own machine uses, which is why this day is safe to run on your laptop and why CI can execute it for real instead of only linting it.

```
client  10.10.0.2  ---  10.10.0.1  .
                                     router (forwards IPv4)
resolver 10.10.1.2  ---  10.10.1.1  '
auth    10.10.2.2  ---  10.10.2.1  '
```

NAMESPACE	ITS ADDRESS	ITS VETH	ROUTER'S VETH	ROUTER'S ADDRESS
client	10.10.0.2/24	veth-cl	veth-rc1	10.10.0.1/24
resolver	10.10.1.2/24	veth-rs	veth-rrs	10.10.1.1/24
auth	10.10.2.2/24	veth-au	veth-rau	10.10.2.1/24

Three separate /24s, one router with a leg in each, and no other connection between them. The leaf namespaces cannot reach each other at all except through the router, and the router needs no default route because it is directly attached to every network in the lab. Learn these three names and three networks now: Day 07 resolves through the `resolver`, Day 08 makes `auth` authoritative for a zone, Day 09 captures packets on the router's legs, and Day 18 rebuilds the wiring with bridges and VLANs.

The network you build is four namespaces and one router, and it is deliberately not bridged and not NAT-ed. For the client to reach the auth network, a packet has to genuinely be forwarded by a Linux box that has been told to forward. Then you break that five ways and learn that the five breakages produce four distinguishable symptoms — which is the actual skill, because on a bad day the symptom is all you get.

What you will work with

- * `ip netns add` and `ip netns exec` - a named network namespace, and a way to run one command inside it. A fresh namespace is emptier than a machine with networking switched off: it has exactly one interface,

- `lo`, and even that starts **DOWN**, so it cannot reach `127.0.0.1` until you bring it up. `ip -n client route` is shorthand for `ip netns exec client ip route` and is worth using for anything that is purely an `ip` command.
- `ip link add ... type veth peer name ...` - a veth is created as a **pair** and only ever exists as a pair: two interfaces, a virtual cable between them, whatever enters one end leaves the other. You cannot create one veth any more than you can create one end of a cable, and deleting either end deletes both. The pair is born in your own namespace and then each end is *moved* where it belongs - and moving an interface into a namespace wipes its addresses, which is why addressing has to come after the move rather than during it.
- `ip addr` and the prefix length - the number after the slash is not decoration. It is what tells the kernel which addresses are on-link, reachable without a router, and it silently creates the **connected route** you will find in the routing table without having typed it. One wrong digit there is failure 5 of `break-and-fix.sh`, and it is the most convincing wrong configuration in this whole repo.
- `ip route` and `ip route get` - `ip route` lists the rules; `ip route get 10.10.2.2` performs the actual lookup and shows you the decision: which route matched, which device, which source address, which next hop. When a routing table looks correct and traffic still goes the wrong way, `route get` is the command that tells you what the kernel concluded instead of leaving you to infer it. Longest prefix always wins, and `default` is the shortest prefix there is.
- `net.ipv4.ip_forward` - the difference between a router and a host with three interfaces. A Linux host that receives a packet addressed to somebody else, with forwarding off, drops it **silently**: no ICMP, no log, no counter you will find on your first look. The only symptom is a ping that never comes back. This sysctl is per-namespace, like nearly everything under `net.ipv4`, so the router's setting and your laptop's setting are unrelated values.
- `ip -br addr` and `ip -br link` - brief output, one line per interface, which is what makes a namespace's whole configuration readable at a glance. Read the **flags**, not just the `inet` line: `state DOWN` and `NO-CARRIER` sit right there and are the most skipped-over fields in this area. An address can be perfectly configured and completely unusable, and only the flags say so.
- `ip neigh` - the ARP table, populated by traffic rather than by configuration, so it is empty until something is sent. When a ping fails you can ask whether the next hop's MAC was ever learned, which separates "my neighbour is not there" from "my neighbour is there and the problem is further away".
- `lab/lab.sh netns-up` - the same topology in one command, for every day after this one. Days 07-10 and 18 begin with either that or today's `setup.sh`; both build identical names and addresses on purpose, so a later day cannot tell which you used.

Verify

Checked automatically:

- ☐ all four namespaces exist
- ☐ the client has an address on 10.10.0.0/24
- ☐ the router forwards IPv4
- ☐ the client reaches the auth network through the router
- ☐ the client has a default route

Only you can confirm:

- `ip` you can draw the topology from `ip route` output alone
- `ping` you saw a ping fail in one direction only, and proved which one

Run the automatic checks with:

```
sudo ./days/day06/verify.sh
```

Root is required today, and this is the one day where that is not a compromise. Entering a network namespace is a privileged operation, so without root there is nothing to check at all - every line would print `SKIP`, and `SKIP` is not a pass. `verify.sh` declares this with `vl_need_root`, so a non-root run says so plainly instead of reporting five red lines you would then go and debug.

Run it wherever you like - including your own laptop. This is the first day with no "wrong machine", because namespaces are isolated from the machine that hosts them. There is no false PASS to warn you about and no VM to be on. If the checks are red, the topology genuinely is not built: run `sudo ./days/day06/scripts/setup.sh`.

CI executes this day for real. A GitHub runner has a Linux kernel, and a Linux kernel is the only thing this day needs - so `scripts/setup.sh` runs and then `verify.sh` runs, on every push. If you break the topology, the pipeline tells you, which is not true of Days 01-05.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>lab-netcheck.sh</code>	The payload. Prints a reachability matrix: every namespace tried against every address in the topology. There is no daemon on a networking day - the thing worth having is a question you can ask repeatedly, so leave this in one terminal and re-run it after every change. Takes an optional namespace name to test just one source.	yes
<code>setup.sh</code>	Builds the whole network by hand in seven visible steps: namespaces, veth pairs, addresses, forwarding, default routes, then shows the routing table it produced and pings across it. Same names and addresses as <code>lab.sh netns-up</code> . Idempotent - run it twice and the second run repairs.	yes
<code>explore-net.sh</code>	Read-only tour in ten sections: what a fresh namespace really contains, how the kernel describes a veth pair, the two routes in the client's table, <code>route get</code> as the decision itself, the router's three legs, ARP arriving with traffic, and what the namespace cannot see.	yes
<code>break-and-fix.sh</code>	Three failures, each repaired: forwarding off, no default route, and a far end that cannot reply. <code>--hard</code> adds two that look like working configuration - a correct address on a dead link, and a <code>/16</code> where a <code>/24</code> belonged.	yes
<code>teardown.sh</code>	Deletes the leaves one at a time so you can watch the router lose a leg with each one, then removes the router and proves all four are gone. Counts stranded interfaces on your own machine, and shows that your <code>ip_forward</code> was never touched.	yes

Every script needs root today, which is unusual for this repo and worth saying out loud rather than leaving you to discover it. It is not privilege creep: `ip netns exec` is privileged, and there is no unprivileged way to look inside another network stack.

Read them before you run them. They are commented as teaching material rather than production code - the comments are half the day.

Run it on the lab

1. No VM today

Day 06 runs on **the machine you are reading this on**. There is nothing to boot, nothing to copy, nothing to SSH into, and no memory to budget. If you still have `node1` running from Day 05 and want the RAM back:

```
./lab/lab.sh down node1          # deletes the VM and its disk
```

The base image stays cached, so `up` next time takes a minute, not a download. Days 07-10 and 18 need no VM either, so that RAM stays yours for a while.

2. Check you have the three tools

```
command -v ip ping sysctl
```

On RHEL-family systems: `sudo dnf install -y iproute iputils procps-ng`. On Debian or Ubuntu: `sudo apt-get install -y iproute2 iputils-ping procps`. `setup.sh` checks all three before it changes anything and stops with the right command for your distribution if any are missing.

3. Build the network

```
cd ~/lab                          # or wherever you cloned it
sudo ./days/day06/scripts/setup.sh # 1. seven steps, each explained
sudo ./days/day06/scripts/lab-netcheck.sh # 2. the reachability matrix
```

Read the setup output as it goes rather than at the end. Step 3 prints the addresses; step 6 prints the client's routing table and points out that one of its two lines was never typed by anyone. If you already built the topology with `sudo ./lab/lab.sh netns-up`, this script repairs rather than complains - the two are interchangeable by design.

The matrix is six addresses against four sources, every combination. That is the picture to keep in a second terminal for the rest of the day.

4. Look at what you built, then break it

```
sudo ./days/day06/scripts/explore-net.sh # 3. the tour
```

Stop at anything you cannot explain and read `man 8 ip-route`, `man 8 ip-netns` or `man 8 ip-address`. Section 5 is the one to slow down on: three `route get` calls giving three different answers, from one routing table with two lines in it.

```
sudo ./days/day06/scripts/break-and-fix.sh          # 4. three failures
sudo ./days/day06/scripts/break-and-fix.sh --hard    # 5. two more, subtler
```

Five failures, four symptoms. This table is the day's real content, and it is worth writing on paper before you run either command:

SYMPTOM	WHAT IT MEANS	WHICH FAILURE
Instant "Network is unreachable"	No route matched. Nothing was ever transmitted, so nothing is on the wire to capture.	2
Timeout, next hop still answers	The packet left and died elsewhere - classically forwarding, or a filter	1
Timeout, and requests are arriving at the far end	The return path is broken, not the forward one	3
Timeout, with address, link and gateway all apparently correct	The kernel decided something you did not intend - ask <code>ip route get</code>	4, 5

Failure 3 is today's second `you`. Nothing about the client or the router changes - the far end simply loses its route home, so the echo request arrives and the reply has nowhere to go. A ping proves a **round trip**, so a failed ping can never tell you which direction failed; `tcpdump` at the far end can, and does. Everything is restored before the script exits, so `verify.sh` is green afterwards. If you interrupt it half way, run `setup.sh` again.

5. Check yourself

```
sudo ./days/day06/verify.sh
```

Expect **5 PASS, 2 YOU, exit 0**. Root is required, and without it every line prints `SKIP` rather than red.

The first `you` is a real exercise, not a formality: run `sudo ip netns exec client ip route`, then the same for `router`, `resolver` and `auth`, and draw the four boxes and three cables on paper from those four outputs alone. If you can do that, you can read a real network's routing tables, which is most of what Days 07-10 ask of you.

6. Optional cleanup

```
sudo ./days/day06/scripts/teardown.sh
```

On this day cleanup costs you nothing, because none of it is persistent. Namespaces live in the running kernel, so a reboot removes all four whether you ask or not, and every day from 07 onwards starts by rebuilding them in about a second. That is a feature: the environment is never stale and never needs migrating.

Run `teardown` once anyway, and watch the middle of the output. It deletes the leaves one at a time and counts the router's remaining legs after each, so you see a veth pair dying from the other end - the deletion you typed was three namespaces away from the interface that disappeared.

Notes

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Next up: **Day 07 — The DNS resolution path.**

days/day07/README.md

This day is provable, not descriptive: **5 automatic checks** and **2 judgement calls** in `verify.sh`.

Trace one name lookup through every layer that can answer it.

PHASE	THE NETWORK
Runs on	Host: network namespaces
Memory	0 MB (no VM at all)
Verified by	lint + CI

Why this day exists

"It is always DNS" is a joke because it is usually true. Knowing which layer answered is the difference between a five minute fix and an afternoon.

A name lookup is not one thing. It is a queue of layers, each of which can answer and stop the search, and the layer that answers is almost never the one being blamed. Today you build that queue by hand and then make the same name return two different addresses depending on which tool asks — not as a trick, but because that is what a real machine does, and because the day you meet it in production you will not have time to learn it.

```

application
|
| getent / any program using glibc
v
/etc/nsswitch.conf ---> files ---> /etc/hosts      10.10.0.99
|
| dns
|
| (answered - stop)
v
/etc/resolv.conf ---> 10.10.1.2 ---> lab-nameserver 10.10.2.2
^
|
dig, straight to the socket, reading none of the above

```

The important asymmetry: **dig is not a resolver**. It is a DNS client. It reads `/etc/resolv.conf` only to find out where to send a packet, and it never consults `nsswitch.conf` or `/etc/hosts` at all. So `dig` tells you what DNS thinks, and `getent` tells you what your application will actually get — and when those two disagree, `getent` is the one that matters and `dig` is the one that explains why.

The second thing this day teaches is where per-namespace DNS configuration actually comes from, because it is not obvious and it is not a kernel feature. `ip netns exec client ...` bind-mounts everything in `/etc/netns/client/` over `/etc/` for the life of that one command. Put `resolv.conf` there and the client has its own nameserver; put it in the wrong directory and the file exists, greps fine, and is never read. That is failure 5.

What you will work with

- * `getent hosts` **VS** `dig` - the two halves of today. `getent` asks glibc, which means it obeys `nsswitch.conf`, reads `/etc/hosts`, and returns what a real program would get; its exit status is 2 when nothing was

found. `dig` opens a UDP socket to a nameserver and prints the reply, so it sees DNS and nothing but DNS. Neither is more correct — they answer different questions, and using the wrong one is how people spend an afternoon debugging a hosts entry with a tool that cannot read hosts files.

- `/etc/nsswitch.conf` - the `hosts:` line is the queue itself, read left to right, first answer wins. `files dns` means a hosts entry is consulted before any packet is sent, so DNS is not slow in that case — it is never asked. Removing `files` from that line breaks nothing visibly and changes every pinned address on the machine, which is failure 1.
- `/etc/hosts` - matched as a **literal string**, with no DNS conventions applied. `www.lab.test` and `www.lab.test.` are two different entries and only one of them will ever match, because the trailing dot that means "fully qualified" in DNS means "a different name" here. That is failure 4, and it survives code review every time.
- `/etc/resolv.conf` - three or four lines that decide where queries go: `nameserver` (up to three, tried in order), `options timeout:` and `attempts:` (which is why a dead nameserver costs you seconds rather than milliseconds), and `search`, which today's file deliberately omits so that every lookup is the name you actually typed. On most modern desktops this file is generated and overwritten — check whether it is a symlink before you edit it, or your fix will vanish at the next reconnect.
- `/etc/netns/<namespace>/` - the bind-mount trick above. Anything you put here shadows `/etc/<same name>` inside that one namespace, and nowhere else. It is how this whole day runs without touching your machine's own resolver, and it is worth remembering as a general technique for testing resolver changes safely.
- `systemd-resolved` and `resolvectl` - the layer this day deliberately does *not* use, and you should know why. On a desktop, `/etc/resolv.conf` often points at `127.0.0.53`, a stub listener, and the real configuration lives in `resolvectl status` — per-interface nameservers, per-domain routing, a cache, and DNSSEC. If your own laptop is set up that way, `dig` against `127.0.0.53` tells you about the stub and not about the upstream. `resolvectl query` is the `getent` -equivalent there. Inside a namespace none of that is running, which is exactly why the plain path is legible today.
- `dig +short`, `+norecurse`, `+trace` - `+short` for scripting and quick answers; the full output for the **flags**, where `aa` means the server claimed authority for the zone and `ra` means it is willing to recurse. `+norecurse` asks a server to answer only from what it already knows, which is how you tell an authoritative server from a cache. `+trace` walks from the root down, one delegation at a time — it needs real internet roots, so it is a tool for tomorrow's public names rather than for this lab's private zone.
- `lab-nameserver` (the payload) - about sixty lines that answer A queries for four names and NXDOMAIN for everything else. It exists so you can watch queries arrive, in a log, with the asker's address, and so today has a nameserver without also having a day's worth of configuration. Day 08 throws it away and runs the real thing.

Verify

Checked automatically:

- ☐ nsswitch consults files before dns
- ☐ a hosts entry beats DNS for the same name
- ☐ the client has a nameserver configured
- ☐ the nameserver answers from the resolver namespace

- `dig` and `getent` are both available to compare

Only you can confirm:

- you can explain why `dig` ignored `/etc/hosts` and `getent` did not
- you followed one name from application call to authoritative answer

Run the automatic checks with:

```
sudo ./days/day07/verify.sh
```

Root is required today, for the same reason as Day 06. Every check reads from inside a namespace, and entering one is privileged. Without root you get `SKIP` on every line, and `SKIP` is not a pass. `verify.sh` declares this with `vl_need_root` so a non-root run says so plainly.

Run it wherever you like - including your own laptop. There is no "wrong machine" on a namespace day. Nothing here reads or writes your own resolver configuration: the checks look inside the `client` namespace, and the files they read live under `/etc/netns/client/`. If the checks are red, the day genuinely is not built.

Every check reads from **inside** the namespace, and that is deliberate. An earlier version of this file checked the host's `/etc/nsswitch.conf`, which would have passed on almost any Linux machine whether you had done the day or not. When a check can pass without the work being done, it is decoration.

CI executes this day for real, on every push. The runner installs `dnsutils` and has `python3`, so the nameserver runs, the queries fly and `verify.sh` reports. That is a genuine gate, unlike Days 01-05.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>lab-nameserver.sh</code>	The payload: a small authoritative nameserver for four names, in about sixty lines. Logs every query with the asker's address, which is what makes "did the packet arrive?" answerable rather than guessable. Written in Python because a DNS answer is a byte layout, not text you can fake with <code>printf</code> .	yes
<code>setup.sh</code>	Builds Day 06's topology if it is missing (a reboot removes it, so that is the normal case, not an error), installs the payload, writes the client's own <code>resolv.conf</code> , <code>nsswitch.conf</code> and <code>hosts</code> under <code>/etc/netns/</code> , starts the nameserver in the <code>resolver</code> namespace, then proves both paths answer and that they disagree . Idempotent - run it twice and the second run restarts cleanly.	yes
<code>explore-dns.sh</code>	Read-only tour in twelve sections: the two different <code>/etc/resolv.conf</code> files, the same name asked two ways, a name only DNS knows, a name nobody knows, why <code>dig localhost</code> returns nothing, reading the <code>aa</code> flag, and the server-side log of everything you just asked.	yes
<code>break-and-fix.sh</code>	Three failures, each repaired: <code>files</code> dropped from <code>nsswitch</code> , a nameserver address that nothing answers on, and a nameserver that is stopped rather than absent. <code>--hard</code> adds two that read as correct configuration - a trailing dot in <code>/etc/hosts</code> , and the right file in the wrong namespace.	yes
<code>teardown.sh</code>	Stops the nameserver, removes the per-namespace files, and shows the client falling back to your machine's own resolver - without anything being unmounted, which explains what the bind mount really was. Leaves Day 06's namespaces alone, because Days 08, 09 and 18 need them.	yes

Every script needs root today. It is not privilege creep: `ip netns exec` is privileged, port 53 is privileged, and `/etc/netns/` is root-owned.

Read them before you run them. They are commented as teaching material rather than production code - the comments are half the day.

Run it on the lab

1. No VM today

Day 07 runs on **the machine you are reading this on**, inside Day 06's namespaces. Nothing to boot, nothing to copy, no memory to budget. If `node1` is still running from Day 05 and you want the RAM back:

```
./lab/lab.sh down node1          # deletes the VM and its disk
```

The base image stays cached, so `up` next time takes a minute, not a download.

2. Check the topology and the tools

Today resolves across Day 06's network. Namespaces live in the running kernel and never survive a reboot, so if you have rebooted since Day 06 they are simply gone - which is why `setup.sh` rebuilds them for you rather than complaining. You can also do it yourself first if you prefer to watch it happen:

```
sudo ./days/day06/scripts/setup.sh  # optional - or: sudo ./lab/lab.sh netns-up
command -v ip getent dig python3
```

On RHEL-family systems: `sudo dnf install -y iproute bind-utils python3`. On Debian or Ubuntu: `sudo apt-get install -y iproute2 dnsutils python3`. `setup.sh` checks all four before it changes anything, and stops with the right command for your distribution if any are missing.

3. Build the resolution path

```
cd ~/lab                          # or wherever you cloned it
sudo ./days/day07/scripts/setup.sh  # 1. files, nameserver, both proofs
```

Read step 5 of its output carefully. It asks for one name two ways and gets two different addresses, and it **fails on purpose** if those two answers ever agree - because if they agree, the hosts file is not being read and the rest of the day would teach you nothing.

4. Look at what you built, then break it

```
sudo ./days/day07/scripts/explore-dns.sh  # 2. the tour, twelve sections
```

Section 1 is the one to slow down on: `cat /etc/resolv.conf` inside the namespace and outside it, two different files, same path. Section 7 is the one people remember - `dig +short localhost` returns nothing at all, on a working machine.

```
sudo ./days/day07/scripts/break-and-fix.sh      # 3. three failures
sudo ./days/day07/scripts/break-and-fix.sh --hard  # 4. two more, subtler
```

Five failures, and **four of the five never print an error message**. That is the point of the day, so keep this table beside you:

SYMPTOM	WHAT IT MEANS	WHICH FAILURE
<code>getent</code> and <code>dig</code> suddenly agree	A layer stopped being consulted. Read the <code>hosts:</code> line.	1
Some names resolve, others hang	The ones that work are not using DNS at all	2
A timeout rather than a refusal	Something is listening and nothing is answering	3
A hosts entry that reads correctly and is ignored	The exact string, dots and all	4
A file that exists, greps fine, and is not seen	Wrong directory under <code>/etc/netns/</code>	5

Failure 3 is worth the detour. A timeout, a refusal and an NXDOMAIN are all reported as "DNS is broken" and they have three different causes: nothing read the packet, nothing was listening, or a server authoritatively said no. Learn to tell them apart from the *shape* of the failure and you skip most of the guessing.

Everything is restored before the script exits, so `verify.sh` is green afterwards. If you interrupt it half way, run `setup.sh` again.

5. Check yourself

```
sudo ./days/day07/verify.sh
```

Expect **5 PASS, 2 YOU, exit 0**.

The second **YOU** is the real exercise: follow one name from an application's point of view all the way to the answer, and be able to name every layer it passed and every layer it skipped. Do it out loud, with `/var/log/lab-nameserver.log` open in another terminal so you can see which of your lookups actually reached the server. Anything that never appears in that log was answered before DNS was reached - and knowing which of your lookups those were is the whole skill.

6. Optional cleanup

```
sudo ./days/day07/scripts/teardown.sh
```

None of this is persistent, so a reboot clears it whether you ask or not. Run `teardown` once anyway and read step 4: the client's `/etc/resolv.conf` goes back to being your machine's own file, and nothing was unmounted to make that happen - which tells you what the `bind` mount actually was.

It deliberately leaves Day 06's namespaces up. Day 08 replaces this toy nameserver with a real authoritative server and a real recursive resolver on the same topology, and Days 09 and 18 need the wiring too.

Notes

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Next up: **Day 08 — Running DNS: authoritative and recursive.**

DAY 08 Day 08 — Running DNS: authoritative and recursive

days/day08/README.md

This day is provable, not descriptive: **5 automatic checks** and **2 judgement calls** in `verify.sh`.

Serve your own zone, then resolve it recursively from another namespace.

PHASE	THE NETWORK
Runs on	Host: network namespaces
Memory	0 MB (no VM at all)
Verified by	lint + CI

Why this day exists

Reading DNS is one skill; owning a zone is another. Days 10, 12 and 17 all need names that resolve to your own machines.

Yesterday you followed a name through the layers that could answer it, and the thing at the end of the chain was sixty lines of Python. Today the thing at the end is a real server, and the day's subject moves with it: not *can something answer*, but **which of two servers answered, and where did it get that from**.

Those two servers are different in kind, not in configuration. One owns data and has the right to say a name does not exist. The other owns nothing and remembers everything. Almost every confusing DNS incident is the gap between them.

```
client 10.10.0.2 ---> resolver 10.10.1.2 ---> auth 10.10.2.2
                                unbound, recursive      unbound, authoritative
                                caches, owns nothing      owns lab.test, caches nothing
                                flags: ra                 flags: aa
                                TTL counts DOWN           TTL never moves
```

Ask both for `www.lab.test` and you get `10.10.2.10` twice. Nothing in the answer itself tells you they are different machines doing different jobs — that is in the flags and the TTL, which is why today spends so long on both.

What you will work with

- **unbound** - the daemon for both halves of today, which is the point. **unbound** is famous as a recursive resolver, but a single `local-zone: "lab.test." static` line turns it into an authoritative server for one name space and nothing else. Same binary, same config syntax, opposite job: one instance holds data and refuses to look anything up, the other holds no data and looks everything up. Configuration files live under `/etc/unbound/lab/` today rather than in the distribution's own directory, so nothing you do here touches your machine's installed resolver.
- **unbound-checkconf** - validates a config before the daemon reads it, and you should run it before every restart. Be clear about what it does and does not catch: it will reject a misspelled option or a malformed address, and it will happily accept a record attached to the wrong name. Both `--hard` failures below pass **unbound-checkconf** cleanly. A validator proves your syntax, never your intent.
- **A zone's five required record types** - **SOA** names the authority and carries five timers (`serial refresh retry expire minimum`); the last one is the **negative** TTL, which is how long a *no* may be cached. **NS**

names the zone's nameserver, and because it points at a name, that name needs an `A` record too or the delegation cannot be acted on. `A` maps a name to an address. `CNAME` is an alias: it answers with a *name* rather than an address, so somebody has to ask a second question. A full recursive resolver chases that for you; a `static` zone like today's hands you the bare alias and stops. Either way the lookup cost two round trips, which is why `CNAME` is banned at a zone apex.

- **local-zone types, especially static and refuse** - `static` means *this zone is mine and it is complete*, so a name inside it with no data gets `NXDOMAIN`. That single word is what earns the fifth check. `local-zone: "." refuse` on the authoritative server is what makes it non-recursive: ask it about `example.com` and it says `REFUSED`, which is a fundamentally different statement from `NXDOMAIN`.
- **stub-zone** - the resolver's only piece of knowledge about `lab.test`: send it to `10.10.2.2` and treat that answer as final. `stub-first: no` means that if the authoritative server is down, the lookup fails rather than quietly falling back to the public internet — a real feature, and a real footgun in the other direction, since the fallback would let an outsider answer for a name that is yours.
- **Authoritative vs recursive, in the flags** - `aa` is the server saying *this came from my own data*. `ra` is *I am willing to go and find things*. The authoritative half has `aa` and no `ra`; the recursive half has `ra` and no `aa`. When someone reports that DNS returns the wrong address, the flags on their answer tell you immediately whether you are debugging your data or somebody's cache.
- **TTL and negative caching** - a TTL is not a performance setting, it is **a promise about how long you are willing to be wrong**. The resolver serves a copy and counts down; the authoritative server serves the record and its TTL never moves. `www.lab.test` is deliberately given a 30 second TTL so you can watch a full expiry inside a minute, and `--hard` shows you the same record at 86400, where a completed, correct migration keeps serving the old address for a day with nothing logging an error.
- **ss -u1pn** - DNS is UDP first, so `-u` matters; `-l` for listening, `-p` for the process, `-n` to stop it turning `53` into `domain` and hiding the number you are grepping for. Run it inside each namespace and both servers own port 53 at once without a collision, which is worth seeing once, because "port 53 is already in use" is the error that sends people rebooting.
- **lab-dnsq (the payload)** - asks both servers the same question and prints status, flags, TTL and answer side by side. There is no daemon to write today, so the payload is instead a question you can ask repeatedly: run it twice and only one of the two TTLs will have moved. That is the cache, visible, with no tooling.

Verify

Checked automatically:

- ☐ something is listening on port 53 in the auth namespace
- ☐ the zone answers with a SOA
- ☐ an A record resolves from the client
- ☐ the resolver namespace also answers for the zone
- ☐ an unknown name returns `NXDOMAIN` not an error

Only you can confirm:

- ☐ you lowered a TTL and watched the cache expire
- ☐ you can name what `REFUSED`, `SERVFAIL` and `NXDOMAIN` each tell you

Run the automatic checks with:

```
sudo ./days/day08/verify.sh
```

Root is required today, for the same reason as Days 06 and 07. Every check reads from inside a namespace, and entering one is privileged. Without root you get `SKIP` on every line, and `SKIP` is not a pass. `verify.sh` declares this with `vl_need_root` so a non-root run says so plainly rather than looking like five failures.

Run it wherever you like - including your own laptop. There is no VM today and no lab-VM guard: everything happens inside network namespaces, which cannot see or affect your machine's own network stack. Your installed resolver keeps running on port 53 throughout, untouched, because the two servers here are in namespaces of their own.

A GitHub runner can build this environment for real, so CI executes `verify.sh` on every push. If the checks pass in CI, they pass because the zone actually answered — not because a linter approved of the script.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>lab-dnsq.sh</code>	The payload: asks both servers the same question and prints status, flags, TTL and answer in two aligned lines. Run it twice to see the cache. Its exit status reflects DNS, so it works in a loop or a pipeline.	yes
<code>setup.sh</code>	Builds Day 06's topology if it is missing, stops Day 07's nameserver so port 53 is free, writes both <code>unbound</code> configs, validates them with <code>unbound-checkconf</code> , starts one server in each namespace, then proves the zone answers, the resolver agrees, an unknown name is <code>NXDOMAIN</code> , and the two TTLs behave differently. Idempotent - run it twice and the second run is a clean restart.	yes
<code>explore-dns-server.sh</code>	Read-only tour in twelve sections: two servers on the same port, the SOA timers, the NS record and its address, the same question to both, the flags, the TTL falling, a CNAME answering with a name instead of an address, negative caching, <code>REFUSED</code> vs <code>NXDOMAIN</code> , a query with no <code>@server</code> at all, the servers' own query logs, and both configs side by side.	yes
<code>break-and-fix.sh</code>	Three failures, each repaired: a stub pointing where nothing answers, an authoritative server stopped while its cache keeps answering correctly, and a client denied by <code>access-control</code> . <code>--hard</code> adds two that pass <code>unbound-checkconf</code> and print no error - an 86400 second TTL during a migration, and a missing trailing dot.	yes
<code>teardown.sh</code>	Stops both servers, shows the client timing out against a nameserver that no longer exists, and removes today's configs and the payload. Leaves the query logs, and leaves Day 06's namespaces up because Days 09 and 18 need them.	yes

All five need root, because all five work inside namespaces.

Read them before you run them. They are commented as teaching material rather than production code — the comments are half the day.

Run it on the lab

1. No VM today

Day 08 runs on **the machine you are reading this on**, inside Day 06's namespaces. Nothing to boot, nothing to copy, no memory to budget. If a VM is still running from Day 05 and you want the RAM back:

```
./lab/lab.sh down node1          # deletes the VM and its disk
```

The base image stays cached, so `up` next time takes a minute, not a download.

2. Check the tools

Today needs `unbound` itself, plus the client tools:

```
command -v ip dig unbound unbound-checkconf ss
```

On RHEL-family systems: `sudo dnf install -y unbound bind-utils iproute`. On Debian or Ubuntu: `sudo apt-get install -y unbound dnsutils iproute2`. `setup.sh` checks all five before it changes anything and stops with the right command for your distribution if any are missing.

If your distribution starts `unbound` as a service on install, leave it alone. Today's instances run from explicit config files inside namespaces and never touch the system one.

3. Build both servers

```
sudo ./days/day08/scripts/setup.sh
```

It rebuilds Day 06's namespaces first if they are gone — they live in the running kernel and never survive a reboot, so that is expected rather than a fault. It also stops Day 07's Python nameserver, which is holding `10.10.1.2:53`; without that step `unbound` would fail with "address already in use", and you would go looking for a bug in today's configuration instead of yesterday's leftovers.

4. Look at what you built, then break it

```
sudo lab-dnsq www.lab.test      # run it twice - one TTL falls, one does not
sudo lab-dnsq web.lab.test      # a CNAME: an answer that is a name, not an address
sudo lab-dnsq lab.test SOA

sudo ./days/day08/scripts/explore-dns-server.sh
sudo ./days/day08/scripts/break-and-fix.sh
sudo ./days/day08/scripts/break-and-fix.sh --hard
```

Do the `--hard` pass. The three ordinary failures announce themselves; the two hard ones leave a service that is running perfectly and answering wrongly, and those are the ones that reach production.

The manual check about a TTL is worth doing by hand rather than reading about:

```
for i in 1 2 3 4 5 6 7 8; do sudo ip netns exec client dig +noall +answer A www.lab.test @10.10.1.2;
sleep 5; done
```

One line on purpose. Eight answers five seconds apart: the TTL counts down 30, 25, 20 ... and then jumps back to 30. That jump is the cache entry expiring and the resolver going to ask the authoritative server again. Nothing announces it. That number is the only evidence.

If every line shows the same TTL, you are not reading a cache - you are reading the authoritative server. Check the address in the command.

5. Check yourself

```
sudo ./days/day08/verify.sh
```

Five checks and two for you to judge. Expect `5 passed, 0 failed, 2 for you to judge`.

6. Optional cleanup

```
sudo ./days/day08/scripts/teardown.sh
```

You do not have to. Nothing in Days 09-20 conflicts with two `unbound` instances in namespaces, and Day 09 is considerably more interesting with a DNS server to point `tcpdump` at. Teardown deliberately leaves Day 06's namespaces up for the same reason.

Notes

Keep your own notes here. What broke, what the error actually said, and what fixed it — that is the part you will come back for.

Two things worth writing down in your own words.

`.test` is a reserved top-level domain, and `unbound` ships a built-in `local-zone` for it that returns `NXDOMAIN` for everything underneath — *before* your `stub-zone` is ever consulted. Your config is right, the authoritative server is up and answering, and the resolver still says the name does not exist. The one clue is the `SOA` in the reply, which names `localhost.` instead of your nameserver: that is a server answering from its own built-in data, not from your zone. `local-zone: "lab.test." nodefault` removes the built-in zone for that one name. Reserved names — `.test`, `.invalid`, `.localhost`, `.example` — are the correct choice for a lab precisely because they can never collide with the real internet, and resolvers know it, which is the same reason they refuse to look them up.

And: `private-domain`. `unbound` discards answers containing RFC1918 addresses by default, to protect you from DNS rebinding attacks. This lab is entirely built on `10.10.0.0/16`, so without `private-domain: "lab.test."` in the resolver config every reply from the authoritative server would come back stripped — `NOERROR`, zero answers, no error logged. It is the most confusing failure in today's stack and it is a security feature working exactly as designed.

Next up: **Day 09 — Packet-level debugging.**

days/day09/README.md

This day is provable, not descriptive: **5 automatic checks** and **2 judgement calls** in `verify.sh`.

Prove where a packet stops, instead of guessing.

PHASE	THE NETWORK
Runs on	Host: network namespaces
Memory	0 MB (no VM at all)
Verified by	lint + CI

Why this day exists

Two people look at the same outage and disagree. One says the request was never sent. The other says it was sent and never answered. Both are certain, neither has evidence, and the call goes on for an hour.

A capture ends that argument in ten seconds, because it is taken in the middle of the path and belongs to neither side.

Today builds nothing. Day 06's four namespaces already exist and already work. What today adds is the ability to prove things about them: which interface a packet leaves by, whether it arrived at the hop in the middle, whether anything was listening when it got there, and why a path that answers ping perfectly can still lose every large transfer.

That last one is the day's real lesson. The failures worth practising are not the ones that print an error.

What you will work with

- `ip route get DEST` - the kernel's routing decision for one destination, not your reading of the table
- `tcpdump -ni IFACE, -c N, -w FILE, -nr FILE` - capture live, capture to a file, re-read it afterwards
- capture filters - `icmp`, `host 10.10.2.2`, `port 53`
- `ip netns exec NS tcpdump ...` - capturing inside a namespace, where the traffic actually is
- `ss -ulpn`, `ss -s` - who is listening, and how many sockets exist
- `ip -s link show` - packet and drop counters that were kept while you were not watching
- `ip neigh` - the MAC behind an address, and what `PERMANENT` means there
- `ping -M do -s SIZE` - a packet that refuses to be fragmented, which is how you find an MTU
- `ping -t 1` - a TTL small enough to die at a known hop

Verify

Checked automatically:

- ☐ all four namespaces from Day 06 are up
- ☐ the client reaches the auth namespace at 10.10.2.2
- ☐ `ip route get` names veth-cl as the way out of the client
- ☐ the saved capture holds at least one ICMP packet
- ☐ lab-trace reports requests and replies on the router leg

Only you can confirm:

- * ☐ you attributed a dropped packet to a specific hop, using a capture rather than a guess
- * ☐ you lowered an MTU, broke a large transfer, and read the cause off the interface

Run the automatic checks with:

```
sudo ./days/day09/verify.sh
```

A GitHub runner can build this environment for real, so CI executes `verify.sh` on every push.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>scripts/setup.sh</code>	Makes sure Day 06's topology is up, installs <code>lab-trace</code> , takes one capture to prove capturing works	yes
<code>scripts/lab-trace.sh</code>	The payload. Route, wire and reply for one destination, in one answer	yes
<code>scripts/explore-packets.sh</code>	Twelve looks at the same network, each with a tool that answers one kind of question	yes
<code>scripts/break-and-fix.sh</code>	Five failures and the shape of evidence each one leaves. <code>--hard</code> adds the two that stay silent	yes
<code>scripts/teardown.sh</code>	Removes <code>lab-trace</code> and the captures, leaves the network alone	yes

Read them before you run them. They are commented as teaching material rather than production code - the comments are half the day.

Run it on the lab

1. No VM today

Everything here happens in network namespaces on the machine you are reading this on. Nothing is installed system-wide except one script under `/usr/local/bin`, and nothing survives a reboot. Run it wherever you like - including your own laptop.

If a VM from an earlier day is still running and you want the memory back:

```
./lab/lab.sh down node1
```

2. Check what you already have

Today reuses Day 06's four namespaces. This is a diagram, not something to paste into a shell:

```
client 10.10.0.2 --- 10.10.0.1 router 10.10.1.1 --- 10.10.1.2 resolver
                        router 10.10.2.1 --- 10.10.2.2 auth
```

If they are gone - and a reboot always removes them - `setup.sh` rebuilds them for you by calling Day 06's setup. That is expected, not a fault.

Day 08's DNS servers are optional. They are not required, but DNS is more interesting to capture than ping, so leave them running if you have them.

3. Build today's view of it

```
sudo ./days/day09/scripts/setup.sh
```

It checks for `ip`, `tcpdump`, `ss` and `ping`, makes sure the topology is up, asks the kernel which interface the client uses, installs `lab-trace`, and then takes one real capture and reads it back. That last step matters: a day about evidence should not start by assuming its own tools work.

If `tcpdump` is missing:

```
sudo dnf install -y tcpdump iproute iputils          # RHEL family
sudo apt-get install -y tcpdump iproute2 iputils-ping # Debian/Ubuntu
```

4. Look at what you built, then break it

```
sudo lab-trace          # the path to 10.10.2.2, end to end
sudo lab-trace 10.10.9.9 # a destination that does not exist
sudo ./days/day09/scripts/explore-packets.sh
```

Then the five failures:

```
sudo ./days/day09/scripts/break-and-fix.sh
sudo ./days/day09/scripts/break-and-fix.sh --hard
```

The first three are loud: an interface down, no route, and a far end with no way back. Read the third one twice - requests arrive, replies never come, and the sender sees exactly what it would see if the destination were switched off.

`--hard` adds the two that stay quiet. An MTU of 1280 that lets every ping through and refuses a 1400-byte packet, and a permanent ARP entry that hands correctly routed packets to a MAC address nobody owns.

Do the MTU one by hand as well, because it is the manual check:

```
sudo ip -n client link set veth-cl mtu 1280
sudo ip netns exec client ping -c2 -W2 10.10.2.2
sudo ip netns exec client ping -c1 -W2 -M do -s 1400 10.10.2.2
sudo ip netns exec client ip -brief link show veth-cl
sudo ip -n client link set veth-cl mtu 1500
```

Small packets fine, large packets gone, and the only place the answer is written down is the interface itself.

5. Check yourself

```
sudo ./days/day09/verify.sh
```

Five automatic checks and two that are yours. The checks need root, because capturing and entering namespaces both do.

6. Optional cleanup

```
sudo ./days/day09/scripts/teardown.sh
```

That removes `lab-trace`, stops any `tcpdump` still running in a namespace, and deletes `/var/log/lab-trace`. The namespaces stay - they belong to Day 06.

Notes

The order of the tools is the whole method, and it is not the order people reach for them in:

1. `ip route get` first. It tells you where to capture, and it catches the one failure a capture can never show you - a packet that was never sent.
2. `tcpdump` in the middle. Not at the sender, where "I sent it" is already believed, and not at the receiver, where "nothing arrived" is already believed.
3. `ss` at the far end. A packet can arrive perfectly and still be refused, because nothing was listening.
4. Counters last. `ip -s link` was counting drops while you were arguing.

Two things worth remembering after today. `tcpdump` has to be running before the traffic exists - an empty capture usually means a late `tcpdump`, not a dead network. And a capture with no `-c` limit and no rotation will fill a disk quietly, which turns the machine you were debugging into a second incident.

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Next up: **Day 10 - TLS on the wire and a private CA.**

DAY 10 Day 10 - TLS on the wire and a private CA

days/day10/README.md

This day is provable, not descriptive: **7 automatic checks** and **2 judgement calls** in `verify.sh`.

Run your own certificate authority and understand what a client actually verifies.

PHASE	THE NETWORK
Runs on	Host: openssl only, no namespaces
Memory	0 MB (no VM at all)
Verified by	lint + CI

Why this day exists

"Invalid certificate" almost never means the certificate is invalid.

Today you issue three certificates from one authority, a minute apart. All three are properly signed by a CA the client trusts. One works. One is refused everywhere because it is out of date. One is refused everywhere because it carries a different name. Nothing about the second and third is corrupt, and no amount of re-issuing, restarting or reinstalling fixes them - only understanding which of three separate questions failed.

Those three questions are the day:

1. Does the chain lead to something this client was told to trust?
2. Is the certificate valid right now?
3. Does the name I asked for appear in it?

They fail independently, they have different fixes, and two of them are not the server's fault at all. Day 17 puts this CA behind a real web server, so the CA you build here is reused.

What you will work with

- `openssl req -x509 -newkey` - a self-signed CA in one command, and why self-signed is what an authority is
- `basicConstraints=CA:TRUE`, `keyUsage=keyCertSign` - what makes a certificate allowed to sign others
- `openssl req -newkey` plus `openssl x509 -req -CA` - a signing request, and the signature over it
- `subjectAltName` - the field that decides hostname matching, and the reason `CN` no longer does
- `openssl verify -CAfile` - checks the chain and the dates, and *never* the hostname
- `openssl s_client -connect -servername -verify_hostname -CAfile` - a real handshake, checked properly
- `Verify return code: 0 (ok)` - the only line in that output that means trusted
- `openssl x509 -noout -modulus | openssl md5` - whether a key and a certificate are actually a pair
- file modes on private keys, and why `0600` on the CA key is not optional

Verify

Checked automatically:

- ☐ a CA certificate exists and is marked as a CA
- ☐ a server certificate carries a `subjectAltName` for `www.lab.test`

- `❑` the server certificate verifies against the CA
- `❑` the private key matches the certificate
- `❑` the CA key is readable only by root
- `❑` a handshake on 127.0.0.1:4433 verifies with the CA and the hostname
- `❑` the same handshake is refused when no CA is supplied

Only you can confirm:

- `❑` you can read an `s_client` chain and say why it was trusted or refused
- `❑` you made verification fail on purpose, by hostname and by expiry

Run the automatic checks with:

```
sudo ./days/day10/verify.sh
```

A GitHub runner can build this environment for real, so CI executes `verify.sh` on every push.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>scripts/setup.sh</code>	Builds the CA, issues three certificates from it, starts a TLS server, and checks one handshake both ways	yes
<code>scripts/lab-tls.sh</code>	The payload. Separates chain, dates and hostname, and says which one failed	yes
<code>scripts/explore-tls.sh</code>	Twelve looks at three certificates that differ only by a name and a date	yes
<code>scripts/break-and-fix.sh</code>	Five certificate failures and the line that identifies each. <code>--hard</code> adds the two that get misdiagnosed	yes
<code>scripts/teardown.sh</code>	Stops the server, frees the port, removes the CA and its keys	yes

Read them before you run them. They are commented as teaching material rather than production code - the comments are half the day.

Run it on the lab

1. No VM today

This day needs `openssl` and nothing else - no namespaces, no VM, no root-owned network state. TLS does not care about topology; it cares about names, dates and signatures, and all three can be got wrong on one machine talking to itself. Run it wherever you like - including your own laptop.

If a VM from an earlier day is still running and you want the memory back:

```
./lab/lab.sh down node1
```

2. Check what you already have

```
openssl version
date -u
```

Both matter. The second one matters more than people expect: TLS is a dated protocol, and a machine whose clock is wrong reports every certificate on earth as expired or not-yet-valid.

3. Build it

```
sudo ./days/day10/scripts/setup.sh
```

That creates `/etc/lab-tls` with mode `0700`, a self-signed CA valid for ten years, three leaf certificates signed by it, and a TLS server on `127.0.0.1:4433`. It then makes one handshake with `-CAfile` and one without, so you see both answers before you read a word of explanation.

To start over from nothing:

```
sudo ./days/day10/scripts/setup.sh --fresh
```

What you end up with:

<code>/etc/lab-tls/ca.crt</code>	self-signed, CA:TRUE, the only thing to trust
<code>/etc/lab-tls/server.crt</code>	good: SAN <code>www.lab.test</code> , valid a year
<code>/etc/lab-tls/expired.crt</code>	same CA, same name, valid until yesterday
<code>/etc/lab-tls/wrongname.crt</code>	same CA, valid dates, SAN <code>other.lab.test</code>

4. Look at what you built, then break it

```
sudo lab-tls
sudo lab-tls 127.0.0.1:4433 other.lab.test
sudo ./days/day10/scripts/explore-tls.sh
```

The tour is worth reading slowly at step 6, where `openssl verify` accepts `wrongname.crt` without complaint. That is correct behaviour and it surprises everyone: `verify` checks the chain and the dates. It is never told what hostname you wanted, so it cannot object to it. Hostname matching happens in the client, later, and that split is why a certificate can pass every check you thought to run and still be refused.

Then the five failures:

```
sudo ./days/day10/scripts/break-and-fix.sh
sudo ./days/day10/scripts/break-and-fix.sh --hard
```

`--hard` adds the two that get blamed on the wrong thing: a certificate and key that were never a pair, which is the only failure here that stops the server from starting, and the same certificate verifying for one client and failing for another because trust lives in the client's own store.

Do the two manual ones by hand as well, because that is what the `you` items ask:

```
# fail by hostname: genuine certificate, name you did not ask for
echo | sudo openssl s_client -connect 127.0.0.1:4433 \
  -servername other.lab.test -verify_hostname other.lab.test \
  -CAfile /etc/lab-tls/ca.crt 2>&1 | grep -E "verify error|Verify return"

# fail by expiry: same CA, same name, yesterday's dates
sudo openssl verify -CAfile /etc/lab-tls/ca.crt /etc/lab-tls/expired.crt
sudo openssl x509 -in /etc/lab-tls/expired.crt -noout -dates
```

Say out loud which of the three questions each one failed. If you can do that from the output alone, the day is done.

5. Check yourself

```
sudo ./days/day10/verify.sh
```

Seven automatic checks and two that are yours. The last automatic one is deliberately a *negative* check - a handshake with no `-CAfile` must be refused. A lab where everything passes has not tested trust at all.

6. Optional cleanup

```
sudo ./days/day10/scripts/teardown.sh
```

That stops the server, proves port 4433 is free, and deletes `/etc/lab-tls` including the CA key.

If you plan to do Day 17 soon, you may prefer to leave the CA in place - Day 17 puts this same authority behind a real web server. It can also rebuild it from scratch, so either choice is fine.

Notes

The three questions again, because they are the entire day and they are the thing to remember in an incident:

1. **Chain** - `unable to get local issuer certificate`. The client's problem. Nothing is wrong with the server, and the fix is on the client: supply the CA, or install it in that client's store.
2. **Dates** - `certificate has expired`. Check `date -u` *before* re-issuing. A wrong clock reports every certificate as bad, and re-issuing under a wrong clock produces a certificate that is broken everywhere else.
3. **Hostname** - `Hostname mismatch`. A genuine, trusted, in-date certificate for a name you did not ask for. Read the `subjectAltName`, not the `CN` - clients stopped honouring `CN` years ago.

Two more worth keeping. Trust is a property of the client, not of the certificate: openssl, curl, Java, Python and Firefox each have their own store, which is why "it works in my browser" is a real observation rather than an excuse. And a certificate and its key are a pair you can check in two seconds with matching modulus hashes - run that first when a service dies immediately after a renewal, because a key mismatch is the one failure that stops the server rather than the client.

Deleting a CA key is final. Everything it signed becomes unrenovable, which is an outage with a date on it rather than an immediate one.

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Next up: **Day 11 - firewalld, and the nftables underneath it.**

docs/curriculum.md#phase-3

PHASE 3

Hardening and configuration management

Days 11–15

11	firewalld, and the nftables underneath it	67
12	SSH hardening, bastions and fail2ban	72
13	SELinux: contexts, booleans and denial triage	77
14	Ansible fundamentals	82
15	Ansible roles: your hardening baseline	87

DAY 11 Day 11 - firewalld, and the nftables underneath it

days/day11/README.md

This day is provable, not descriptive: **5 automatic checks** and **1 judgement calls** in `verify.sh`.

Close everything, open only what you need, and see the kernel rules your commands produced.

PHASE	HARDENING AND CONFIGURATION MANAGEMENT
Runs on	VM: node1
Memory	~2 GB (one VM)
Verified by	lint + your lab

Why this day exists

Most people learn the front end and never look underneath, so they cannot debug it when the front end lies.

And it does lie - not by being wrong, but by answering a narrower question than the one you asked. `firewall-cmd --query-port=8080/tcp` tells you what firewalld intends. It does not know whether anything is listening, whether another program wrote an nftables rule that runs first, or whether the change you made is in effect or merely on disk. All three of those produce a service that is "open" and unreachable.

So today you build both halves - a real service and the policy in front of it - and then you learn to ask the three questions separately: is something listening, does the policy allow it, and did that intent reach the kernel.

What you will work with

- `firewall-cmd --list-all` and `--zone` - inspect the active firewalld policy instead of assuming which zone applies. Interfaces and sources are assigned to zones, and a perfectly written rule in the wrong zone protects nothing.
- `--add-port`, `--add-service`, `--permanent`, and `--reload` - distinguish the live runtime policy from the saved policy used after a reboot. You will make temporary changes, persist the ones you intend to keep, and prove that a reload produces the expected result.
- **Rich rules** - express conditions that a simple open port cannot, including source addresses, logging, rejection, and rate limits. They are still firewalld policy, but precise enough to document who is allowed to reach what.
- `nft list ruleset` - look below firewalld at the nftables rules it generated. Firewalld is the manager; nftables is the packet-filtering machinery the kernel actually evaluates.
- **nftables chains, hooks, and priorities** - follow a packet through the base chains attached to kernel hooks and understand why rule order is more than top-to-bottom text. Priorities decide which chains see a packet first.
- `ss -tulpn` - confirm which processes are listening and on which addresses before blaming the firewall. A closed service and a blocked service can look identical from another host, so verify the listener first and exposure second.

Verify

Checked automatically:

- ☐ firewalld is running and enabled

- ☐ your service port is open
- ☐ the rule is permanent, not runtime only
- ☐ firewalld built a real nftables table
- ☐ the default zone is not trusted

Only you can confirm:

- ☐ you reloaded and the rules survived, and you can point to each chain

Run the automatic checks with:

```
sudo ./days/day11/verify.sh
```

CI can only lint this day. Nothing on a GitHub runner has SELinux, firewalld, systemd units you control, or a second host to reach over SSH - so the checks below are proven by running `verify.sh` on your own lab, and nowhere else.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>setup.sh</code>	Starts and enables firewalld, forces the default zone to <code>public</code> , installs a real web service on 8080, opens the port permanently and reloads, adds one rich rule, and shows the nftables table it all produced. Idempotent.	yes
<code>lab-fw.sh</code>	The payload. Answers the three questions side by side - what is listening, what the policy intends, what the kernel will do - and names the mismatch when they disagree. Installed as <code>/usr/local/bin/lab-fw</code> .	yes
<code>explore-firewall.sh</code>	Twelve read-only stops: zones, services as named port lists, runtime versus permanent, rich rules, then down into <code>nft</code> chains, hooks and priorities.	yes
<code>break-and-fix.sh</code>	Three failures and their repairs: a change that vanishes at the next reload, an open port with nothing behind it, and a zone that enforces nothing. <code>--hard</code> adds the two that get misdiagnosed.	yes
<code>teardown.sh</code>	Stops the service <i>before</i> closing its port, removes the port and rich rule from both configurations and proves it, then deletes the unit and payloads. Leaves firewalld itself running.	yes

Read them before you run them. They are commented as teaching material rather than production code - the comments are half the day.

Run it on the lab

1. On your laptop, bring up node1

Day 11 runs on `node1`, the same VM as Days 02 to 05. It needs no extra disk and nothing from those days.

```
./lab/lab.sh status          # what is already running?
./lab/lab.sh up node1        # 2 GB, about a minute
```

Days 06 to 10 needed no VM at all, so if you tore `node1` down after Day 05 it is gone along with its disk. A rebuilt `node1` is a blank Rocky image.

2. Copy the repo onto the VM

```
./lab/lab.sh push node1      # carries both days/ and lab/
./lab/lab.sh ssh node1
```

Both directories have to travel: the day scripts source `lab/on-lab-vm.sh`, and without it every one of them refuses to run.

3. Work through the day on the VM

Check the machine before typing anything that changes state - today's scripts change the default zone and open a port:

```
hostname                # must print: node1
cd ~/lab/days/day11
```

The Rocky 9 cloud image ships `firewalld` but not always the rest, so install today's tools first:

```
sudo dnf install -y firewalld nftables iproute curl
```

`nft` is the one people are missing - `firewalld` works without the command-line tool because it talks to the kernel directly, which is exactly why so few people ever look underneath.

Look at the defaults before you change them:

```
sudo firewall-cmd --state
sudo firewall-cmd --get-default-zone
sudo firewall-cmd --list-all
sudo nft list tables
```

On an untouched image the default zone is `public`, `--list-all` shows `services: cockpit dhcpv6-client ssh` and no ports, and `nft list tables` already shows `table inet firewalld`. Nothing you have done yet - `firewalld` wrote that table the moment it started.

In this order:

```
less scripts/setup.sh      # 1. read it BEFORE running it
sudo ./scripts/setup.sh    # 2. service, zone, open port, rich rule
```

It prints each step, and step 6 is the one to slow down on: the chain list is your own `firewall-cmd` commands, translated.

```
curl -s http://127.0.0.1:8080/ # 3. prove the service is real
sudo /usr/local/bin/lab-fw
sudo /usr/local/bin/lab-fw 8080
```

`/usr/local/bin/lab-fw` with no argument ends with a section called "where the two disagree": ports open with nothing behind them, and listeners the firewall is blocking. On a tidy machine it prints `none`. On a machine that three people have administered, it rarely does.

```
sudo ./scripts/explore-firewall.sh # 4. the tour
```

Stop at stop 9. type `filter hook input priority 0` is the whole story of how `firewalld` gets its say, and it is also the reason failure 4 below works. Read `man 5 firewalld.richlanguage` and `man 8 nft` on the VM for anything you cannot explain.

Now the manual check, by hand, because that is what the `you` item asks:

```

sudo firewall-cmd --add-port=9099/tcp          # runtime only, on purpose
sudo firewall-cmd --list-ports                # 8080 and 9099
sudo firewall-cmd --reload
sudo firewall-cmd --list-ports                # 9099 is gone. 8080 survived
sudo nft list table inet firewalld | grep -E 'chain |hook '

```

That is the day in five lines: one rule survived the reload and one did not, and you can point at the chain each one lives in. Say out loud which chain accepts 8080 before you move on.

```

sudo ./scripts/break-and-fix.sh              # 5. three failures, three fixes
sudo ./scripts/break-and-fix.sh --hard       # 6. the two that get misdiagnosed

```

Failure 4 is the one worth the time: another nftables table hooked at priority `-300` drops the packet before firewalld's chains ever see it, and `firewall-cmd --query-port` keeps answering `yes` throughout. Lower priority number runs first. That is how Docker and half the cluster tooling in the world silently overrule a firewall that reports itself healthy.

Failure 5 is deliberately described and not executed - it removes SSH and reloads, which would end your session with no console to rescue you. Read it and remember `--timeout=120`.

4. Check yourself

```

sudo ./days/day11/verify.sh

```

Five automatic checks and one that is yours. Note what the fifth one is really doing: `the default zone is not trusted` is not a style rule. A machine in the `trusted` zone runs firewalld, reports active and enabled, and filters nothing at all.

5. Optional cleanup

```

sudo ./scripts/teardown.sh

```

That stops the service first, then closes the port and removes the rich rule from both configurations, and checks each removal rather than assuming it. firewalld stays running - Days 12 and 13 assume a machine that still filters.

6. Back on your laptop

```

exit
./lab/lab.sh down node1          # frees the memory; takes the disk with it

```

Day 12 uses `node1` again, so you can leave it running if you have the RAM.

Notes

The three questions, in the order that saves the most time:

1. **Is anything listening?** `ss -tulpn`. If the answer is nothing, or `127.0.0.1`, no firewall change will ever help. A process bound to loopback refuses the connection itself.
2. **Does the policy allow it?** `firewall-cmd --list-all`, and then both `--list-ports` and `--permanent --list-ports`. If those two disagree, someone forgot `--permanent` or forgot `--reload`, and the fix depends on which.

3. **Did that reach the kernel?** `nft list ruleset` . If firewalld says open, something is listening, and the connection still fails, the answer is here - usually another table with a lower priority number.

Two more worth keeping. `--add-service=http` is exactly `--add-port=80/tcp` with a name a human can read, so there is nothing to be afraid of in service files - `firewall-cmd --info-service=ssh` prints the whole definition. And `--timeout=120` is the single best habit on this day: a runtime rule that removes itself after two minutes, so a mistake made over SSH repairs itself while you are still logged in.

A firewall you cannot see working is a firewall you cannot debug. That is why the day ends in `nft` rather than in `firewall-cmd` .

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Next up: **Day 12 - SSH hardening, bastions and fail2ban.**

DAY 12 Day 12 - SSH hardening, bastions and fail2ban

days/day12/README.md

This day is provable, not descriptive: **5 automatic checks** and **2 judgement calls** in `verify.sh`.

Keys only, one group allowed in, a jail on the log, and a jump host in front.

PHASE	HARDENING AND CONFIGURATION MANAGEMENT
Runs on	VM: node1
Memory	~2 GB (one VM)
Verified by	lint + your lab

Why this day exists

SSH is the door. It is also the service most often left with defaults that a scanner finds in minutes.

But the reason this day takes a whole day is not the hardening - that is six lines in a file. It is that SSH is the one service where a configuration mistake removes your ability to fix the configuration. Every other day in this course, a bad change means a broken service. Here it means a broken service *and* no way in.

So the work is really about a habit: read the effective policy rather than the file, test the policy for a named user rather than by logging out, and validate before loading. `sshd -T` and `sshd -T -C user=you` are the two commands that turn this from a nervous edit into a boring one.

What you will work with

- `sshd -T` - print the effective server configuration after includes, defaults, and precedence have been resolved. Reading this output is safer than trusting a file that may not be the value sshd actually uses; `sshd -T -C user=...` goes further and evaluates policy for one named connection.
- `/etc/ssh/sshd_config.d/` - keep lab hardening in a focused drop-in instead of rewriting the distribution file. On Rocky 9 the include appears near the top, and sshd keeps the first value it reads for most keywords, so precedence is deliberately part of the exercise.
- `PasswordAuthentication` **and** `PermitRootLogin` - separate password access from root access rather than treating “SSH is hardened” as one switch. `prohibit-password` still permits root keys, while `no` forbids root completely; you should be able to explain which policy you chose.
- `AllowUsers` **and** `AllowGroups` - add an explicit admission list on top of valid credentials. The lab uses a group so future access changes happen with group membership rather than another risky sshd configuration edit.
- `ProxyJump` **and** `~/.ssh/config` - reach a destination through a bastion and then encode both hops as reusable client configuration. `ssh -G HOST` shows the client's effective choices just as `sshd -T` shows the server's.
- `fail2ban-client status sshd` - verify that fail2ban is reading SSH failures and maintaining the jail that can add firewall bans. You will deliberately ban and unban an address so a refusal caused by fail2ban is distinguishable from a broken sshd configuration.

Verify

Checked automatically:

- ☐ password authentication is off
- ☐ root cannot log in with a password
- ☐ login is restricted to a named user or group
- ☐ the config is valid
- ☐ fail2ban is watching sshd

Only you can confirm:

- ☐ you reached node1 with ProxyJump through a bastion, and you can explain each hop
- ☐ you triggered a ban on purpose and then unbanned yourself

Run the automatic checks with:

```
sudo ./days/day12/verify.sh
```

CI can only lint this day. Nothing on a GitHub runner has SELinux, firewalld, systemd units you control, or a second host to reach over SSH - so the checks below are proven by running `verify.sh` on your own lab, and nowhere else.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>setup.sh</code>	Confirms you have a key before it disables passwords, creates the <code>labssh</code> group and puts you in it, writes the hardening drop-in, validates with <code>sshd -t</code> and reloads, then configures a fail2ban jail with a deliberately short ban. Idempotent.	yes
<code>lab-ssh.sh</code>	The payload. Shows the files, the effective config, and where they disagree, plus the jail and the refusals in the log. With a username it answers "would this person get in, and why". Installed as <code>/usr/local/bin/lab-ssh</code> .	yes
<code>explore-ssh.sh</code>	Twelve read-only stops: include order, drop-in precedence, the four keywords that decide access, <code>-C</code> for one user, <code>ssh -G</code> on the client side, host key fingerprints, the log lines fail2ban reads, and the jail.	yes
<code>break-and-fix.sh</code>	Three failures and their repairs: a drop-in that never wins, an allow list that excludes you, and banning yourself. <code>--hard</code> adds the Match block that reverses your policy and the one that is not survivable.	yes
<code>teardown.sh</code>	Releases bans first, removes the drop-ins, validates before reloading, and proves the allow list is gone rather than assuming it. Leaves sshd and fail2ban running.	yes

Read them before you run them. They are commented as teaching material rather than production code - the comments are half the day.

Run it on the lab

1. On your laptop, bring up node1

Day 12 runs on `node1`. Day 11 left it running; if you took it down, a rebuilt `node1` is fine - this day needs nothing from Day 11 except the habit of checking what the kernel really has.

```
./lab/lab.sh status      # what is already running?
./lab/lab.sh up node1    # 2 GB, about a minute
```

The curriculum lists this day as `control + node1`, because a bastion needs two hosts. Two VMs use about 3.5 GB. **You do not need the second VM for the automatic checks** - all five run on `node1` alone - and section 3 below gives you a one-host version of the ProxyJump exercise. Bring up `control1` as well only if you have the memory to spare.

2. Copy the repo onto the VM

```
./lab/lab.sh push node1          # carries both days/ and lab/
./lab/lab.sh ssh node1
```

`push` is the step people skip. Editing a file on your laptop changes nothing on the VM until this runs.

3. Work through the day on the VM

Check where you are before changing the service you are connected through:

```
hostname                # must print: node1
cd ~/lab/days/day12
```

fail2ban is not in the base Rocky repositories - it comes from EPEL:

```
sudo dnf install -y epel-release
sudo dnf install -y fail2ban fail2ban-firewalld openssh-server
```

Before you change anything, read what is already there:

```
sudo sshd -T | sort | head -20
sudo grep -nE '^s*Include' /etc/ssh/sshd_config
ls -l /etc/ssh/sshd_config.d/
```

That `Include` line is at the **top** of the file on Rocky 9, and `sshd` keeps the **first** value it reads for most keywords. So a drop-in beats the main file - the opposite of how Apache or nginx behave. Drop-ins also use lexical order, which is why the lab policy is named `00-lab-hardening.conf`: it must be read before Rocky and cloud-init's `50-*` files. The later-file trap is the subject of failure 1.

Open a second terminal to your laptop now and leave it connected to `node1`. Everything below is safe, but the habit is the point: never make the only session you have the one you are experimenting on.

In this order:

```
less scripts/setup.sh          # 1. read it BEFORE running it
sudo ./scripts/setup.sh        # 2. key check, group, drop-in, reload, jail
```

Step 1 of the script is the one to notice: it refuses to continue if your account has no `authorized_keys`. Disabling passwords on an account with no key is the single most common way people lose a cloud VM.

```
sudo /usr/local/bin/lab-ssh
sudo /usr/local/bin/lab-ssh "$USER"
sudo /usr/local/bin/lab-ssh root
```

Type the full path. `sudo` on Rocky uses its own `secure_path`, which does not include `/usr/local/bin`, so `sudo lab-ssh` will tell you the command does not exist even though it is installed.

Compare the last two. Your account is in `labssh` and gets in; `root` is not, and the verdict says so - even though `PermitRootLogin prohibit-password` suggests root has a way in. The allow list is checked first.

```
sudo ./scripts/explore-ssh.sh    # 3. the tour
```

Stop at stop 7. `sshd -T -C user=NAME` is the command that makes this day safe: it answers "what would sshd decide for this person" without requiring you to log out and find out.

Now the ProxyJump exercise, which is the first `you` item. **With two VMs**, use `control` as the bastion:

```
# on your Laptop
ssh -J lab@control lab@node1
```

With one VM, you can still prove you understand the mechanic, because a jump host is just an SSH connection opened through another SSH connection:

```
# on your laptop - node1 is both the jump host and the destination
ssh -J lab@192.168.122.13 lab@127.0.0.1
```

The second hop is made *from* `node1`, so `127.0.0.1` means `node1` itself. Then write it down properly so you never type the flag again:

```
cat >> ~/.ssh/config <<'EOF'

Host node1
    HostName 192.168.122.13
    User lab

Host behind
    HostName 127.0.0.1
    User lab
    ProxyJump node1
EOF

ssh -G behind | grep -E '^(hostname|user|proxyjump) '
ssh behind hostname
```

`ssh -G` prints the client's effective config the way `sshd -T` prints the server's. When a connection lands somewhere you did not expect, that is the first command, not the last.

```
sudo ./scripts/break-and-fix.sh    # 4. three failures, three fixes
sudo ./scripts/break-and-fix.sh --hard # 5. the two that get misdiagnosed
```

Failure 4 is the one worth the time. `sshd -T` reports `passwordauthentication no` while `sshd -T -C user=you` reports `yes`, because a `Match` block re-enabled it for exactly the account an attacker would target. Both commands are correct. An audit that only runs the first one passes a machine that is not hardened.

4. Check yourself

```
sudo ./days/day12/verify.sh
```

Five automatic checks and two that are yours. The second `you` item wants a ban you caused on purpose:

```
sudo fail2ban-client status sshd
sudo fail2ban-client set sshd banip 127.0.0.1
sudo fail2ban-client status sshd    # Currently banned: 1
sudo fail2ban-client set sshd unbanip 127.0.0.1
```

The jail is set to a two-minute ban on purpose. A ten-hour default on a lab machine teaches you nothing except how to rebuild a VM.

5. Optional cleanup

```
sudo ./scripts/teardown.sh
```

It releases bans before touching the config, validates before reloading, and checks that the allow list is really gone. sshd and fail2ban stay - Days 13 and 14 reach this machine over SSH.

6. Back on your laptop

```
exit
./lab/lab.sh down node1          # frees the memory; takes the disk with it
```

Day 13 uses `node1` again, so you can leave it running if you have the RAM.

Notes

The order that keeps you logged in, every time:

1. **Make the credential work first.** Key in `authorized_keys`, user in the allowed group. Build the allow list before you enforce it, never after.
2. **Validate.** `sudo sshd -t` parses the config without loading it. It catches typos, not policy mistakes.
3. **Ask about a person, not a file.** `sudo sshd -T -C user=you,host=localhost,addr=127.0.0.1`. This is the step that catches the policy mistakes `-t` cannot.
4. **Reload, do not restart.** `systemctl reload sshd` re-reads the config and leaves established sessions alone. A restart plus a bad policy ends the session you would have used to fix it.
5. **Prove a new login works from a second terminal** before you close the first one.

Two things worth keeping. `PermitRootLogin prohibit-password` is not `no` - root may still log in with a key, which is a reasonable choice for automation and a surprise in an audit, so decide which one you meant. Some Rocky/OpenSSH builds print the equivalent legacy spelling `permitrootlogin without-password` in `sshd -T`; the setup and verifier accept either spelling because both disable password login for root. Also avoid `producer | grep -q` for validation when `pipefail` is active: an early successful match can close the pipe, give the producer `SIGPIPE`, and make the whole pipeline look like a failure.

A fail2ban ban is a firewall rule, not an SSH setting: when a healthy sshd refuses a healthy client for no visible reason, `fail2ban-client status sshd` is the second thing to check and `sshd -T` is a waste of time.

The difference between someone who hardens SSH confidently and someone who does it once a year with sweaty palms is not knowledge of the keywords. It is having a second terminal open.

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Next up: **Day 13 - SELinux: contexts, booleans and denial triage.**

Day 13 - SELinux: contexts, booleans and denial triage

days/day13/README.md

This day is provable, not descriptive: **7 automatic checks** and **2 judgement calls** in `verify.sh`.

Serve content from a non-default path with SELinux still enforcing.

PHASE	HARDENING AND CONFIGURATION MANAGEMENT
Runs on	VM: node1
Memory	~2 GB (one VM)
Verified by	lint + your lab

Why this day exists

This is the day that separates operators from people who type `setenforce 0`.

The reason that command is so tempting is that it always works. The service comes back, the alert clears, and the ticket closes. What it never does is explain anything - and because the fix is invisible in every configuration file, the same outage returns after the next reboot or the next package update, usually to somebody else.

Denial triage is a procedure, not a talent. Six commands, in a fixed order, and the answer is always one of four things: the mode, a file label, a port label, or a boolean. Once you have run the procedure a few times, SELinux stops being an obstacle and becomes the thing that tells you precisely which access your service was missing.

The work here is deliberately the realistic case: content in `/srv/www` instead of `/usr/share/nginx/html`. Correct owner, correct mode, 403 anyway.

What you will work with

- `getenforce`, `sestatus`, `setenforce` - read the mode before anything else, because every other answer depends on it. `setenforce` changes only the running kernel; `/etc/selinux/config` decides what comes back after a reboot, which is the same runtime-versus-permanent split you met in `firewalld` on Day 11.
- `ls -Z`, `ps -Z`, `id -Z` - see the label on a file, a process and yourself. Contexts are `user:role:type:level`, and for almost all troubleshooting only the **type** matters: `nginx` runs as `httpd_t`, and the question is always whether `httpd_t` may touch the type on the other object.
- `semanage fcontext -a -t` **and** `restorecon` - record what a path's label *should* be, then apply that rule to the files. This pair is the centre of the day: the rule is the reason, and `restorecon` is the enforcement of the reason.
- `chcon` **versus** `semanage` - `chcon` writes a label straight onto the inode and policy never learns about it, so the next relabel silently undoes your fix. Being able to explain that difference out loud is most of what this day is for.
- `semanage port` - ports carry labels too. A valid config on a free port can still fail to bind, and nothing in `nginx -t` or the service log will mention SELinux.

- `getsebool -a`, `setsebool -P`, `semanage boolean -l -C` - booleans are switches the policy author left for you, which makes them always preferable to a custom module. Without `-P` the change is runtime only; `-c` lists exactly what somebody changed on this machine, which is the first thing to read on a host you did not build.
- `ausearch -m avc -ts recent`, `audit2allow`, `sesearch` - the denial record, the rule generator, and the way to ask "is this allowed" without testing it in production. `audit2allow` is a suggestion engine, not an authority: read every rule it writes before loading it.

Verify

Checked automatically:

- ☐ SELinux is enforcing
- ☐ the web root carries a web content label
- ☐ the label rule is permanent, not just a chcon
- ☐ restorecon is a no-op, so labels match policy
- ☐ nginx actually serves the content
- ☐ a boolean you set survives, recorded as permanent
- ☐ a custom policy module is loaded

Only you can confirm:

- ☐ you fixed a denial by relabelling, not by disabling SELinux
- ☐ you built a module from a real AVC with `audit2allow` and read it before loading

Run the automatic checks with:

```
sudo ./days/day13/verify.sh
```

CI can only lint this day. Nothing on a GitHub runner has SELinux, a policy store, or a service whose labels you control - so the checks above are proven by running `verify.sh` on your own lab, and nowhere else.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>setup.sh</code>	Refuses to continue unless SELinux is enforcing, creates <code>/srv/www</code> , adds the fcontext rule and then applies it with <code>restorecon</code> , serves it with nginx on 8080, sets one boolean with <code>-P</code> , and builds the <code>lab_selinux</code> module from source it prints before loading. Idempotent.	yes
<code>lab-se.sh</code>	The payload. Mode, the three labels that must agree (file, process, port), every local change someone made to policy, recent denials, and a closing section that names the mismatch. With a path argument it answers "what does SELinux know about this file". Installed as <code>/usr/local/bin/lab-se</code> .	yes
<code>explore-selinux.sh</code>	Twelve read-only stops: mode now versus after reboot, contexts on files and processes, policy's opinion versus the disk, the rule behind a label, port labels, booleans and their descriptions, <code>sesearch</code> , the denial log, and the service end to end.	yes
<code>break-and-fix.sh</code>	Four failures and their proper repairs: a <code>chcon</code> that a relabel reverts, an httpd type that still cannot be read, an unlabelled port that refuses a valid bind, and a boolean that was off. <code>--hard</code> describes the two that are not demonstrations: <code>setenforce 0</code> as an incident fix, and a full filesystem relabel.	yes
<code>lab_selinux.te</code>	The policy module in source form. Read the comments before <code>setup.sh</code> loads it - the <code>require</code> block is the list of everything the rule touches, and the permissions are named one by one for a reason.	-
<code>teardown.sh</code>	Stops the service before removing the policy that let it run, unloads the module, removes the local port rule, returns the boolean to its default, removes the fcontext rule before the directory, and leaves SELinux enforcing.	yes

Read them before you run them. They are commented as teaching material rather than production code - the comments are half the day.

Run it on the lab

1. On your laptop, bring up node1

```
./lab/lab.sh status          # what is already running?
./lab/lab.sh up node1       # 2 GB, about a minute
```

Day 12 left `node1` running; reuse it. A rebuilt `node1` is also fine - this day needs nothing from Day 12.

2. Copy the repo onto the VM

```
./lab/lab.sh push node1     # carries both days/ and lab/
./lab/lab.sh ssh node1
```

`push` is the step people skip. Editing a file on your laptop changes nothing on the VM until this runs.

3. Work through the day on the VM

```
hostname                    # must print: node1
cd ~/lab/days/day13
```

The tools live in four different packages, which is the usual reason a triage session stalls before it starts:

```
sudo dnf install -y nginx policycoreutils policycoreutils-python-utils \
checkpolicy setools-console audit curl
```

Before you change anything, read what is already there:

```
sestatus
ls -Zd /srv /usr/share/nginx/html
semanage fcontext -l -C          # what has anyone changed here before?
```

That second command is the day in one line: `/usr/share/nginx/html` is already labelled for the web server, and `/srv` is not. Nothing about ownership or mode differs.

In this order:

```
less scripts/setup.sh           # 1. read it BEFORE running it
sudo ./scripts/setup.sh         # 2. enforcing, label rule, nginx, boolean, module
sudo lab-se                     # 3. the whole picture in one screen
sudo lab-se /srv/www            # 4. everything about one path
sudo ./scripts/explore-selinux.sh # 5. the twelve stops
sudo ./scripts/break-and-fix.sh  # 6. four failures, four proper fixes
sudo ./scripts/break-and-fix.sh --hard # 7. the two that are described only
sudo ./verify.sh                # 8. seven checks, two for you
```

Do the manual half properly, because it is the half that transfers. Cause a real denial, then fix it by relabelling:

```
sudo chcon -R -t var_t /srv/www
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/ # 403
sudo ausearch -m avc -ts recent | tail -5                       # the reason
sudo restorecon -RFv /srv/www                                   # the fix
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/ # 200
```

Then build a module the real way and read it before loading anything:

```
sudo ausearch -m avc -ts recent | audit2allow -m lab_candidate # READ this
sudo ausearch -m avc -ts recent | audit2allow -M lab_candidate # then build
sudo semodule -i lab_candidate.pp
sudo semodule -r lab_candidate # and remove it
```

If `audit2allow` suggests a rule for a denial that a boolean already covers, use the boolean. A module is a permanent exception nobody reviews; a boolean is a documented, supported switch.

4. Clean up

```
sudo ./scripts/teardown.sh      # Leaves SELinux enforcing, on purpose
```

```
exit
./lab/lab.sh down node1         # frees the memory; takes the disk with it
```

Notes

The procedure, which is the only thing worth memorising:

1. **Is it enforcing?** `getenforce`. If it is permissive, SELinux is not your problem.
2. **Was anything denied?** `ausearch -m avc -ts recent`. An empty result is an answer: the fault is elsewhere, so stop guessing.
3. **Who and what?** In the AVC: `scontext` is the process, `tcontext` is the object, `tclass` and the permission in `{ }` are the operation.

4. **Is there a boolean?** `semanage boolean -l | grep <service>` . Check before writing policy.
5. **Is it a label?** `ls -Z` the object, `matchpathcon` it, and compare. Different answers mean a relabel, not an exception.
6. **Fix the cause:** `semanage fcontext + restorecon` , `semanage port -a` , `setsebool -P` , or a module you read first.

Three things worth keeping. A 403 with correct ownership and correct mode is almost always a label, because Unix permissions and SELinux are two independent checks and both must pass. `chcon` is a debugging tool, not a fix - if you cannot point at the `semanage fcontext` rule that keeps a label in place, the label is temporary. And ports and booleans are labelled objects just like files, which is why "it is not a file, so it cannot be SELinux" is wrong so often.

The difference between someone who runs SELinux confidently and someone who disables it on first contact is not knowledge of the type names. It is the habit of reading the denial before changing anything.

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Next up: **Day 14 - Ansible fundamentals.**

DAY 14 Day 14 - Ansible fundamentals

days/day14/README.md

This day is provable, not descriptive: **6 automatic checks** and **2 judgement calls** in `verify.sh`.

Replace one day of manual work with a playbook that is safe to run twice.

PHASE	HARDENING AND CONFIGURATION MANAGEMENT
Runs on	control -> node1
Memory	~3.5 GB (two VMs)
Verified by	lint + your lab

Why this day exists

You have now configured hosts by hand for thirteen days. This is the day that work becomes repeatable, and idempotency stops being a buzzword.

The word means one thing in practice: **run it twice and the second run reports no changes**. That single measurement is what separates configuration management from a shell script in YAML. A playbook that changes something on every run cannot tell you whether a host has drifted, because it always says yes - and a fleet you cannot ask "is anything different?" is a fleet you do not actually manage.

Everything else today serves that idea. Modules describe a state instead of running a command. `--check` predicts. Handlers restart a service only when its configuration moved. Variables live in files next to the play instead of in your shell history.

Ansible itself is deliberately boring: no agent, no daemon, no port. It is SSH, Python on the far side, and YAML on yours. If `ssh lab@node1` does not work, nothing in this day will - which is why `setup.sh` settles that before it writes a single line of inventory.

What you will work with

- `ansible.cfg` and `ansible-config dump --only-changed` - Ansible reads `ANSIBLE_CONFIG`, then `./ansible.cfg`, then `~/.ansible.cfg`, then `/etc/ansible/ansible.cfg`, and **stops at the first one it finds**. They are not merged, so a forgotten file in your home directory silently changes every run. `--only-changed` prints exactly what a project's config did, which is the shortest honest description of its opinions.
- **Inventory, in INI and in YAML** - `inventory/hosts.ini` and `inventory/hosts.yml` describe the same hosts, and you will compare them with `ansible-inventory --graph`. INI is terse and has one nasty habit: a typo in a `[group:vars]` header becomes a host named `[group:vars]`. YAML is verbose and exact.
- `ansible-inventory --graph` and `--host node1` - the tree, and then one host with **every variable already merged**. Never work out precedence in your head: `host_vars` beats a named group, which beats `group_vars/all.yml`, and `-e` on the command line beats all of it.
- `ansible -m ping` - not ICMP. It opens SSH, runs Python on the far side and gets a pong back, so a pass means the entire transport works and a failure is almost always SSH or Python rather than Ansible.
- `package`, `service`, `copy`, `template`, `user`, `file`, `lineinfile` - the modules that make up most real playbooks. Each takes a **state** rather than an action, which is what makes a second run quiet. `copy` and `template` compare checksums, so "changed" from them always means the host drifted.

- `lineinfile` and its `regexp` - the module people misuse most. Without a `regexp` it is idempotent for the exact line you wrote and appends a new one as soon as the value changes. For a file you own entirely, use `template` instead; `lineinfile` is for files that belong to a package.
- **One file, one owner.** Point `lineinfile` at a file that `copy` or `template` also manages and both tasks report `changed` on every run, forever: `copy` restores its checksum, `lineinfile` puts its line back, and neither is wrong. `site.yml` shows the split deliberately - `copy` owns `lab-day14.txt`, `template` owns `lab-info.conf`, and `lineinfile` owns only its own line in `settings.conf`. If a second run will not go quiet, look for two tasks fighting over one path before you suspect the module.
- `--syntax-check`, `--list-tasks`, `--check --diff` - parse, then read the blast radius, then see the prediction. All three are free and read-only. They also have a limit worth knowing: `command` and `shell` tasks are skipped in check mode, so a dry run of a play built from shell commands predicts nothing at all.
- **notify and handlers** - a handler runs once, at the end, and **only if the notifying task reported changed**. That is the feature: services restart when their configuration moves and not otherwise. A failed play drops pending handlers, which is how a host ends up with new config on disk and an old process running.
- `become` - privilege per task, not per run. The project starts with `become = false` on purpose, so reading the playbook tells you which steps touch the system. `become` is `sudo`, so a run that hangs in silence is usually `sudo` waiting for a password nobody typed.

Verify

Checked automatically:

- ☐ you are running this as yourself, not as root
- ☐ the inventory parses
- ☐ node1 answers a ping module
- ☐ the playbook has valid syntax
- ☐ a first run completes with no failures
- ☐ a second run changes nothing

Only you can confirm:

- ☐ `--check` predicted the same changes the real run made
- ☐ you can explain which module was not idempotent and why

Run the automatic checks with:

```
./days/day14/verify.sh
```

Without `sudo`. Every other lab day wants root; this one refuses it. As root, `~/ssh` is `/root/.ssh`, the key named in the inventory is not there, and all five checks fail as `UNREACHABLE` for a reason that has nothing to do with your work. The first check exists to catch exactly that.

CI can only lint this day. Nothing on a GitHub runner has SELinux, firewalld, systemd units you control, or a second host to reach over SSH - so the checks above are proven by running `verify.sh` on your own lab, and nowhere else.

Scripts for today

SCRIPT	WHAT IT DOES	ROOT?
<code>setup.sh</code>	Refuses to run as root , installs the pushed SSH key with mode 0600, proves plain <code>ssh</code> to <code>node1</code> works before writing any inventory, then builds <code>~/ansible-lab</code> : config, both inventory formats, <code>group_vars</code> , <code>host_vars</code> , files, templates, <code>site.yml</code> and <code>not-idempotent.yml</code> . Ends by running the play twice and showing you <code>changed=0</code> . Idempotent.	no
<code>lab-ansible.sh</code>	The payload. Which config won, the inventory as a tree, both formats side by side, reachability, variables as resolved rather than as written, and a closing section naming the three places the answers disagree. <code>lab-ansible node1</code> for one host; <code>lab-ansible drift</code> for a check run. Installed as <code>/usr/local/bin/lab-ansible</code> .	no
<code>explore-ansible.sh</code>	Twelve read-only stops, ending with the six-command order to use on a project you have just inherited. Safe against an estate you do not understand yet.	no
<code>break-and-fix.sh</code>	Four failures with their fixes: <code>command</code> instead of a module, <code>lineinfile</code> with no <code>regexp</code> , handler expectations, and a missing <code>become</code> . <code>--hard</code> describes the two that teach nothing by happening: an ad-hoc <code>shell</code> against <code>all</code> , and <code>become: true</code> everywhere. Writes its broken plays to <code>/tmp</code> , never into your project.	no
<code>teardown.sh</code>	The undo, written as a playbook with every state inverted - which is the argument for configuration management in one script. Dry run first, then apply, then prove each removal. Leaves <code>rsyslog</code> alone because Rocky shipped it. <code>--all</code> also deletes the project.	no

Read them before you run them. They are commented as teaching material rather than production code - the comments are half the day.

Run it on the lab

This is the first day since Day 05 that genuinely needs two VMs: `control1` runs Ansible, `node1` is managed. Together they are about 3.5 GB.

1. On your laptop, bring up both VMs

```
./lab/lab.sh status           # what is already running?
./lab/lab.sh up control node1 # ~3.5 GB total, a minute or two
```

`node1` can be the same one Days 11-13 used; nothing here conflicts with it. A rebuilt `node1` is also fine - this day installs everything it needs.

2. Copy the repo and the key onto control

```
./lab/lab.sh push control1      # carries days/ and lab/
./lab/lab.sh push control1 ~/.ssh/id_ed25519
```

That second line is the one people miss. `control1` can reach `node1` over the network, but it holds no credential for it, and Ansible has no way to authenticate that SSH could not already do. `setup.sh` moves the pushed key into `~/.ssh/id_ed25519` with mode 0600 and deletes the copy from `~/lab` .

Be clear-eyed about what that is: a private key on a control node means whoever owns the control node owns every managed host. Real estates use a key generated on the control node and authorised on the fleet, an SSH agent with forwarding, or short-lived certificates. Here it is one throwaway VM reaching another.

You also need `node1`'s address, from the same `status` output:

```
./lab/lab.sh status           # note node1's address, on the node1 line
./lab/lab.sh ssh control
```

Every address in this README is an example. libvirt hands out leases in whatever order the VMs booted, and they move when a VM is rebuilt, so read yours from `status` every time. `setup.sh` checks the address before it builds anything: it refuses an address that belongs to `control` itself, and when nothing answers it tells you whether that was no route, a refused connection, or a rejected key - three different problems that look identical from the `ansible` command line, where they all arrive as `UNREACHABLE`.

3. Work through the day on control

```
hostname           # must print: control
sudo dnf install -y ansible-core
cd ~/lab/days/day14
```

`ansible-core` is the whole dependency. It pulls Python and nothing else, and the managed host needs no agent - only the Python that Rocky already ships.

In this order:

```
less scripts/setup.sh           # 1. read it BEFORE running it
./scripts/setup.sh 192.168.122.42 # 2. EXAMPLE address - use your own
lab-ansible                     # 3. the whole picture in one screen
lab-ansible node1               # 4. one host, variables and facts
lab-ansible drift               # 5. what a run would change now
./scripts/explore-ansible.sh    # 6. the twelve stops
./scripts/break-and-fix.sh      # 7. four failures, four fixes
./scripts/break-and-fix.sh --hard # 8. the two that are described only
./verify.sh                    # 9. six checks, two for you
```

Note what is missing from every line above: `sudo`.

Then do the manual half, because it is the half that transfers. Prove `--check` tells the truth by breaking one thing on the host by hand and asking Ansible about it before fixing it:

```
cd ~/ansible-lab
ansible node1 -b -m shell -a 'echo broken >> /etc/lab-day14/lab-day14.txt'
ansible-playbook site.yml --check --diff      # predicts exactly one change
ansible-playbook site.yml                    # makes exactly that change
ansible-playbook site.yml                    # changed=0 again
```

That is drift detection, and it is only possible because every task describes a state. Now watch the same idea fail:

```
ansible-playbook not-idempotent.yml --check    # predicts nothing
ansible-playbook not-idempotent.yml           # changed=3
ansible-playbook not-idempotent.yml           # changed=3, forever
ansible node1 -b -m command -a 'cat /etc/lab-day14/lab-day14.txt'
ansible-playbook site.yml                     # copy puts the file back
```

The second `you` item is that comparison in your own words: which task was not idempotent, and why the module version is.

4. Clean up

```
./scripts/teardown.sh           # a playbook with every state inverted
```

Keep `~/ansible-lab` - Day 15 turns `site.yml` into a role. Pass `--all` only if you want to start from nothing.

```
exit  
./lab/lab.sh down control node1 # frees ~3.5 GB; takes both disks with it
```

Notes

The order to use on a playbook you did not write:

1. `ansible --version` - which config file is in play.
2. `ansible-config dump --only-changed` - what it changed.
3. `ansible-inventory --graph` - which hosts, in which groups.
4. `ansible-inventory --host <one>` - the variables, already merged.
5. `ansible-playbook <play> --list-tasks` - what it would do.
6. `ansible-playbook <play> --check --diff` - what it would change today.

All six are read-only and take under a minute. They are the difference between applying a playbook and detonating one.

Three things worth keeping. A second run that reports `changed=0` is the only honest test of a play, so run everything twice before you believe it. `--check` is a prediction, not a guarantee, and it is worthless for `command` and `shell` tasks because those are skipped entirely - a play built from shell commands has no dry run. And privilege belongs on tasks rather than plays: `become: true` everywhere works right up until a typo in a path, and then it is an incident instead of a permission error.

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Next up: **Day 15 - Ansible roles: your hardening baseline.**

DAY 15 Day 15 — Ansible roles: your hardening baseline

days/day15/README.md

This day is provable, not descriptive: **9 automatic checks** and **2 judgement calls** in `verify.sh`.

Phase: Hardening and configuration management **Runs on:** `control` -> `node1` + `node2` **Time:** about two hours

Day 14 put every task in one playbook. That works until the playbook is four hundred lines and two people are editing it. Today you take Days 11, 12 and 13 — `firewalld`, `sshd`, `fail2ban`, `SELinux` — and turn them into a role: a directory with a layout everybody already knows, variables a user can override without editing your code, and handlers that reload the right daemon once.

Then you point it at `node2`, a machine you have never logged into, and it comes out matching `node1`.

The part worth your attention is not the YAML. It is that a role has four places a variable can come from, three of them beat your defaults, and a role can report `changed=0` on a host it never touched.

What you will work with

- `ansible-galaxy init` and the role layout every role on earth uses: `defaults/`, `vars/`, `tasks/`, `handlers/`, `templates/`, `files/`, `meta/`
- `defaults/main.yml` as the role's public API — the lowest precedence in Ansible, and therefore the only safe place for a value someone may change
- `vars/main.yml` for the role's internals — near the top of the precedence list, so inventory cannot quietly reprint a path your handlers depend on
- `import_tasks` with tags, so one role can be run one area at a time
- handlers, and the two cases where they do not fire: a failed play, and `--check`
- `ansible-vault` with a password file, `group_vars/lab/` as a directory, and the `vault_` naming convention that makes secrets greppable
- `--tags`, `--skip-tags`, `--limit`, `--list-tasks`, `--list-tags`
- drift: `--check --diff` against a host somebody edited by hand
- `ansible-inventory --host`, the only honest answer to "which value won"

No collections. `ansible-core` ships none, so there is no `ansible.posix` `selinux` module and no `community.general` `firewalld` module here. SELinux is `lineinfile` plus `getenforce` / `setenforce` with a `when: ;` `firewalld` is `firewall-cmd --query / --add` with a reload handler. Slightly longer, and you see exactly what a module does: read the state, decide, write, report.

Three VMs, and the memory

Today is the only day that needs all three, and it is the peak of the course:

VM	MEMORY	WHY
<code>control</code>	1536 MB	runs Ansible, nothing else
<code>node1</code>	2048 MB	already hardened by hand on Days 11–13
<code>node2</code>	2048 MB	clean host — the one the role has to fix

About **5.5 GB** in total. Nothing in this lab is below 1536 MB, which is the minimum Rocky 9 recommends and the number `virt-install` warns about. Below it, `dnf` gets killed by the OOM killer and prints a bare `killed` with no explanation.

On an 8 GB laptop, close the browser first. If it is tight, bring `node1` up after the role has been applied to `node2` — `--limit` exists for exactly this.

Scripts

SCRIPT	WHAT IT DOES
<code>scripts/setup.sh [node1-addr] [node2-addr]</code>	checks ssh to both nodes, writes the inventory, the vault, the role and <code>hardening.yml</code> , then runs it twice
<code>scripts/lab-harden.sh</code>	installed as <code>lab-harden</code> . Policy, per-host data, and what the hosts actually report
<code>scripts/explore-roles.sh</code>	twelve read-only stops through the role you just built
<code>scripts/break-and-fix.sh</code>	five ways a role reports success and delivers nothing. <code>--hard</code> describes two lock-outs rather than causing them
<code>scripts/teardown.sh</code>	the inverse play. <code>--all</code> also deletes the role and playbook
<code>verify.sh</code>	eight automatic checks, two for you to judge

Run it

On your laptop, in the repository:

```
./lab/lab.sh up control node1 node2      # ~5.5 GB
./lab/lab.sh status                      # read BOTH addresses
./lab/lab.sh push control                 # carries days/ and lab/
./lab/lab.sh push control ~/.ssh/id_ed25519
./lab/lab.sh ssh control
```

Both push lines, in that order. `push control` with no path copies the repository; `push control <path>` copies **only** that one file. Give it the key and skip the first line and `~/lab` on `control` holds a key and nothing else, which is a confusing way to discover that `~/lab/days/day15` does not exist.

The key is the second line for Day 14's reason: `control` can reach the nodes over the network but holds no credential for them, and Ansible cannot authenticate where plain `ssh` could not. `setup.sh` moves the pushed copy to `~/ssh/id_ed25519` at mode 0600 and deletes it from `~/lab`.

`lab.sh status` prints something like:

```
node1  running  192.168.122.75    # 1. EXAMPLE address – use your own
node2  running  192.168.122.61    # 2. EXAMPLE address – use your own
```

Those two addresses are examples. DHCP leases move every time a VM is rebuilt, so read them yourself each session rather than copying them from here.

Do you need Day 14's setup first?

No. `setup.sh` writes its own `ansible.cfg` if `~/ansible-lab` does not have one, which is what happens on a freshly rebuilt `control` VM — the project lives in the VM's home directory, not in the repository, so `lab.sh down` followed by `up` takes it with it.

If Day 14's project is still there, today extends it and leaves `site.yml` alone, so Day 14's `verify.sh` keeps passing.

If you want Day 14's playbook back as well — not needed for anything today:

```
cd ~/lab/days/day14 && ./scripts/setup.sh <node1-addr>
```

Run that one first if you run it at all; Day 15 then adds the vault line to the config it wrote rather than writing its own.

Then on `control1`, as the `lab` user and **without** `sudo`:

```
cd ~/lab/days/day15
./scripts/setup.sh 192.168.122.75 192.168.122.61 # your addresses
```

Run with no arguments after the first time and it reuses what is already in the inventory.

`setup.sh` stops at the first thing that is wrong and tells you which thing. SSH before Ansible, as on Day 14: a failing `ssh` is a failing `ansible`, and Ansible's `UNREACHABLE` looks identical whether the host is down, the port is shut or the key is wrong.

Then, in order:

```
lab-harden           # both hosts, side by side
lab-harden node2      # one host, in detail
lab-harden drift      # what a run would change now
./scripts/explore-roles.sh # read the role you just built
./scripts/break-and-fix.sh # five failures that look like success
./verify.sh
```

What to actually look at

The playbook is six lines. Three days of manual hardening, applied to any number of hosts, readable by somebody who has never opened the role. That is the entire argument for roles.

node1 opens port 8080 and node2 does not. Same role, different data. The port is in `group_vars/webserver.yml` because Day 13 put nginx there; the role's default is an empty list. Nothing was edited to make that happen.

Editing `defaults/main.yml` does not change node1's ports. Defaults are the lowest precedence in Ansible, and `group_vars` beats them. No error, no warning, no effect — the second failure in `break-and-fix.sh`, and the one you will meet in a real repository first.

`ansible-inventory --host node1` is the answer. Do not work precedence out in your head. Ask.

Order in `tasks/main.yml` is a safety property. The `labssh` group exists and your account is in it *before* `sshd` is told `AllowGroups labssh`. Reverse those two and `sshd -t` still validates the file happily — a group does not have to exist for the config to parse — the reload succeeds, and the next connection is refused. Including Ansible's own.

`--limit node-2` exits 0. `skipping: no hosts matched`, green tick, nothing deployed. Set `host_pattern_mismatch = error` in `ansible.cfg` and a typo fails the run instead of passing it.

The secret is ciphertext in git. `group_vars/lab/vault.yml` starts with `$ANSIBLE_VAULT`, the password lives in `~/vault-pass-lab` at `0600` outside the project, and the role reads `hardening_alert_token`, which points at

`vault_hardening_alert_token`. Grep for `vault_` and you have found every secret the project reads. The cost: a diff of the vault file tells you nothing.

A `--check` **run on a clean host reports failures**. Tasks that read a file an earlier task was supposed to create have nothing to read in a dry run. Check mode being honest, not a bug — and the reason `--check` is a hint, not a contract.

Teardown

```
./scripts/teardown.sh          # undo the role on node2 only
./scripts/teardown.sh --all    # node1 too, and delete the role and playbook
```

The inverse of a play is a play, not `rm` on the host. The teardown leaves SELinux enforcing, leaves `firewalld` installed, and leaves the `labssh` group alone — read the comments in the generated playbook for why each of those is deliberate.

Then, on your laptop:

```
./lab/lab.sh down control node1 node2
```

Day 16 needs two VMs again.

Done when

`./verify.sh` prints **8 passed, 0 failed** and you can say, without looking it up, why a value in `group_vars` beats the same value in `defaults/main.yml` — and what `--limit` does when it matches nothing.

docs/curriculum.md#phase-4

PHASE 4

Production operations

Days 16–20

16	WireGuard: a private network between hosts	92
17	Reverse proxy and TLS termination	96
18	Bridges, VLANs and link aggregation	101
19	Intrusion detection and audit alerting	105
20	Backup, restore and the restore drill	111

DAY 16 Day 16 — WireGuard: a private network between hosts

days/day16/README.md

This day is provable, not descriptive: **12 automatic checks** and **2 judgement calls** in `verify.sh`.

Phase: Production operations **Runs on:** `control` + `node1`, on both of them **Time:** about ninety minutes

Every environment has a network you are not supposed to expose: a database port, a metrics endpoint, an admin interface that was written in 2011. The usual answer is a VPN, and the usual VPN is too large to hold in your head.

WireGuard is about four thousand lines of kernel code and a config file with six keys in it. Today you build a tunnel between two VMs, by hand, twice — because each host generates its own private key and the keys are exchanged out of band, which is most of the point.

The part worth your attention is not `wg-quick up`. It is that one line, `AllowedIPs`, is simultaneously a routing table and an access control list, and that WireGuard's answer to anything that fails to authenticate is silence. No error, no log line, nothing. Four of today's failures look exactly like a working tunnel until you ask for the handshake.

What you will work with

- `wg genkey` and `wg pubkey`, and `umask 077` **before** you generate rather than `chmod` after — a private key only has to leak once
- `/etc/wireguard/wg0.conf`: `[Interface]` for this host, `[Peer]` for the other, and no notion of client or server anywhere
- `AllowedIPs` in both of its jobs: a route outbound, a source-address filter inbound — so the two ends are coupled and the symptom shows at the healthy one
- `wg show` versus `wg showconf`: live kernel state, and the fact that nothing re-reads the file after the interface comes up
- `wg-quick up / down / strip`, which is a shell script wrapping `ip(8)` — `Address` and `MTU` are its keys, not WireGuard's
- `wg syncconf`, which applies a changed file without dropping the tunnel or your session with it
- `systemctl enable wg-quick@wg0`, and the difference between a tunnel that works now and one that works after a reboot
- `firewalld` and one UDP port, with no TCP fallback to fall back to
- MTU: $1500 - 80 = 1420$, and what happens when you get it wrong (ping passes, transfers hang)

No kernel module to build. WireGuard has been in the mainline kernel since 5.6, so Rocky 9 needs `wireguard-tools` and nothing else — no DKMS, no elrepo, nothing to compile.

Two VMs, and the memory

VM	MEMORY	TUNNEL ADDRESS	WHY
<code>control</code>	1536 MB	<code>10.20.0.1/24</code>	one end
<code>node1</code>	2048 MB	<code>10.20.0.2/24</code>	the other end

About **3.5 GB** in total, down from Day 15's three VMs. Nothing in this lab is below 1536 MB, which is the minimum Rocky 9 recommends and the number `virt-install` warns about.

The `10.20.0.0/24` addresses are the tunnel's own. They are not the `192.168.122.x` lab addresses and they do not replace them — the tunnel cannot carry the packets that build the tunnel.

Scripts

SCRIPT	WHAT IT DOES
<code>scripts/setup.sh</code>	pass 1: keys, <code>wg0.conf</code> , prints this host's public key. Run on both VMs
<code>scripts/setup.sh <peer-key> <peer-addr></code>	pass 2: adds the peer, opens the port, enables the unit, proves the handshake
<code>scripts/lab-wg.sh</code>	installed as <code>lab-wg</code> . <code>status</code> , <code>keys</code> , <code>routes</code> , <code>watch</code>
<code>scripts/explore-wg.sh</code>	twelve read-only stops through the tunnel you just built
<code>scripts/break-and-fix.sh</code>	four failures that look like success. <code>--hard</code> describes two lock-outs rather than causing them
<code>scripts/teardown.sh</code>	down, disabled, port closed. <code>--all</code> also deletes the keys
<code>verify.sh</code>	twelve automatic checks, two for you to judge

Run it

On your laptop, in the repository:

```
./lab/lab.sh up control node1      # ~3.5 GB
./lab/lab.sh status                # read BOTH addresses
./lab/lab.sh push control          # carries days/ and lab/
./lab/lab.sh push node1           # today runs on both hosts
./lab/lab.sh ssh control
```

Both hosts need the repository today, which is new — every earlier day ran its scripts from `control` only. `push <vm>` with no path copies `days/` and `lab/`; `push <vm> <path>` copies **only** that one file.

`lab.sh status` prints something like:

```
control  running  192.168.122.188  # EXAMPLE address — use your own
node1    running  192.168.122.105  # EXAMPLE address — use your own
```

Those are examples. DHCP leases move every time a VM is rebuilt, so read them yourself each session rather than copying them from here.

Day 14 and Day 15 are not prerequisites. Today touches nothing they built.

Pass 1, on each host

On `control`:

```
cd ~/lab/days/day16
sudo ./scripts/setup.sh
```

It prints a public key. In another terminal, on `node1`:

```
cd ~/lab/days/day16
sudo ./scripts/setup.sh
```

That prints a different public key. Two hosts, two keypairs, neither private key having moved anywhere. If you find yourself copying a file called `wg0.key` between machines, stop — that is not how this works.

Pass 2, on each host, with the other one's key

On `control`, using `node1`'s public key and `node1`'s **lab** address:

```
sudo ./scripts/setup.sh <node1-public-key> 192.168.122.105
```

And on `node1`, using `control`'s:

```
sudo ./scripts/setup.sh <control-public-key> 192.168.122.188
```

The second one to run is the one that reports a handshake. The first will say it is waiting, which is correct — a tunnel needs both ends to hold the other's key, and until then packets arrive and get dropped without comment.

If the hostnames are not `control` and `node1`, tell it which end it is:

```
sudo ROLE=control ./scripts/setup.sh <peer-key> <peer-addr>
```

Then, in order:

```
sudo lab-wg          # interface, peer, handshake, traffic
sudo lab-wg keys      # what is public and what is not
sudo lab-wg routes    # AllowedIPs as the routing table
sudo ./scripts/explore-wg.sh # read the tunnel you just built
sudo ./scripts/break-and-fix.sh # four failures that look like success
sudo ./verify.sh
```

Before `verify.sh`, reboot one of the VMs and watch the tunnel come back by itself. One of the two manual checks is exactly that, and `enabled` is a claim until you have tested it.

What to actually look at

There is no server. Both ends listen on the same port, both can initiate, and the only asymmetry is that one of them was told where the other is. `Endpoint` is a hint, and it updates itself when a peer moves — which is why WireGuard survives a laptop changing wifi mid-session, and why a DHCP lease change on `node1` is not fatal.

AllowedIPs is two things. Outbound it is a route: `wg-quick` turned each entry into `ip route add`. Inbound it is an ACL: a packet from that peer whose source is not listed is dropped silently. So narrowing it on one side only means traffic leaves and never returns — and the end you would naturally debug is the one that is configured correctly. This is the second failure in `break-and-fix.sh` and the one that costs people an afternoon.

The handshake is the only evidence. An interface with an address, a peer and a route looks identical in `ip addr` whether it works or not. `wg show wg0` and `latest handshake: never` is the whole diagnosis. `wg show wg0 transfer` is the second question: sent climbing, received zero means your packets are leaving and being dropped at the far end.

The crypto is not configurable. No cipher list, no TLS version, no downgrade to negotiate. That removes a whole category of misconfiguration, and costs you the ability to fix a broken primitive with a config change — it would take a new protocol version.

Nothing re-reads the file. `wg-quick up` parsed it once. Edit `wg0.conf` afterwards and the kernel keeps what it was given, with no warning that the two disagree. `diff <(wg-quick strip wg0) <(wg showconf wg0)` is the check, and `wg synconf` is the fix that does not drop the tunnel — on a management VPN, `down && up` drops your own session.

MTU is where the day goes if it goes. 1420 is 1500 minus 80 bytes of WireGuard overhead. Set it to 1500 and `ping` succeeds, `ssh` logs in, and `scp` stalls at 0%, because only full-size packets die and the ICMP that would have explained it is usually dropped somewhere in between.

`wg-quick strip` and `wg showconf` **do not print the same text.** Both describe the same tunnel, but `strip` echoes your file with your order, spacing and comments while `showconf` prints the kernel's normalized version. Diff them raw and it always looks broken. Collapse the whitespace, keep the peer settings, sort both sides — then a difference means a real unapplied edit, which is the only thing worth checking.

`firewall-cmd --permanent` **needs the daemon.** On a freshly built VM, `firewalld` is installed and stopped, and `--permanent` does not quietly fall back to editing files — it prints `Firewalld is not running` and fails, which is the first thing this day's `setup.sh` used to do. `firewall-offline-cmd` writes the same XML with no daemon to ask. `setup.sh` now uses whichever applies, then starts and enables `firewalld`, because a port "open" in a policy nothing has loaded is a comment.

Port scans see a closed port. WireGuard does not reply to anything that fails to authenticate, including with an error. A real security property, and a real tax when you are the one debugging.

Teardown

On each VM:

```
sudo ./scripts/teardown.sh          # down, disabled, port closed, keys kept
sudo ./scripts/teardown.sh --all    # keys and payload deleted too
```

Keys kept means `sudo wg-quick up wg0` puts the tunnel straight back. Keys deleted means the far end now lists a public key that does not exist anywhere — so if you delete them on one host, run `setup.sh` again on both.

Then, on your laptop:

```
./lab/lab.sh down control node1
```

Done when

`./verify.sh` prints **12 passed, 0 failed**, the tunnel came back after a reboot without you touching it, and you can say — without looking it up — what `AllowedIPs` does to a packet going out and to a packet coming in.

Next up: **Day 17 — Reverse proxy and TLS termination.**

DAY 17 Day 17 — Reverse proxy and TLS termination

days/day17/README.md

This day is provable, not descriptive: **19 automatic checks** and **2 judgement calls** in `verify.sh`.

Phase: Production operations **Runs on:** `node1` **Time:** about ninety minutes

Almost nothing serves its own TLS. The application listens on some high port, in plain HTTP, on the loopback interface, and something in front of it holds the certificate and does the arithmetic. That something is usually nginx, and the arrangement is so common that most people have never seen the inside of it.

Today you build it: nginx on `node1` terminating TLS for `www.lab.test`, in front of a backend on `127.0.0.1:8080` that has never heard of a certificate. The certificate is signed by the CA you built on Day 10, and the CA goes into this host's trust store — the first time anything in this repository installs a trust anchor. Day 10 deliberately stopped one step short of that.

Day 13 will fight you here, which the curriculum promised and meant. nginx runs as `httpd_t`, and `httpd_t` may not open a socket to your backend until a boolean says it can. The symptom is `502 Bad Gateway`. The word SELinux appears nowhere in it.

What you will work with

- `proxy_pass`, and the three headers that go with it — `Host`, `X-Real-IP`, `X-Forwarded-For` — because the backend otherwise sees every request as arriving from `127.0.0.1`, which is true and useless
- `listen 443 ssl` with `ssl_certificate` and `ssl_certificate_key`, and a `listen 80` server block whose only job is `return 301`
- `--bind 127.0.0.1` on the backend: the entire boundary between private and public here, and not the firewall, which never sees loopback traffic at all
- `subjectAltName`, which is the field clients match, and `CN`, which is decoration — a certificate with no SAN matches no hostname at all
- `openssl verify -CAfile` versus `openssl s_client -servername <name> -verify_hostname <name>`: the first checks the chain and the dates, the second also checks the name
- `openssl x509 -noout -modulus | openssl md5` against the key's, because "both files are valid" does not mean "these two belong together"
- `update-ca-trust` and `/etc/pki/ca-trust/source/anchors/`, and the fact that it satisfies curl and wget while Firefox, Java and node keep their own stores and go on refusing
- `setsebool -P httpd_can_network_connect on`, and what the missing `-P` buys you: a proxy that works until the next reboot
- `restorecon` on the installed certificate and key, since a file copied into `/etc/pki/tls` from elsewhere carries the label of where it came from
- `nginx -t` versus `nginx -T` versus what the workers are actually running — three different questions with three different answers
- `systemctl reload nginx`, not `restart`: reload replaces workers and lets requests in flight finish
- `502` versus `504`, and reading the error log to tell a refusal from silence

One VM, and the memory

VM	MEMORY	WHAT RUNS ON IT	WHY
node1	2048 MB	nginx, the backend, the CA	nginx plus a python process needs the headroom

About **2 GB** in total, down from Day 16's two VMs. Nothing in this lab is below 1536 MB, which is the minimum Rocky 9 recommends and the number `virt-install` warns about.

Days 14, 15 and 16 are not prerequisites. Today touches nothing they built — no `ansible.cfg`, no project directory, no tunnel. The days it leans on are Day 10 for the CA, Day 11 for `firewalld` and Day 13 for SELinux.

The shape you are building:

```
client → :443  nginx  → 127.0.0.1:8080  python3 -m http.server
      TLS ends here      plain HTTP, loopback only, no certificate
      :80 → 301 https
```

The name comes from `/etc/hosts` — `127.0.0.1 www.lab.test`, written by `setup.sh`. Day 08's zone serves `www.lab.test` properly, and if you have that running you can point this VM's resolver at it instead and delete the hosts entry. The shortcut is deliberate: a name in `/etc/hosts` works on exactly one host, which is worth feeling once.

`/etc/lab-tls` is where Day 10 kept the CA. If it is not on this VM, `setup.sh` issues one there with Day 10's own commands, so the day stands up on a freshly built host. A day may lean on another day's lesson; it may not lean on another day's leftovers.

Scripts

SCRIPT	WHAT IT DOES
<code>scripts/setup.sh</code>	eleven steps: packages, backend and its unit, CA, certificate, trust store, <code>/etc/hosts</code> , nginx config, the SELinux boolean, <code>firewalld</code> , payload, proof. Idempotent
<code>scripts/lab-proxy.sh</code>	installed as <code>lab-proxy</code> . <code>status</code> , <code>certs</code> , <code>test</code> , <code>logs</code>
<code>scripts/explore-proxy.sh</code>	twelve read-only stops through the proxy you just built
<code>scripts/break-and-fix.sh</code>	four real failures, each fixed in front of you. <code>--hard</code> describes two that break nothing and expose everything
<code>scripts/teardown.sh</code>	proxy, backend, ports and name removed. <code>--all</code> also removes the certificates, the CA and the trust anchor
<code>verify.sh</code>	nineteen automatic checks, two for you to judge

Run it

On your laptop, in the repository:

```
./lab/lab.sh up node1          # ~2 GB
./lab/lab.sh status            # read the address
./lab/lab.sh push node1       # carries days/ and lab/
./lab/lab.sh ssh node1
```

`lab.sh status` prints something like:

```
node1      running  192.168.122.140  # EXAMPLE address – use your own
```

That is an example. DHCP leases move every time a VM is rebuilt, so read it yourself each session rather than copying it from here.

Then, on `node1`:

```
cd ~/lab/days/day17
sudo ./scripts/setup.sh
```

It ends by proving the whole path: a handshake it verified itself, a request over `https://www.lab.test/`, and the same content fetched from the backend directly with no TLS anywhere near it.

Then, in order:

```
sudo lab-proxy          # units, listeners, ports, SELinux
sudo lab-proxy certs    # what the certificate claims, and who signed it
sudo lab-proxy test     # a verified handshake and a real request
sudo lab-proxy logs     # the proxy's two logs, and the backend's journal
sudo ./scripts/explore-proxy.sh # read the proxy you just built
sudo ./scripts/break-and-fix.sh # four failures, four fixes
sudo ./scripts/break-and-fix.sh --hard
sudo ./verify.sh
```

If you edit the scripts on your laptop afterwards, push them again before you re-run anything — the VM has its own copy and will happily keep running the old one:

```
./lab/lab.sh push node1
```

Before `verify.sh`, do one thing by hand:

```
curl -s http://127.0.0.1:8080/
```

No certificate, no verification, no encryption. That is the hop you are trusting, and the reason it is acceptable is that it never leaves the host.

What to actually look at

TLS terminates somewhere, and you should be able to point at it. After nginx, this traffic is ordinary HTTP. Over loopback that is fine. Over a network it is the thing people think they have stopped doing — and it is what Day 16's tunnel would be for, if the backend lived on another host.

subjectAltName, not **CN**. Clients have matched names against the SAN for years and browsers stopped reading CN entirely. `break-and-fix.sh` serves a certificate for `other.lab.test` and everything about it is valid: signed by a CA this host trusts, in date, correct key. Every client still refuses it, because "is this certificate good" and "is this certificate for the name I asked for" are two separate questions.

`openssl verify` **has never checked a hostname**. It will tell you `ok` about a certificate no browser will accept. The command that answers the real question is `openssl s_client -connect host:443 -servername <name> -verify_hostname <name> -CAfile <ca>`, and the only line that means yes is `Verify return code: 0 (ok)`.

SNI decides which certificate you get. Connect without `-servername` and nginx answers with its default server block. On a host with one site you never notice; on a host with six you get somebody else's certificate

and a confusing afternoon. Stop 5 of the tour shows both.

A valid certificate and a valid key are not a pair. Nothing in `openssl verify` checks that. Compare the moduli — `lab-proxy certs` prints both — because the failure mode is nginx refusing to start after a renewal, with an error about a key not matching, at the worst possible moment.

502 is not a generic error. `502` with `Connection refused` in the error log means something answered the connect attempt with a refusal, instantly: the backend is down. `502` with `Permission denied` means the connect was never allowed: SELinux. `504` means nobody answered at all and nginx gave up waiting: dropped packets, a firewall rule, a hung application. Three different places to look, and the status code narrows it before you read anything.

The boolean is the day's real lesson. nginx is `httpd_t`, and SELinux treats "open a TCP socket to 127.0.0.1:8080" as a policy decision rather than a network one. With `httpd_can_network_connect` off, the config is perfect, the certificate is perfect, the backend is running and answering — and every request is a 502. The only place the cause is named is the audit log. Use `-P`, or you have built a proxy that stops working at the next reboot.

`restorecon`, every time you copy a certificate in. A file inherits the label of where it was created, not where it now lives. The certificates go under `/etc/pki/tls` precisely because that tree is already labelled `cert_t`, and `setup.sh` relabels anyway, because assuming is how you get a permission denied on a file whose `ls -l` looks flawless.

nginx read the certificate once. Same lesson as Day 16's `wg showconf`, in the place it costs the most: renewal. Replace the certificate on disk and the workers keep serving the old one indefinitely. `nginx -t` passes, because the file on disk is valid. `openssl s_client | openssl x509 -noout -serial` against the serial on disk is the check, and `systemctl reload nginx` is the fix. A renewal cron that forgets the reload expires in production while the new certificate sits on disk, unused.

`reload`, not `restart`. Reload starts new workers with the new configuration and lets the old ones finish what they were doing. Restart drops every connection in flight, including the upload somebody is twenty minutes into.

HTTP/2 is spelled two different ways, and the wrong one is fatal. Before nginx 1.25.1 it is a parameter on the listen line, `listen 443 ssl http2;`. From 1.25.1 it is its own directive, `http2 on;`. Rocky 9 ships 1.20, so `http2 on;` there is `unknown directive "http2"` and nginx refuses to start at all — not a warning, a parse error. `setup.sh` reads `nginx -v` and writes whichever form applies, and prints which one it chose. Most nginx examples on the internet are one side or the other of that split with no version next to them.

Trust stores are per-consumer. `update-ca-trust` writes the bundle that curl, wget, git and most of the system read, which is why `curl https://www.lab.test/` needs no `--cacert` after today. Firefox, Chrome's profile store, Java's `cacerts` and node's bundled roots are all separate, and none of them will trust this certificate. "I added the CA" is always followed by "to what".

"Not filtered" is not "allowed". A rebuilt `node1` may not have firewalld installed at all — Day 11 installed it, and a fresh VM has never run Day 11. With no firewalld, 443 answers perfectly, because nothing is filtering anything. `setup.sh` now installs firewalld if it is missing, starts and enables it, and only then adds the ports, because a port "open" in a policy that no daemon has loaded is a comment. Day 16 taught the daemon-versus-offline split; this is the layer under it — the package itself.

8080 stays closed in firewalld, on purpose. `verify.sh` asserts that it is closed. The backend is private because it is bound to loopback; the closed port is the second layer, and the `--hard` section shows what happens when either one is quietly given up.

The four failures

BREAK	WHAT YOU SEE	WHY
<code>httpd_can_network_connect</code> off	<code>502</code> , <code>Permission denied</code> in the error log	nginx is <code>httpd_t</code> ; the upstream connect is a policy decision
certificate for <code>other.lab.test</code>	<code>openssl verify</code> says OK, every client refuses	chain and dates are one question, hostname is another
backend stopped	<code>502</code> with <code>Connection refused</code>	a refusal is instant; dropped packets would give <code>504</code> after the timeout
certificate replaced, no reload	disk and wire disagree, <code>nginx -t</code> passes	nginx reads certificates once, when the workers start

The `--hard` pair are described rather than caused: binding the backend to `0.0.0.0`, and opening 8080 in firewalld "to test something". Neither one fails. Both serve the same content unencrypted, off to the side of the proxy, with nothing in the access log to say so — because those requests never touch nginx at all. Worth doing once, by hand, with `ss -tlnp` open in another window.

Teardown

On `node1`:

```
sudo ./scripts/teardown.sh          # proxy, backend, ports, name; certs kept
sudo ./scripts/teardown.sh --all    # certificates, CA and trust anchor too
```

Certs kept means `sudo ./scripts/setup.sh` puts the whole thing back in seconds. `--all` removes the CA key as well, so the next run issues a new CA — and anything that trusted the old one now trusts nothing. That is the correct behaviour and a useful thing to have watched happen.

The SELinux boolean is left on by both. It is a policy change, it costs nothing, and turning it off would break any other `httpd_t` service on the host.

Then, on your laptop:

```
./lab/lab.sh down node1
```

Done when

`sudo ./verify.sh` prints **19 passed, 0 failed**, 2 for you to judge, and you can say out loud where TLS stops, what protects the hop after it, and which log line told you the last failure was SELinux and not nginx.

Next up: **Day 18 — Bridges, VLANs and link aggregation.**

DAY 18 Day 18 — Bridges, VLANs and link aggregation

days/day18/README.md

This day is provable, not descriptive: **22 automatic checks** and **2 judgement calls** in `verify.sh`.

Phase: Production operations **Runs on:** this machine — network namespaces, no VM **Memory:** 0 MB **Time:** about seventy-five minutes

Every virtual machine you have started in this course was plugged into a bridge. Every container host has one. A bridge is a switch written in software, and once you turn on VLAN filtering it is a switch with all of the same lies available: a port that is up and carries nothing, a port that is in the VLAN you meant and puts traffic in another one, a trunk that silently stops being a trunk.

Today you build one switch with two VLANs and five things plugged into it, then prove that two hosts on the same wire, in the same bridge, in the same kernel, cannot reach each other — with no firewall anywhere in the topology. Then you bond two cables into one interface and pull one of them out while traffic is running.

This day costs nothing and runs everywhere, because all of it lives in network namespaces on the machine you are reading this on. Namespaces are kernel objects: they do not survive a reboot, so re-running `setup.sh` is normal, not a repair. Day 18's namespaces are named apart from Day 06's, so both topologies can be up at the same time and Days 07–09 keep working.

What you will work with

- `ip link add br0 type bridge vlan_filtering 1`, and what the old `brctl show` cannot tell you, which is everything interesting
- `bridge vlan show`, the only command that shows per-port VLAN membership — `ip link` shows a port as UP whether it is in a VLAN or in none
- **PVID** versus **Egress Untagged**: the tag applied on the way in, and the tag removed on the way out. Two settings, one port, and a host that ends up in the wrong network when they disagree
- access ports versus trunk ports, and why an access host needs no VLAN configuration at all — the switch does the tagging on its behalf
- `ip link add link eth0 name eth0.10 type vlan id 10`, the other half of a trunk: one link, two subinterfaces, two addresses, two networks
- VLAN 1, which is added to every new bridge port for you — and to the bridge device itself, whose own row needs `vid 1 self` to remove. It is the quiet way two "isolated" VLANs end up sharing a third one
- `bridge fdb show`, and the fact that the forwarding database is per VLAN — the same MAC can be reachable in one VLAN and invisible in another
- `ip link add bond0 type bond mode active-backup miimon 100`, enslaving legs, and why the address goes on `bond0` and never on a leg
- `/proc/net/bonding/bond0`, `Currently Active Slave`, and why `active-backup` does not fail back when the old leg returns
- why `802.3ad` is not the default here: LACP needs the switch to agree, and a bond configured for LACP against a switch that is not doing LACP is worse than no bond at all
- `net.ipv4.ip_forward` per namespace, set rather than assumed, because a fresh namespace can inherit a host where Docker or a VPN turned it on

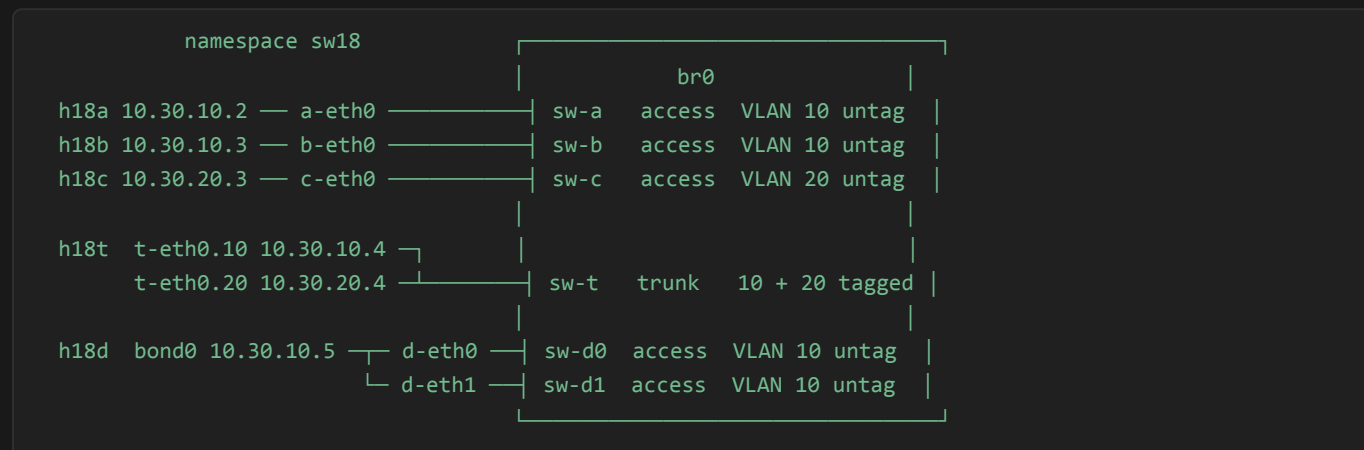
- the difference between segmentation that is enforced at layer 2 and segmentation that is a firewall rule you can turn off

No VM, and the memory

WHERE	MEMORY	WHAT RUNS	WHY
this machine	0 MB	six network namespaces	a bridge, veth pairs and a bond are kernel objects; a VM would add nothing

This is the same trick as Days 06–09. It is also why Day 18 is one of the days CI executes for real on every push: a GitHub runner can build this topology exactly as your laptop does.

The topology



One switch. `br0` has no IP address and the namespace has forwarding off, so there is nothing in this picture that could route between VLAN 10 and VLAN 20 even if you asked it to.

Scripts

SCRIPT	WHAT IT DOES
<code>scripts/setup.sh</code>	nine steps: tools and kernel modules, namespaces, the VLAN-aware bridge, the veth pairs, per-port VLAN membership, addresses, the trunk subinterfaces, the bond, the payload, and a proof that isolation actually holds. Idempotent
<code>scripts/lab-vlan.sh</code>	installed as <code>lab-vlan</code> . <code>status</code> , <code>vlangs</code> , <code>test</code> , <code>bond</code>
<code>scripts/explore-vlangs.sh</code>	twelve read-only stops through the switch you just built
<code>scripts/break-and-fix.sh</code>	four real failures, each fixed in front of you. <code>--hard</code> leaves them live and adds the worst one
<code>scripts/teardown.sh</code>	removes today's six namespaces and nothing else. <code>--all</code> also removes <code>lab-vlan</code> and the backups
<code>verify.sh</code>	twenty-two automatic checks, two for you to judge

Run it

No VM today. From the repository root on your own machine:

```
cd days/day18
sudo ./scripts/setup.sh
```

Then look at what you built:

```
sudo lab-vlan vlans      # per-port membership, and whether filtering is on
sudo lab-vlan test       # every pair that should work, and every pair that must not
sudo lab-vlan bond       # which leg is carrying traffic right now
```

The tour, then the failures, then the check:

```
sudo ./scripts/explore-vlans.sh
sudo ./scripts/break-and-fix.sh
sudo ./verify.sh
```

If `setup.sh` says the bonding module is missing, the VLAN half of the day still runs in full and `verify.sh` will not fail you for a kernel you did not build.

What to actually look at

Do this before anything else. Ping across the VLAN boundary and watch it fail, then find the reason in a table rather than in a firewall:

```
sudo ip netns exec h18a ping -c2 -W1 10.30.10.3      # works
sudo ip netns exec h18a ping -c2 -W1 10.30.20.3      # does not
sudo bridge -n sw18 vlan show
```

Both hosts are in the same bridge and both are up. The only difference is one number in that table.

The two columns that are the whole configuration. In `bridge vlan show`, a port with `PVID Egress Untagged` is an access port: untagged frames arriving get that VLAN, and frames leaving have the tag stripped, so the host on the far end never learns a VLAN exists. A port listing several VLANs with no PVID is a trunk: tagged in, tagged out, and the host has to do the tagging itself. That difference is all a switch port is.

`ip link` **cannot see any of this.** Compare:

```
sudo ip -n sw18 -brief link show master br0
sudo bridge -n sw18 vlan show
```

The first says every port is UP. It says that whether the port is in the right VLAN, the wrong VLAN, or no VLAN at all. When someone reports "the interface is up but there is no traffic", this is usually the pair of commands that ends the conversation.

The bridge is in the VLAN tables too. `br0` appears as its own row, and removing VLAN 1 from it needs the `self` flag: `bridge vlan del dev br0 vid 1 self`. Forget it and the switch itself still has a foot in a VLAN nobody configured — harmless here, and exactly the kind of leftover that turns into a leak on a real switch.

The trunk host is doing work the access hosts are not. `h18t` has two subinterfaces on one link and an address on each; `t-eth0` itself has no address. `h18a` has one plain interface and no VLAN configuration whatsoever. They are on the same switch, in the same VLAN, talking to each other.

Filtering is a flag, and the tables are decoration without it. This is what `--hard` turns off, and it is the failure worth internalising: every line of `bridge vlan show` stays exactly correct while the switch ignores all of it.

The bond moves nothing when a cable dies. Watch `Currently Active Slave` change in `/proc/net/bonding/bond0` while a ping keeps running. The address never moves because it was never on a leg.

The four failures

BREAK	WHAT YOU SEE	WHY
port removed from its only VLAN	link UP, address present, nothing arrives	membership is checked on ingress; <code>ip link</code> has no column for it
right VLAN, wrong PVID	pings still work — to the wrong network	membership and PVID are separate settings; untagged frames follow the PVID
trunk made untagged for one VLAN	one VLAN keeps working, the other goes dark	the tag is stripped on egress and the host's subinterface never sees it
active bond leg taken down	nothing happens, which is the point	the address is on <code>bond0</code> ; failover is invisible above it

And with `--hard` , a fifth: `vlan_filtering 0` , where the configuration is perfect and inert, all four VLAN faults are left live, and `sudo ./scripts/setup.sh` puts everything back.

Teardown

```
sudo ./scripts/teardown.sh          # today's six namespaces
sudo ./scripts/teardown.sh --all    # also lab-vlan and /tmp/day18-broken
```

Deleting a namespace deletes everything inside it, and a veth pair with one end gone disappears entirely — so the bridge, the ports, the VLAN subinterfaces and the bond all go with it. There is nothing left behind to clean up, and Day 06's namespaces are untouched.

A reboot does the same thing whether you ask for it or not.

Done when

`sudo ./verify.sh` prints **22 passed, 0 failed**, 2 for you to judge, and you can point at a line of `bridge vlan show` and say whether that port is an access port or a trunk, what happens to an untagged frame arriving on it, and why the isolation you just proved is not a firewall rule.

Next up: **Day 19 — Intrusion detection and audit alerting.**

DAY 19 Day 19 — Intrusion detection and audit alerting

days/day19/README.md

This day is provable, not descriptive: **22 automatic checks** and **2 judgement calls** in `verify.sh`.

Phase: Production operations **Runs on:** `node1` **Time:** about ninety minutes

Days 11 to 13 were prevention: a firewall, a hardened sshd, SELinux in enforcing mode. Prevention has a failure mode you cannot see from the inside — it either worked or something got through, and nothing on the machine tells you which.

Today you install the two sensors that answer the two questions nobody can answer by watching a screen. **Suricata** sees what crossed the network interface. **auditd** sees which syscall touched which file, and which *login* account was behind it. Neither one prevents anything. That is not a gap in the setup, it is the category — and it is why both of them exist on the same host as Day 11's firewall rather than instead of it.

Installing them is one `dnf` line. The work is that a detection stack has a failure mode of its own: running, enabled, valid configuration, zero detection. Four of today's five failures look exactly like a healthy machine.

What you will work with

- `suricata -T -c /etc/suricata/suricata.yaml`, the config-and-rules test — one missing semicolon in one rule stops the whole engine rather than that rule, so this runs before every reload
- the `af-packet` section, which names the interface to capture from. The packaged default is `eth0` and this VM's interface is not called that; an IDS on the wrong interface is the quietest outage in this course
- `rule-files`, absolute paths versus names under `default-rule-path`, and why your own rules never live where `suricata-update` can overwrite them
- rule syntax — `action proto src port -> dst port (options)`, `msg` as the line you will read at 3am, `itype`, the `dns.query` sticky buffer, `classtype`
- sid numbering, which is a namespace and not a label: reuse a public ruleset's number and one of the two rules silently loses
- `eve.json`, one JSON object per line, with alerts, flows, DNS, TLS and stats all in the same file, and `jq` as the only sane way to read it
- `auditctl -w path -p wa -k key`, `-a always,exit -F arch=b64 -S execve`, and why the key is not a comment
- `audit` versus `uid` — the login uid survives `su` and `sudo`, so it answers "who did this" where `uid` only answers "as whom"
- `/etc/audit/rules.d/` with `augenrules --load`, and `augenrules --check`, the only command that compares the kernel's rules with your files
- `systemctl restart auditd` being refused, which is correct and surprises everyone exactly once
- `ausearch -k`, `ausearch -i`, `aureport --summary` — searching by key, translating numbers into names, and counting before reading
- EPEL, again: Suricata is not in the base Rocky repositories, the same way Day 12's fail2ban was not

One VM, and the memory

VM	MEMORY	WHAT RUNS ON IT	WHY
node1	2048 MB	Suricata, auditd	Suricata's detection engine is the memory in this day; auditd costs almost nothing

About **2 GB** in total, the same single VM as Day 17. Nothing in this lab is below 1536 MB, which is the minimum Rocky 9 recommends and the number `virt-install` warns about.

No ruleset is downloaded. `suricata-update` is deliberately not run — today is about rules you wrote, and the Emerging Threats open ruleset would not be kind to a 2 GB VM. `setup.sh` creates an empty `/var/lib/suricata/rules/suricata.rules` so the packaged configuration stays valid, and adds your own file to `rule-files` by absolute path.

Days 14 to 18 are not prerequisites. Today touches nothing they built — no `ansible.cfg`, no project directory, no tunnel, no bridge. The day it leans on is Day 12, for the EPEL repository and for the habit of reading logs a daemon wrote about somebody else's traffic.

The shape you are building:



Two daemons, two blind spots, and no connection between them. Suricata knows nothing about files; auditd knows nothing about packets. An attacker who logs in with a stolen key produces no interesting packets, and an attacker exploiting a service produces no interesting logins — which is why the answer is both and not either.

Scripts

SCRIPT	WHAT IT DOES
scripts/setup.sh	eight steps: packages from EPEL, interface detection, <code>suricata.yaml</code> , two local rules, <code>-T</code> then the service, audit rules plus <code>augenrules --load</code> , the payload, and a live trigger of both sensors that fails the run if nothing was recorded. Idempotent
scripts/lab-ids.sh	installed as <code>lab-ids</code> . <code>status</code> , <code>alerts</code> , <code>audit</code> , <code>watch</code> , <code>prove</code> , <code>trigger</code>
scripts/explore-detection.sh	twelve read-only stops across both sensors
scripts/break-and-fix.sh	four real failures, each fixed in front of you. <code>--hard</code> leaves all of them live and adds a fifth
scripts/teardown.sh	lab rules, the canary and the payload removed. <code>--all</code> also stops Suricata and restores the packaged config
verify.sh	twenty-two automatic checks, two for you to judge

Run it

On your laptop, in the repository:

```
./lab/lab.sh up node1           # ~2 GB
./lab/lab.sh status             # read the address
./lab/lab.sh push node1        # carries days/ and lab/
./lab/lab.sh ssh node1
```

lab.sh status prints something like:

```
node1      running    192.168.122.140    # EXAMPLE address — use your own
```

That is an example. DHCP leases move every time a VM is rebuilt, so read it yourself each session rather than copying it from here.

Then, on node1:

```
cd ~/lab/days/day19
sudo ./scripts/setup.sh
```

It enables EPEL if it has to, detects the interface your default route actually uses, writes two rules of your own with local sids, loads audit rules for /etc/shadow, /etc/sudoers, a canary file and every root command run from a login session — and then **fires both sensors on purpose** and fails if neither recorded anything. A detection stack nobody has ever seen fire is a detection stack you do not know works.

Then, in order:

```
sudo lab-ids           # both daemons, the capture interface, rule counts
sudo lab-ids alerts     # eve.json, readably
sudo lab-ids audit      # what auditd recorded, by key
sudo lab-ids watch      # follow alerts live, Ctrl-C to stop
sudo lab-ids prove      # trigger, wait, and time the flush
sudo lab-ids trigger     # fire both sensors again, on demand
sudo ./scripts/explore-detection.sh # read the stack you just built
sudo ./scripts/break-and-fix.sh    # four failures, four fixes
sudo ./scripts/break-and-fix.sh --hard
sudo ./verify.sh
```

If you edit the scripts on your laptop afterwards, push them again before you re-run anything — the VM has its own copy and will happily keep running the old one:

```
./lab/lab.sh push node1
```

Seeing the flush interval for yourself

Worth doing once, before verify.sh. One session, no coordination:

```
sudo lab-ids prove
```

It counts the alerts in eve.json, fires both sensors, then waits up to 90 seconds for the count to go up and tells you how long it actually took. That delay is the flush interval, and everything anyone ever builds on top of an IDS — dashboards, tickets, pages — reads that file after it.

The delay is the point, and it is longer than it feels like it should be. Suricata batches writes to eve.json; tens of seconds is normal on a 2 GB VM. If you fire the sensors and look immediately, you see nothing, and nothing looks exactly like broken.

If you want to watch it happen rather than be told about it, that takes **two separate SSH sessions to node1** — the watching command never returns on its own, so pasting both into one prompt just glues them together and gets you a `jq` parse error. On your laptop, open a second terminal and run `./lab/lab.sh ssh node1` in it, so you have two sessions on the VM:

```
# session A - Leave this running; it prints a heartbeat every 15s
sudo lab-ids watch

# session B
sudo lab-ids trigger
```

Then wait. Up to a minute. `Ctrl-C` session A when the alert lines appear. The heartbeat is there so you can tell "waiting" from "hung" — which is exactly the ambiguity `lab-ids prove` exists to remove.

Neither command is magic. `lab-ids watch` is only this, if you would rather type it out:

```
sudo tail -n 0 -F /var/log/suricata/eve.json \
| grep --line-buffered '"event_type":"alert"' \
| jq -c --unbuffered '{sid: .alert.signature_id, msg: .alert.signature}'
```

What to actually look at

The interface, twice. These two must name the same device:

```
sudo awk '/^af-packet:{f=1} f && /interface:/{print; exit}' /etc/suricata/suricata.yaml
ip route show default
```

When they disagree, `systemctl status suricata` is green, `suricata -T` passes, the rules are loaded, and nothing will ever alert. There is no error anywhere on the machine, and nothing in the logs to grep for. That is the `--hard` failure, and it is the reason `setup.sh` reads the device from the routing table instead of trusting the packaged `eth0`.

active is not ready. This one cost the first run of this day. Suricata parses the config, builds a detection engine out of every rule, creates capture threads, and only then looks at a single packet. `systemctl is-active` goes green at the start of that, not the end, and on a 2 GB VM the rest takes tens of seconds. Traffic sent in the gap did not happen as far as the IDS is concerned, and nothing in `eve.json` says so — there is simply no alert. The line that means ready lives in `suricata.log`:

```
sudo grep 'Engine started' /var/log/suricata/suricata.log
sudo lab-ids status # prints it, and the rule count
```

`setup.sh` waits for that line, then sends traffic on a loop rather than once, because `eve.json` is flushed on an interval as well. A health check that asks systemd whether a sensor is working gets this wrong on every IDS, agent and log shipper there is.

auditd buffers, so "no event" can mean "not written yet". The kernel hands each record to auditd the moment it happens, but `auditd.conf` ships `flush = INCREMENTAL_ASYNC` with `freq = 50`, so auditd writes `audit.log` every 50 records. Read a watched file once on an idle VM and `ausearch` finds nothing, while the event has in fact been recorded and is sitting in a buffer. This is exactly how a log search can say "clean" about a machine that is not:

```
sudo grep -E '^(flush|freq)' /etc/audit/auditd.conf
for i in $(seq 1 60); do sudo cat /etc/lab-canary >/dev/null; done
sudo ausearch -k lab_canary -ts today | tail
```

`setup.sh` generates a burst past `freq` rather than a single access, for this reason.

The file and the kernel, for audit rules. `auditctl -l` prints what the kernel is enforcing;

`/etc/audit/rules.d/` holds what you intended. They can disagree in both directions — a rule written and never loaded, or a rule loaded by hand and never saved:

```
sudo augenrules --check      # 'No change' is the only good answer
```

This is the same lesson as Day 11's `sysctl` files, Day 16's `wg showconf` and Day 17's certificate-without-a-reload, and it is the fourth appearance on purpose: **a configuration file is a request, not a state.**

One bad rule takes down every rule. The ruleset is compiled as a unit. A missing semicolon does not disable that signature, it stops the engine from starting — so a reload without `suricata -T` first can leave a host with no detection at all, which is strictly worse than the one rule you were adding.

audit is the field that matters. Read one event in full:

```
sudo ausearch -k shadow_watch -ts today -i | tail -20
```

`uid=0` tells you almost nothing on a machine where everybody uses `sudo`. `audit` is the account that logged in, it survives `su` and `sudo`, and it is the reason `auditd` answers questions `journalctl` cannot.

The canary is the shape a good rule has. `/etc/lab-canary` is watched with `-p rwa`, so even reading it is recorded. Nobody has a legitimate reason to read it, so every single event is interesting — and a rule whose every event is interesting is the only kind worth alerting on. Compare it with the ICMP rule, which is a fine lab rule and would be pure noise in production. Naming that distinction is one of the two judgement calls.

Alert volume is where tuning starts.

```
sudo jq -r 'select(.event_type=="alert") | .alert.signature' \
/var/log/suricata/eve.json | sort | uniq -c | sort -rn
```

The signature at the top of that list is either your most important detection or the one nobody will ever read again. An IDS is a log generator; if nothing reads the log on a schedule you have bought storage costs and a feeling.

sids are a namespace. 1000000–1999999 is the local range; anything lower belongs to a public ruleset.

Today's rules use the 9000000 block so lab rules are obvious in a log, and `verify.sh` asserts that no lab sid sits in the public ranges — a collision means your rule or theirs quietly loses, with no error either way.

auditd is not restarted with systemd. Try it:

```
sudo systemctl restart auditd
```

The refusal is deliberate — `auditd` is started by the kernel's audit subsystem, not by you. Rules are reloaded with `augenrules --load`, and the daemon itself with `service auditd restart` if you genuinely must.

Neither sensor stops anything. Worth saying out loud once, because the instinct after Days 11–13 is to look for the blocking switch. Suricata has one — IPS mode, inline, with `drop` rules — and it is not today's lab,

because an IPS you have not tuned is an outage generator pointed at your own users.

The four failures

BREAK	WHAT YOU SEE	WHY
a rule missing its semicolon	<code>suricata -T</code> fails for every rule	the ruleset compiles as one unit; reloading without testing leaves no detection at all
a valid rule never reloaded	the file has it, the engine does not, nothing alerts	editing a rule file is a request; <code>systemctl reload suricata</code> is the state change
an audit rule written but not loaded	<code>grep</code> in <code>rules.d</code> passes, <code>auditctl -l</code> is missing it	an audit built by reading files would pass this machine; <code>augenrules --check</code> is what catches it
<code>auditctl -D</code>	every file is perfect and nothing is recorded	the kernel's rule set is runtime state, and no configuration file reflects that it was emptied

And with `--hard`, a fifth: Suricata capturing on `lo`. Running, enabled, config valid, `-T` passing, rules loaded, watching an interface where none of the traffic you care about goes. All five are left live at once, `verify.sh` should fail, and `sudo ./scripts/setup.sh` puts everything back.

Teardown

On `node1`:

```
sudo ./scripts/teardown.sh          # lab rules, the canary, the payload
sudo ./scripts/teardown.sh --all    # also stops suricata, restores its config
```

Deleting an audit rules file is not enough — the rules are in the kernel until something reloads them, so teardown runs `augenrules --load` and then checks that the watches are really gone rather than assuming it.

auditd is never stopped, and `eve.json` is never deleted, by either mode. Turning off the record of who did what in order to tidy up is a habit worth not building, and deleting logs during cleanup is how incidents become unexplainable.

Then, on your laptop:

```
./lab/lab.sh down node1
```

Done when

`sudo ./verify.sh` prints **22 passed, 0 failed**, 2 for you to judge, and you can say which of today's two sensors would have caught a stolen SSH key, which would have caught a port scan, why neither would have stopped either, and what `augenrules --check` compares.

Next up: **Day 20 — Backup, restore and the restore drill.**

DAY 20 Day 20 - Backup, restore and the restore drill

days/day20/README.md

This day is provable, not descriptive: **22 automatic checks** and **2 judgement calls** in `verify.sh`.

Prove you can get the data back, on a schedule, and know how long it takes.

PHASE	PRODUCTION OPERATIONS
Runs on	VM: control + node1
Memory	~3.5 GB (two VMs)
Verified by	lint + your lab

Why this day exists

An untested backup is a rumour. This is the last day because it is the one that decides whether everything before it was worth configuring.

Two sentences get said in incidents, and only one of them is useful. "We have backups" is a hope. "A full restore of that dataset took four minutes and I ran it on Tuesday" is an answer. Today you produce the second sentence about your own lab, with a number you measured yourself.

Three ideas people skip, built into the day:

- A backup on the same disk as the data is not a backup. So the repository lives on **control**, and **node1** writes to it over ssh.
- A backup nobody has restored is a rumour. So `setup.sh` finishes by restoring into a scratch tree and comparing it with `diff -r` - and fails if they differ.
- A backup on a schedule is only real once the schedule has fired. So the timer is set up to fire within thirty seconds, and both setup and verify look at `LastTriggerUsec` rather than at `enabled`.

What you will work with

- `restic init / backup / snapshots / ls / diff` - a content-addressed, deduplicating, encrypted repository
- `restic restore --target` and `diff -r` - the restore drill, and the only proof that matters
- `restic check`, `restic check --read-data` - integrity, which is not the same as the files being present
- `restic forget --keep-daily --keep-weekly --prune` - retention, so the repository does not fill the disk it lives on
- `restic unlock` - clearing a stale lock left by a killed run
- `sftp:user@host:/path` repositories over ssh, with a key instead of a password
- `RESTIC_PASSWORD_FILE` VS `RESTIC_PASSWORD` - why the path is the safe half
- `systemd` timers: `OnCalendar`, `OnActiveSec`, `Persistent`, `AccuracySec`, `list-timers`
- `EnvironmentFile=` - the reason a backup can work by hand and fail on the timer
- `useradd --shell` and why `/sbin/nologin` quietly breaks sftp

The shape of it

```
node1                                control
+-----+                          +-----+
| /srv/data          the data |    | user restic, /bin/bash |
| /etc/restic/password 0600 |    | /srv/restic/repo      0700 |
| /etc/restic/env       |    | /srv/restic/.ssh/         |
| /usr/local/bin/lab-backup |    |   authorized_keys      |
| restic-backup.service |    |                               |
| restic-backup.timer    | ----> | no restic installed   |
| /root/.ssh/id_ed25519  | ssh  | no password, cannot read it |
| /var/tmp/restore the drill |    |                               |
+-----+                          +-----+
```

The direction matters. node1 pushes to control; control has no key to node1 and no way to read what it stores. If control is compromised, the attacker gets encrypted blobs. If node1 is compromised, the attacker can delete snapshots - which is why the real world adds append-only repositories, and why you should know that this lab does not.

Scripts for today

SCRIPT	WHAT IT DOES
<code>scripts/setup.sh</code>	Role-aware. On control: the <code>restic</code> user and the repository. On node1: restic, the data, the key, the password, <code>restic init</code> , the payload, the unit, the timer, and the first restore drill.
<code>scripts/lab-backup.sh</code>	Installed as <code>/usr/local/bin/lab-backup</code> . Subcommands: <code>status</code> , <code>run</code> , <code>snapshots</code> , <code>drill</code> , <code>check</code> , <code>forget</code> , <code>unlock</code> . Also the <code>ExecStart</code> of the service.
<code>scripts/explore-backup.sh</code>	Twelve read-only stops: the repository, dedup, what is inside a snapshot, the unit, the timer, the journal, the restore. Changes nothing.
<code>scripts/break-and-fix.sh</code>	Four failures, each diagnosed and repaired. <code>--hard</code> leaves a fifth for you: green everywhere, and the snapshots are empty.
<code>scripts/teardown.sh</code>	Role-aware. Removes the timer, unit, payload and restore tree. <code>--all</code> also removes the password, the data, and (on control) the repository.

The whole day, in order

Everything below is a step, and every step says which host it runs on. Top to bottom, once:

	WHERE	COMMAND
Step 1	laptop	<code>./lab/lab.sh up control node1 , status , push both, ssh both</code>
Step 2	control	<code>sudo ./scripts/setup.sh</code>
Step 3	node1	<code>sudo ./scripts/setup.sh</code> - prints a key, exits 0
Step 4	control	<code>sudo ./scripts/setup.sh '<the key it printed>'</code>
Step 5	node1	<code>sudo ./scripts/setup.sh <control-ip></code> - the real work
Step 6	node1	<code>sudo lab-backup drill</code> - the restore drill
Step 7	node1	<code>sudo ./scripts/explore-backup.sh</code> - read what you built
Step 8	node1	<code>sudo ./scripts/break-and-fix.sh</code> - four failures
Step 9	node1	<code>sudo ./verify.sh</code> - 22 checks
Optional	node1	<code>sudo ./scripts/break-and-fix.sh --hard</code> - the silent one
When done	node1, then control	<code>sudo ./scripts/teardown.sh</code>

Nothing runs on control except steps 2 and 4. Control creates the `restic` user, authorises node1's key, and is finished. Every other command today is on node1.

Step 1 - both VMs, from your laptop

On your laptop, in the repository:

```
./lab/lab.sh up control node1      # ~3.5 GB
./lab/lab.sh status                # read BOTH addresses
./lab/lab.sh push control          # carries days/ and lab/
./lab/lab.sh push node1           # today runs on both hosts
./lab/lab.sh ssh control
```

Both hosts need the repository today, same as Day 16. `push <vm>` with no path copies `days/` and `lab/` ; `push <vm> <path>` copies **only** that one file.

`lab.sh status` prints something like:

```
control  running  192.168.122.108  # EXAMPLE address - use your own
node1    running  192.168.122.134  # EXAMPLE address - use your own
```

Those are examples. DHCP leases move every time a VM is rebuilt, so read them yourself each session rather than copying them from here.

You want **two terminals** for this day - one SSH session per host - because the two setup passes alternate between them. On your laptop, in a second terminal:

```
./lab/lab.sh ssh node1
```

Steps 2 to 5 - setup, in four passes

Setup runs **four times**, alternating hosts, because a public key has to travel between two machines and no single command on one machine can do that. Each pass tells you the next one.

Step 2, on control:

```
cd ~/lab/days/day20
sudo ./scripts/setup.sh
```

Creates the `restic` user, the repository directory, and stops.

Step 3, on node1:

```
cd ~/lab/days/day20
sudo ./scripts/setup.sh
```

Installs restic, creates `/srv/data`, generates root's ssh key, prints the exact command to run on control, and exits 0. That exit is not a failure - read what it printed.

Step 4, on control: paste the command it printed, key and all:

```
sudo ./scripts/setup.sh 'ssh-ed25519 AAAA... root@node1'
```

Step 5, on node1, with control's address:

```
sudo ./scripts/setup.sh 192.168.122.108
```

That pass does the real work: the password file, the environment file, `restic init`, the payload, the unit, the timer, a wait of up to 90 seconds for the timer to actually fire, and the restore drill. It prints the restore time. Write it down.

Until step 5 finishes there is no `/etc/restic/env` and no `/usr/local/bin/lab-backup`, so `lab-backup` and `explore-backup.sh` will tell you to run setup first. That is the expected answer, not a fault.

If you edit the scripts on your laptop afterwards, push them again before you re-run anything - each VM has its own copy and will happily keep running the old one:

```
./lab/lab.sh push control
./lab/lab.sh push node1
```

Re-running setup after a script change does not need a teardown. `setup.sh` is written to be run again: it leaves `/srv/data`, `/etc/restic` and root's ssh key alone, and rebuilds the payload, the unit and the timer from the new copy. So one pass on node1 is enough, and control is not involved at all:

```
# on node1, with control's address
cd ~/lab/days/day20
sudo ./scripts/setup.sh 192.168.122.108
```

The address is also remembered in `/etc/restic/control-ip`, so `sudo ./scripts/setup.sh` with no argument works too once step 5 has succeeded once. Only tear down when you want the schedule, the data or the repository *gone* - see the last section.

Step 6 - the drill, on node1

Step 5 already ran this once. Run it again yourself, because this is the part to take seriously:

```
sudo lab-backup drill
```

It restores `latest` into `/var/tmp/restore`, times it, runs `diff -r /srv/data /var/tmp/restore/srv/data`, compares the mode of a 0600 file, and writes the number of seconds to `/var/lib/lab-backup/last-restore-`

seconds .

Three things it does that a casual restore does not:

- It restores into an **empty** tree every time. Restore on top of last week's restore and a missing file will pass a `diff` .
- It compares, and the comparison decides the exit status. A restore that ran is not a restore that worked.
- It checks permissions. Bytes back with the wrong mode means you just made a secret world-readable during an incident.

Step 7 - look at what you built, on node1

The read-only tour. It changes nothing, so there is no way to get this wrong:

```
sudo ./scripts/explore-backup.sh
```

And the things worth running on their own:

```
sudo lab-backup status          # repository, schedule, last run, last restore time
sudo lab-backup snapshots       # what is actually in the repository
sudo lab-backup run             # take a snapshot now, then apply retention
sudo lab-backup check           # integrity of the repository structure
systemctl cat restic-backup.service
systemctl list-timers restic-backup.timer --no-pager
journalctl -u restic-backup.service -n 30 --no-pager
```

The two commands people never run until it is too late:

```
sudo lab-backup snapshots      # is there a snapshot at all, and how big
sudo lab-backup drill          # does it come back, and in how long
```

Both of those go through `lab-backup` , which loads the environment for you. `restic` on its own does not have it, and `/etc/restic/env` is 0600 root-only, so `. /etc/restic/env` as your own user gives you `Permission denied` and `restic` then says `Fatal: Please specify repository location` . Load it inside a root shell instead:

```
sudo bash -c 'set -a; . /etc/restic/env; set +a; restic snapshots'
sudo bash -c 'set -a; . /etc/restic/env; set +a; restic ls latest | wc -l'
```

Setup also symlinks the payload to `/usr/sbin/lab-backup` . Without that, `sudo lab-backup` reports `command not found` even though `/usr/local/bin/lab-backup` exists and is on your own `PATH` - because `sudo` ignores your `PATH` and uses `secure_path` from `/etc/sudoers` , which on Rocky does not list `/usr/local/bin` . If you are on a host set up before this fix, use the full path: `sudo /usr/local/bin/lab-backup status` .

In `lab-backup status` , read **last run exit** . A `1` there with snapshots in the repository means your backups are only happening when you run them by hand - which is the state this day is about. `journalctl -u restic-backup.service` tells you why in one line.

Two columns in `list-timers` are worth trusting: **LAST** and **NEXT** . `systemctl is-enabled` only promises the timer will start at boot. It says nothing about whether a backup has ever happened.

`lab-backup unlock` is deliberately not in the list above. There is no lock to clear until something is killed mid-write, which is exactly what step 8 does - so the only place you need `unlock` today is failure 4 and the repair that follows it.

One thing to know before step 9: the timer is `OnCalendar=*:0/10`, so a backup starts at every tenth minute of the hour and holds a lock on the repository while it writes - an exclusive one while `forget --prune` runs. That is normal, not a fault. `verify.sh` now waits for a running backup to finish before it touches the repository, so you no longer have to time your own commands around the schedule. If you want the schedule out of the way while you experiment:

```
sudo systemctl stop restic-backup.timer      # quiet while you poke at it
sudo systemctl start restic-backup.timer     # put it back when you are done
```

Step 8 - break it on purpose, on node1

```
sudo ./scripts/break-and-fix.sh
```

Four failures, each one shown, diagnosed and repaired:

1. **The password file is gone.** The repository is intact and unreadable, which is the same thing as lost. This is the only failure in the file you cannot fix from this host.
2. **The repository host is unreachable.** A network error wearing a backup error's clothes. Prove the transport with `ssh` before you suspect restic.
3. **It works by hand and fails on the timer.** `EnvironmentFile=` is removed from the unit. Your shell has `RESTIC_REPOSITORY`; systemd does not, and never did.
4. **A stale lock.** A backup is killed mid-write, the lock outlives it, and every run after that refuses. `restic unlock`, never `rm` - and note that plain `unlock` removes only locks it can *prove* are stale. A lock it cannot prove is stale survives, `verify.sh` keeps failing `no stale lock is holding the repository`, and clearing it needs a deliberate `restic unlock --remove-all` once you have checked that nothing is backing up on any host. `sudo lab-backup unlock` does exactly that escalation, and refuses to escalate while `restic-backup.service` is actually running.

Then, when you want the one that teaches the most:

```
sudo ./scripts/break-and-fix.sh --hard
```

An exclude pattern swallows `/srv/data`. The service exits 0, a new snapshot appears, the timer is green, and any dashboard you could build from systemd says the backup is healthy. The snapshots contain nothing.

Nothing is hidden - `systemctl cat restic-backup.service`, `cat /etc/restic/exclude` and `restic ls latest` tell you everything. Fix it, then prove the fix with `sudo lab-backup drill` rather than with a green unit. The original unit is kept at `/tmp/day20-broken/restic-backup.service.prehard`.

The full repair by hand. The order is the lesson - the exclude file goes first because the unit is what reads it, and the backup goes last because it is the only thing that puts content back into `latest`:

```
sudo rm -f /etc/restic/exclude
sudo install -m 0644 /tmp/day20-broken/restic-backup.service.prehard \
/etc/systemd/system/restic-backup.service
sudo systemctl daemon-reload
sudo lab-backup unlock && sudo lab-backup run && sudo lab-backup drill
```

All four lines matter, in that order:

1. `rm` the exclude file - while it exists every backup you take is empty, so doing this after the backup fixes nothing.
2. reinstall the saved unit - it is what pointed at the exclude file.
3. `daemon-reload` - until you do, systemd is still holding the broken unit in memory.
4. `unlock && run && drill` - `unlock` clears the lock failure left behind on purpose, `run` writes a snapshot that has content in it, and `drill` is the only line that proves any of it worked.

Restoring the unit alone leaves the empty snapshot as `latest`, so the drill still reports `DIFFERENT` - this is what an untested backup looks like and verify still fails a restore reproduces the source tree exactly.

Running `verify.sh` while the day is still broken is supposed to fail - 18 passed, 4 failed is the correct score for that state, not a bug.

When a lock will not go away

`verify.sh` asks about locks before it runs `restic check`, because `check` takes an exclusive lock itself - a lock question asked after it can end up reporting your own check. `sudo lab-backup unlock` prints the contents of each lock first, which names the host and pid that wrote it.

A lock is only removable if nothing owns it. If `no stale lock is holding the repository` stays red **after** an unlock, the lock is not stale - some restic process is still alive and refreshing it every few minutes. One live lock takes three checks down with it, which is why this looks like three unrelated problems:

RED LINE	WHY THE LOCK DOES IT
<code>no stale lock is holding the repository</code>	the lock is there
<code>the repository passes an integrity check</code>	<code>restic check</code> needs the repository to itself and refuses
<code>the scheduled service has run and exited 0</code>	the timer fired, hit the lock, and exited 1

So find the owner before you remove anything:

```
pgrep -af restic           # is a backup still running?
sudo pkill -9 -f 'restic backup' # only once you know it is an orphan
sudo lab-backup unlock      # now the lock really is stale
sudo lab-backup run && sudo lab-backup check
sudo systemctl start restic-backup.service # prove the timer path works again
sudo ./verify.sh
```

`restic unlock --remove-all` on a lock that still has a living owner is pointless - the owner takes it again a second later, and on a shared repository you have just let two backups write at once.

If `verify.sh` straight after this repair shows `at least one snapshot exists` Or `no stale lock is holding the repository` red, look at `systemctl is-active restic-backup.service` before you suspect the repair. A scheduled run locks the repository, and a locked repository fails both of those checks. Current `verify.sh` waits that out for you.

If you would rather start the day over than repair it, do it **on node1 only**. Both commands below run on node1, and you do not touch control at all - `--hard` only ever changed node1:

```
# on node1, in ~/Lab/days/day20
sudo ./scripts/setup.sh 192.168.122.108
```

One pass is enough, and no teardown is needed: setup removes the leftover exclude file, clears a stale lock, rewrites the unit and timer, notices if `latest` is empty and takes a snapshot with content in it, then re-runs the drill. Add `sudo ./scripts/teardown.sh` first only if you also want the data and the password gone.

That address is the **argument**, not another host to log into: it is control's IP, which is how node1 is told where the repository lives. Same value you used in step 5, and node1 saved it at `/etc/restic/control-ip` if you have forgotten it - so `sudo ./scripts/setup.sh` with no argument works too.

This is the failure that ends companies. Not a backup that errors - those get noticed. A backup that reports success for eleven months and restores an empty directory.

Step 9 - verify, on node1

Run on **node1**, with sudo, because the checks read root-only files and talk to the repository with root's key:

```
cd ~/lab/days/day20
sudo ./verify.sh
```

If it prints `a scheduled backup is running - waiting for it to finish`, that is the timer rather than a problem: the checks start when the run ends.

Checked automatically:

- ☐ the repository environment file exists and is root-only
- ☐ the repository is on another host, not this one
- ☐ the environment names a password file, not a password
- ☐ the password file exists and is 0600 root:root
- ☐ the data being protected still exists
- ☐ the repository exists and is readable
- ☐ at least one snapshot exists
- ☐ the repository passes an integrity check
- ☐ the latest snapshot contains the data path
- ☐ the latest snapshot is not empty
- ☐ no stale lock is holding the repository
- ☐ the backup payload is installed and executable
- ☐ a retention policy is configured
- ☐ the retention policy prunes, not just forgets
- ☐ the backup service unit exists
- ☐ the unit carries the repository in its own environment
- ☐ the scheduled service has run and exited 0
- ☐ backups run on a timer
- ☐ the timer is loaded and running right now
- ☐ the timer has actually fired at least once
- ☐ a restore reproduces the source tree exactly
- ☐ the restore preserved file permissions

Only you can confirm:

- ☐ you timed a full restore and can state the number in minutes
- ☐ the repository password is not stored next to the repository

CI can only lint this day. Nothing on a GitHub runner has SELinux, firewalld, systemd units you control, or a second host to reach over SSH - so the checks above are proven by running `verify.sh` on your own lab, and nowhere else.

When you are done - tear it down

Two hosts, so there are two teardowns, and the order matters if you want the repository gone: node1 first, control second.

On node1 - removes the timer, the unit, `/usr/local/bin/lab-backup`, the exclude file, `/var/tmp/restore`, `/var/lib/lab-backup` and `/tmp/day20-broken`:

```
sudo ./scripts/teardown.sh
```

It deliberately keeps `/etc/restic` and `/srv/data`. Delete the password and every snapshot on control becomes unreadable, so that needs the flag:

```
sudo ./scripts/teardown.sh --all      # also deletes /etc/restic and /srv/data
```

On control - nothing happens without `--all`, because the repository is the only copy of the data:

```
sudo ./scripts/teardown.sh           # prints what exists, changes nothing
sudo ./scripts/teardown.sh --all      # deletes /srv/restic and the restic user
```

To rebuild afterwards, you are back to the full four passes - the key exchange has to happen again. Or just delete both VMs from your laptop, which is faster and takes the disks with them:

```
./lab/lab.sh down control node1      # deletes both VMs and their disks
```

Notes

Keep your own notes here. What broke, what the error actually said, and what fixed it - that is the part you will come back for.

Things this day deliberately leaves unfinished, so you know they exist:

- **Append-only repositories.** node1 can delete snapshots it wrote. Real setups use `rest-server --append-only` or a storage backend with object lock, so ransomware on the client cannot erase the backups.
- **Off-host password custody.** The password lives only on node1. Lose that host completely and the repository is landfill. In production the password goes in a password manager or a KMS, not next to the machine it protects.
- `restic check --read-data`. Today's check verifies structure. Reading every byte back is the only version that catches storage silently returning the wrong data, and it is slow enough to schedule weekly rather than hourly.
- **Monitoring the absence of backups.** Nothing here pages you when the timer stops firing. `OnFailure=`, a dead-man's-switch ping, or a check on snapshot age are each one small unit away.

That is the curriculum. There is no capstone by design - the lab itself was the project, and it is still running.

CONTRIBUTING.md

This is a learning repository with a lab attached. That shapes everything below: the goal is not the largest possible feature set, it is that **every claim in here is true**.

Issues and pull requests are welcome. Fixing something that does not work on your distribution, or on your hardware, is the most useful contribution there is.

The three rules that matter

1. Never claim something works that has not been executed

This repository distinguishes carefully between *lint-clean* and *verified*, and that distinction is load-bearing.

- 7 of the 20 days can be executed for real on a GitHub runner, because an Ubuntu runner is a full VM with `sudo`. Those days are genuinely tested in CI: namespaces, DNS servers, `tcpdump`, `openssl`, LVM on a loopback file.
- The other 13 days need SELinux, firewalld, systemd units you own, or a second host over SSH. CI can only lint them. Their checks are proven by running `days/dayNN/verify.sh` on your own lab, and nowhere else.

Each day's README states which of the two it is. If you add or change a day, say plainly what is proven and by whom. A green check must never imply more than it tested.

If you write something you could not run, say so in the pull request. That is not a failing — it is the difference between a useful contribution and a plausible one.

2. Never add a simulation, mock, or offline mode

However convenient it looks. If a thing cannot be verified for real on the hardware described in `lab/README.md`, the honest move is to say so, not to fake the output and print a green tick.

One clarification, because it comes up: **network namespaces are not a simulation**. They are real interfaces, real routing tables, real packets, real captures — the same kernel machinery containers are built from. That is precisely why five days of networking cost no memory. Do not describe them as fake, and do not "upgrade" them to VMs for realism they already have.

3. Nothing dangerous runs on the contributor's host

Every command that hardens, firewalls, relabels, partitions or locks something out must run **inside a VM**, which is disposable. The host is only ever allowed to run `lab.sh`, `virsh`, `ssh` and `scp`.

The namespace days are the single exception and are safe by construction: `netns-up` creates namespaces and veth pairs, edits no file anywhere, and `netns-down` removes all of it. So does a reboot.

A patch that expects the reader to `dnf install` something on their laptop, or edits a file outside `$LAB_HOME`, will be sent back.

Before you open a pull request

```
./tests/cli.sh # 100+ checks, no VM and no root needed
```

Optionally, and recommended:

```
pip install --user pre-commit && pre-commit install
pre-commit run --all-files      # shellcheck, whitespace, large-file guard
```

If you changed anything a day does, also run that day's checks on a real lab and paste the output:

```
./lab/lab.sh push control
./lab/lab.sh ssh control
cd ~/lab/days/dayNN && sudo ./verify.sh
```

Keeping a day consistent

A day is three files that have to agree with each other:

FILE	WHAT IT HOLDS
days/dayNN/README.md	the objective, the work, the checks, what CI proves
days/dayNN/verify.sh	those same checks, as assertions
days/dayNN/scripts/*	the scripts the day walks you through

If you add a check to `verify.sh`, add it to the README's check list too, and the other way round. A page that claims a check its verifier does not run is the one bug this repository cannot tolerate, because the whole point is that the claims are true.

The same goes for the tier label and RAM figure at the top of each day page: if a day starts needing a second VM, the day page, `README.md` and `docs/HANDOFF.md` all have to say so.

Writing day scripts

Day scripts are **teaching material that happens to be executable**. Optimise for the reader, not for elegance.

- Comment the *why*, not the *what*. `# systemd has not read that yet` earns its line; `# reload systemd` does not.
- Say what a flag actually does the first time it appears.
- Every script that changes something must be **safe to run twice**, and should be the way back to a known-good state.
- Anything destructive gets a root check and a clear refusal, not a surprise.
- Print the command before running it when the point is for the reader to learn the command.
- Long output gets truncated with a note, so the terminal stays readable.

Two mistakes this repository has already made

Both were caught before release. Do not reintroduce them.

`((var++))` under `set -e`. Post-increment returns the *pre*-increment value, so the first `((count++))` returns `0`, which is a non-zero exit status, which kills the script. Use `count=$((count + 1))`. Every counter in this repository uses that form for exactly this reason.

Never pass an unchecked PID to `kill`. `systemd` reports `ExecMainPID` as `0` when a service is not running, and `kill -9 0` signals *every process in the caller's process group* — as root over SSH, that includes your own session. Check for `0` and empty before killing anything.

For reference, one shape that looks dangerous but is not: a false test in a non-final `&&` position, as in `[[ -n "$x" ]] && y=1`, does **not** trigger `set -e`. Verified, not assumed.

Never commit

The `.gitignore` covers these, but know why:

- VM disks and base images (`*.qcow2`, `*.img`, `*.iso`) — they belong under `$LAB_HOME`, never in git. A pre-commit hook rejects anything over 256 KB.
- Keys and certificates (`*.pem`, `*.key`, `*.crt`, `ca/`) — Day 10 generates a private CA. It stays on your lab.
- Packet captures (`*.pcap`) — Day 09 produces them and they contain whatever was on the wire.
- Ansible vault passwords (`.vault_pass`, `vault-password`).

If a secret does reach a commit, deleting the file in a later commit **does not remove it** — it stays reachable in history and every scanner will keep finding it. Say so immediately rather than quietly pushing a fix on top.

Commits

Conventional commits, with a body that explains the reasoning:

```
feat(day07): DNS resolution path scripts
fix(lab): guard against an empty ExecMainPID before kill
docs(handoff): record the Day 15 platform decision
chore(ci): pin the checkout action
```

One concern per commit. If the subject needs "and", it is two commits.

Adding or changing a day

Keep these in sync, or the repository starts lying:

1. `days/dayNN/README.md` and `days/dayNN/verify.sh`, together.
2. The CI matrix in `.github/workflows/ci.yml` if the day's tier changed.
3. The tier and memory tables in `README.md` and `docs/HANDOFF.md`.
4. The check counts in `docs/HANDOFF.md` §5.
5. `docs/HANDOFF.md` §10 lists every one of these pairings, and the **Last updated** date at its top.

Day dependencies are real and easy to break — Day 06 builds the topology that 07, 08, 09 and 18 use; Day 10's CA and Day 08's zone are consumed by Day 17; Days 11–13 feed the Ansible role in Day 15.

`docs/HANDOFF.md` §4 has the full list. Check it before reordering anything.

Reporting something that does not work

Most valuable report there is. Please include:

- your distribution and `uname -r`
- `./lab/lab.sh check` output
- the exact command and its full output
- for a failing check, `sudo ./days/dayNN/verify.sh` in full

"It failed" is hard to fix. "`virt-install` rejected `--os-variant rocky9` on Debian 12" is fixed the same day.

License

Contributions are accepted under the [MIT License](#), the same terms as the rest of the repository.

docs/HANDOFF.md

Complete state of this repository in one file. Written to be pasted into a fresh chat so an assistant can pick the work up cold, with no other context.

Last updated: 2026-09-15 **Repo:** `bash-mastery-linux` **Status:** scaffold complete, 20 days written, **Days 01–20 scripts written — the course is complete**; Days 12 through 17 and Day 19 verified on real Rocky 9 lab VMs, Day 18 verified on the owner's laptop (it needs no VM), Day 20 written and not yet run **Executed against real KVM hardware through Day 19**. See §8. **Licence:** MIT (`LICENSE`). Contribution rules: `CONTRIBUTING.md`.

0. How to use this file

If you are an assistant reading this in a new chat: this file plus the repository is the whole project. Read §2 before proposing anything, because several design choices are settled and re-litigating them wastes time.

If you change anything in this repo, update this handoff in the same pass. That is the point of the file. §10 says what to touch.

1. What this project is

Twenty days of Linux operations — host, network, security hardening, and configuration management — carried out on real machines the learner builds themselves.

It is deliberately **not** a software project. There is no application, no capstone deliverable, and nothing simulated. The learner gets three Rocky Linux 9 virtual machines plus a kernel-level network lab, and operates them until they behave.

Why there is no capstone

A capstone is a thing you build. Operations is a diagnostic discipline, not a constructive one — the exam for a developer is whether the thing works, the exam for an operator is whether you can find out why it does not.

So instead of a final project, the lab persists and accumulates. Day 06 builds a network and Days 07–10 and 18 operate inside it. Days 11–13 harden a host by hand and Day 15 turns that work into an Ansible role applied to `node2`, which starts clean. Day 20 restores data the earlier days created. Nothing is thrown away and nothing is rebuilt from a template.

If a capstone were ever forced onto this, the only sensible one would be "a hardened internal service platform: private CA, DNS zone, bastion, firewalled web tier, deployed by one Ansible role" — but it would collapse Days 10–17 into integration debugging and cost more than it teaches. Decided against.

Scope boundaries

In scope: systemd, users and permissions, processes and cgroups, storage and LVM, logs and time, interfaces and routing, DNS, packet capture, TLS and a private CA, firewalld and nftables, SSH hardening, SELinux, Ansible, WireGuard, reverse proxying, VLANs and bridges, intrusion detection, backup and restore.

Out of scope, deliberately: containers and Kubernetes, cloud providers, CI/CD pipelines as a subject, application development, mail servers, Samba, iSCSI, kernel compilation.

2. Settled design decisions

Do not reopen these without being asked.

DECISION	VALUE	WHY
Name	<code>bash-mastery-linux</code>	Owner's choice
Length	20 days, 4 phases of 5	Started at 15, extended to 20 on request
Capstone	none	See §1
Application	none	Ops curriculum, not a software one
Simulation / offline mode	forbidden	Everything must run for real
Guest distro	Rocky Linux 9	SELinux is RHEL-family; Ubuntu ships AppArmor
Hypervisor	KVM + libvirt	In-kernel on Linux; no VirtualBox or Multipass
Host	Linux, 8 GB RAM	Owner runs Ubuntu with only Docker/Podman installed
Networking days	<code>ip netns</code> on the host	Real kernel networking at zero RAM cost
SELinux	its own day (13)	Requested specifically
Verification	three tiers	See §5
Day scripts	shipped, written day by day	Days 01 through 20 done — every day now ships scripts
Blast radius	nothing the owner runs may put the laptop at risk	Every destructive step happens inside a VM or a namespace, both disposable
Ansible's job	configuration manager, nothing more	Days 14-15 only. It deploys a hardening baseline to VMs; it is not the subject of the course
Script style	commented as teaching material	The comments are half the lesson; these are not production scripts and should not be tightened into them

Two points that are easy to get wrong

Network namespaces are not a simulation. They are real interfaces, real routing tables, real packets and real `tcpdump` captures — the same kernel machinery containers are built from. This matters because it is what makes five of the twenty days cost no memory. Do not describe them as fake or as a stand-in for "proper" networking.

Nothing dangerous ever runs on the host. This is a hard requirement, not a preference. Every command that hardens, firewalls, relabels, partitions or locks something out runs inside a Rocky VM, which can be deleted and rebuilt in a minute with `lab.sh down` then `up`. The host only ever runs `lab.sh`, `virsh`, `ssh` and `scp`.

The namespace days (06-10, 18) are the one thing that touches the host, and they are safe by construction: `netns-up` creates named namespaces and veth pairs, edits no file anywhere, and `netns-down` removes all of it. A reboot also removes all of it. No host service, no `sysctl` outside the namespaces, no rule in the host's own firewall tables.

If even that is unwanted, the escape hatch already works and costs 1 GB: push the repo to the control VM and run the topology inside it — `sudo ~/lab/lab/lab.sh netns-up`. The script only needs `ip` and root, both of which the guest has. Same script, same checks, zero host involvement.

Containers cannot replace the VMs. Containers share the host kernel, so there is no separate systemd, no separate disks and no separate SELinux state. That rules out Days 01–05 and 13 entirely. The VMs are not a convenience.

3. The lab

One script owns the entire environment: `lab/lab.sh` (586 lines, 31 functions, 12 subcommands).

```
./lab/lab.sh check           # prerequisites, with per-distro install hints
./lab/lab.sh image          # download the Rocky 9 base image (~1 GB, once)
./lab/lab.sh up [vm...]     # create VMs (default: control node1)
./lab/lab.sh status         # VMs, their IPs, and namespaces
./lab/lab.sh ssh <vm>       # log in
./lab/lab.sh push <vm> [path] # copy days/ and lab/ into ~/lab on a VM
./lab/lab.sh add-disk <vm> [GB] # attach a blank disk (Day 04 needs this)
./lab/lab.sh down [vm...]   # delete VMs and their disks
sudo ./lab/lab.sh netns-up   # build the Days 06-10 and 18 network
sudo ./lab/lab.sh netns-status # show it and ping-test it
sudo ./lab/lab.sh netns-down # tear it down
./lab/lab.sh destroy        # everything
```

`push` is how the repository reaches a VM. It `scp`s the given paths (default `days` and `lab`) into `~/lab` on the guest and re-applies the execute bits, because some `scp` builds drop them. Push both directories, not just `days`: every `verify.sh` sources `../../lab/verify-lib.sh`, so a day pushed on its own cannot check itself. There is deliberately no shared folder and no guest agent — one copy command is easier to reason about than a mount that silently stops syncing.

Configuration (all overridable by environment variable)

VARIABLE	DEFAULT
LAB_HOME	<code>\$HOME/.local/share/bash-mastery-linux</code>
LAB_BASE_URL	<code>https://dl.rockylinux.org/pub/rocky/9/images/x86_64/Rocky-9-GenericCloud-Base.latest.x86_64.qcow2</code>
LAB_USER	<code>lab</code>
LAB_NET	<code>default</code> (the libvirt NAT network)
SSH_KEY	<code>\$HOME/.ssh/id_ed25519</code> (generated if absent)
LAB_DISK_SIZE	<code>10G</code>

Derived paths: `$LAB_HOME/images`, `$LAB_HOME/disks`, `$LAB_HOME/seed`. Nothing is written inside the repository.

The virtual machines

VM	RAM	ROLE
<code>control</code>	1536 MB	Where the learner sits. Ansible runs from here
<code>node1</code>	2048 MB	The machine that gets configured and broken
<code>node2</code>	2048 MB	Starts clean. Only Day 15 needs it

Disks are thin qcow2 overlays on one shared base image (`qemu-img create -f qcow2 -F qcow2 -b <base>`), so three VMs cost barely more than one until packages are installed. Budget ~12 GB of disk.

VMs are created with `virt-install --import --cloud-init , --graphics none , --noautoconsole , virtio disk` and network. The `--os-variant` value is probed via `osinfo-query` with a fallback chain `rocky9 → rhel9.0 → rhel9-unknown → generic` , because older `osinfo-db` packages do not know Rocky 9.

cloud-init sets the hostname, creates `$LAB_USER` with passwordless sudo, and installs the public half of `$SSH_KEY` . **SELinux is deliberately left enforcing** — Day 13 depends on it.

The namespace network (Days 06–10, 18)

```
client 10.10.0.2 ---- 10.10.0.1 [router] 10.10.1.1 ---- 10.10.1.2 resolver
                        10.10.2.1 ---- 10.10.2.2 auth
```

Four namespaces: `client` , `router` , `resolver` , `auth` . Three veth pairs (`veth-cl/veth-rc1` , `veth-rs/veth-rrs` , `veth-au/veth-rau`). The router has `net.ipv4.ip_forward=1` and every namespace gets a default route through it. `netns-up` finishes by pinging `client → auth` to prove the topology works.

Costs 0 MB and needs only `iproute2` and root. This is the recommended entry point for anyone who does not want to install a hypervisor yet.

4. The twenty days

Four phases of five. Each phase ends somewhere the learner could stop and still have gained something whole.

Phase 1 — The host (01–05). One machine, understood properly. **Phase 2 — The network (06–10)**. Real kernel networking, no memory cost. **Phase 3 — Hardening and configuration management (11–15)**. Lock a host down by hand, then make it repeatable. **Phase 4 — Production operations (16–20)**. What turns a configured host into one you can rely on.

DAY	TITLE	RUNS ON	VERIFIED BY
01	systemd and the boot path	VM: control	lint + lab
02	Users, sudo, permissions and ACLs	VM: node1	lint + lab
03	Processes, signals, cgroups v2 and limits	VM: node1	lint + lab
04	Storage: LVM, filesystems and mount units	VM: node1 + extra disk	lint + CI
05	Logs and time: journald, logrotate and chrony	VM: node1	lint + lab
06	Interfaces, routing and building the namespace lab	Host: namespaces	lint + CI
07	The DNS resolution path	Host: namespaces	lint + CI
08	Running DNS: authoritative and recursive	Host: namespaces	lint + CI
09	Packet-level debugging	Host: namespaces	lint + CI
10	TLS on the wire and a private CA	Host: namespaces	lint + CI
11	firewalld, and the nftables underneath it	VM: node1	lint + lab
12	SSH hardening, bastions and fail2ban	VM: control + node1	lint + lab
13	SELinux: contexts, booleans and denial triage	VM: node1	lint + lab
14	Ansible fundamentals	control → node1	lint + lab
15	Ansible roles: your hardening baseline	control → node1 + node2	lint + lab
16	WireGuard: a private network between hosts	VM: control + node1	lint + lab
17	Reverse proxy and TLS termination	VM: node1	lint + lab
18	Bridges, VLANs and link aggregation	Host: namespaces	lint + CI
19	Intrusion detection and audit alerting	VM: node1	lint + lab
20	Backup, restore and the restore drill	VM: control + node1	lint + lab

Dependencies between days

Days are otherwise self-contained, but these links are intentional and must not be broken when editing:

- **06 → 07, 08, 09.** Day 06 builds the namespace topology the rest use. Day 18 is the exception: it builds its own namespaces (`sw18` , `h18a` ...) so both topologies can be up at once and Days 07-09 keep working.
- **10 → 17.** Day 17 serves TLS using the private CA issued on Day 10.
- **08 → 17.** Day 17 needs `www.lab.test` to resolve. Day 08's zone serves it properly; `setup.sh` writes `127.0.0.1 www.lab.test` to `/etc/hosts` so the day works without it, and says so in the README.
- **11, 12, 13 → 15.** Day 15 turns that manual hardening into an Ansible role.
- **04 → 20.** Day 20 backs up and restores `/srv/data` from Day 04.
- **14 → 15.** Roles build on the playbook basics.

Memory budget

DAYS	NEEDS	RAM
01	one VM (<code>control</code>)	~1 GB
02–05, 11–13, 17, 19	one VM (<code>node1</code>)	~2 GB
06–10, 18	no VM at all	0 MB
14, 16, 20	two VMs	~3.5 GB
15	three VMs	~5.5 GB

Peak is Day 15 only. Designed against an 8 GB laptop.

5. The verification model

This is the part most likely to be misunderstood, so it is spelled out.

Verification is split into three tiers because pretending CI can prove everything produces a badge that proves nothing.

TIER	WHAT IT CHECKS	WHERE IT RUNS	DAYS
lint	<code>bash -n</code> , shellcheck	GitHub Actions	all 20
CI	the day executed for real	GitHub Actions	04, 06, 07, 08, 09, 10, 18
lab	the day executed for real	the learner's VMs	the other 13

GitHub runners are full Ubuntu VMs with `sudo` , so namespaces, DNS servers, `tcpdump` , `openssl` , VLANs and **LVM on a loopback file** are all genuinely real there. That is 7 of 20 days machine-verified in CI.

The other 13 cannot be faked on a runner: Ubuntu has no SELinux, no `firewalld`, and no second host to reach over SSH. Those days are still *automated* — just by the learner, on their own lab.

`days/dayNN/verify.sh`

Every day has one. **102 automatic checks and 31 judgement items** across the twenty days. Each script sources `lab/verify-lib.sh` , declares its checks, and ends with `v1_summary` . Exit status is 0 only when nothing failed.

Four outcomes:

	MEANING
PASS	the command succeeded
FAIL	the command failed — the day is not done
SKIP	a prerequisite is missing, so the check could not run
YOU	a judgement call; printed as a reminder, never affects exit status

The `SKIP` path matters: running a lab-tier day on the wrong machine reports what is missing rather than producing false failures. Verified behaviour on an Ubuntu host with no SELinux:

```

Day 13 – SELinux: contexts, booleans and denial triage
SKIP  SELinux is enforcing
SKIP  the web root carries a web content label
YOU   you fixed a denial by relabelling, not by disabling SELinux

could not run: missing getenforce semanage
This day runs elsewhere - see the "Runs on" line in its README
7 checks skipped, 2 for you to judge.
exit=0

```

lab/verify-lib.sh API (100 lines)

```

v1_init "Day 01 - systemd and the boot path" # header
v1_need systemctl semanage                  # missing -> later checks SKIP
v1_need_root                                # not root -> later checks SKIP
v1_check "description" 'shell command string' # PASS / FAIL / SKIP
v1_manual "description"                      # YOU, never fails
v1_summary                                   # totals; exit 1 if any FAIL

```

Convention: `v1_check` command strings are written inside single quotes, so use double quotes within the command itself. Keep to it and the checks stay readable and quote-safe.

Checks test the **work product**, not knowledge. Examples: `semanage fcontext -l` contains `/srv/www` (permanent, not a transient `chcon`); `restorecon -nvr` produces no output; a second `ansible-playbook` run reports `changed=0`; a `restic restore diff -r s` clean against the source.

6. CI

`.github/workflows/ci.yml` (79 lines), three jobs:

JOB	WHAT IT DOES
<code>lint</code>	<code>bash -n</code> on every <code>*.sh</code> , then <code>shellcheck -x -S warning</code>
<code>verifiable-days</code>	matrix over days 04, 06, 07, 08, 09, 10, 18; runs <code>sudo bash lab/ci-day.sh <NN></code>
<code>coverage</code>	<code>if: always()</code> ; prints exactly which days were not proven and why

The `coverage` job exists so a green run never implies more than it did. It names all 13 lab-only days and the reason each is unprovable on a runner.

lab/ci-day.sh (57 lines)

Builds an environment for one day, then runs its `verify.sh`:

1. If `days/dayNN/scripts/setup.sh` exists, run it.
2. Else, if the day is 06, 07, 08, 09 or 18, run `lab/lab.sh netns-up`.
3. Else, print `SKIPPED` (not yet implemented) and **exit 0**.

`verify.sh` **only runs when the day's work exists**. A `setup.sh` means it exists. Day 06 is the single exception: the namespace topology *is* its work, so `netns-up` alone is enough. For days 07, 08, 09 and 18 the topology is only the floor - the zone, the resolver and the bridge are the day's work, and running `verify.sh` before those scripts exist fails every time. That is a missing script, not a broken repo, so those jobs skip and exit 0.

Step 3 is why CI is green today: no day scripts exist yet, so there is nothing to stand up. Each CI day becomes genuinely verified the moment its `setup.sh` is written. This is honest rather than convenient — the alternative was a failing pipeline that trains people to ignore it.

Permissions are pinned to `contents: read`. Actions used: `actions/checkout@v4`.

7. How a day is put together

Every file in this repository is hand-written and hand-edited. There is no generator and no build step.

A day is three files that must agree with each other:

FILE	WHAT IT HOLDS
days/dayNN/README.md	objective, why it matters, the work, the checks, what CI proves
days/dayNN/verify.sh	those same checks, as assertions, using the <code>v1_*</code> API in §5
days/dayNN/scripts/*	the scripts the day walks the reader through

The one rule: a check listed on the page must exist in `verify.sh`, and the other way round. A page claiming a check its verifier does not run is the single failure this repository cannot tolerate, because every claim in it is supposed to be true. Add or remove a check in both places, in the same commit.

Day pages follow a fixed order so they read the same way: title and metadata line, objective, why it matters, the work, then - once the day has scripts - a **Scripts for today** table and a **Run it on the lab** runbook, then the check list. Copy the shape from `days/day01/README.md`, which is the only complete example so far.

The metadata line carries the day's tier and memory cost. If either changes, §4 and §5 here, the tier table in `README.md` and the CI matrix in `ci.yml` all have to change with it. §10 lists every one of those pairings.

8. Verified state, and what is not verified

What has actually been run

CHECK	RESULT
tests/cli.sh	397 passed, 0 failed (was 382; Day 20's five scripts added checks)
bash -n on all 90 shell scripts	0 failures
lab/lab.sh --help	stops cleanly at the memory budget
lab/lab.sh check	runs every section, prints the full summary
lab/lab.sh bogus	FAIL unknown subcommand , exit 1
lab/lab.sh netns-down as non-root	FAIL this subcommand needs root , exit 1
days/day13/verify.sh on Ubuntu	7 SKIP, 2 YOU, exit 0
lab/ci-day.sh 04	no longer skips: days/day04/scripts/setup.sh now exists, so CI executes it and then verify.sh . Never yet run on a real runner
lab/ci-day.sh 99 / no argument	exit 2 with usage
First real CI run (2026-09-04)	days 04, 06, 10 green; 07, 08, 09, 18 now skip; lint fixed
ci.yml structure and tabs	parses, no tabs
lab.sh push with no argument	FAIL usage: ... push <vm> [path...] , exit 1
lab.sh push bogus	FAIL unknown vm , exit 1
lab.sh --help after the patch	lists 14 subcommands including console and diagnose
set -e behaviour of [[cond]] && x=y	confirmed safe: a false test in a non-final && position does not exit

What has never been run

lab.sh has never touched real KVM hardware. The sandbox it was written in has no /dev/kvm , no libvirt, no virsh , no ip , and no network. Unverified:

- VM creation (virt-install), including the --os-variant fallback chain
- cloud-init user creation and SSH key injection
- base image download
- namespace and veth wiring (netns-up / netns-status / netns-down)
- disk attach (add-disk)
- every one of the 102 automatic checks, in its passing state
- lab.sh push against a live guest (scp , the remote chmod , the SSH options)
- **all five Day 01 scripts.** They are bash -n clean and were read line by line, but nothing in the sandbox has systemd as PID 1, so systemctl , journalctl , systemd-analyze and systemd-cgls have never executed. The likeliest failure is cosmetic (a property name, or systemd-cgls output shape), not structural

./lab/lab.sh check is the most tested part and the correct first command. If virt-install rejects --os-variant rocky9 , the host osinfo-db predates Rocky 9 — use rhe19.0 ; the fallback chain attempts this

already, and `lab/README.md` carries the manual `virt-install` commands.

First real host run (2026-09-06)

`./lab/lab.sh check` behaved correctly on Ubuntu with no virtualization stack installed: every detection section was right and the install hint it printed was the right one. Two faults surfaced from the owner pasting all four step-1 commands at once, and both are fixed:

- `cmd_image` and `cmd_up` ran despite `check` having failed, so a 1 GB download completed and then died on `qemu-img: command not found`. Both now call `require_lab_tools` first, which dies with the install hint.
- A failed qcow2 validation left the bad download in place, so a retry would have trusted it. `cmd_image` now deletes it and says to re-run.

`days/day01/README.md` step 1 was also rewritten to run the four commands one at a time, with `check` stated as a gate and the apt install spelled out.

The apt hint itself was wrong, and is fixed. It named `qemu-kvm`, which Debian and Ubuntu now ship only as a virtual package with no installation candidate. apt therefore aborts and installs *nothing*, so the following `systemctl enable --now libvirtd` fails with "Unit libvirtd.service does not exist" and looks like a second, unrelated fault. It is not. The correct package is `qemu-system-x86`; plain `qemu-system`, which the QEMU website suggests, installs every CPU architecture and is wrong for a KVM lab.

`pkg_hint()` now emits `qemu-system-x86 libvirt-daemon-system libvirt-clients libvirt-daemon-config-network virtinst`, and a new `enable_hint()` prints `libvirtd` or the modular `virtqemud.socket` `virtnetworkd.socket` depending on what the host actually has, since recent libvirt ships no `libvirtd.service`. `enable_hint()` is called from all three places that used to hardcode the unit name. `README.md`, `lab/README.md` and `days/day01/README.md` were corrected to match.

The libvirt URI trap (fixed 2026-09-06)

`check` reported `network 'default' is defined but inactive` while `sudo virsh net-list --all` on the same machine showed it `active`. The Active-field parser was verified correct against realistic `net-info` output using a fake `virsh`, so the disagreement was not parsing. Two real causes, both now handled:

1. **No libvirt URI was pinned.** Root gets `qemu:///system`; an unprivileged user can silently fall back to `qemu:///session`, which holds none of this lab's VMs or networks. `lab.sh` now exports `LIBVIRT_DEFAULT_URI="${LIBVIRT_DEFAULT_URI:-qemu:///system}"`, so the script, the user's shell and `sudo` all read the same daemon. This was a latent fault that would have surfaced again on `up`, `status` and `ssh`.
2. **A first-call race.** The first `virsh` call after boot can socket-activate `virtnetworkd`, which then autostarts the network - so the network genuinely reads inactive during that call and is active a second later, which is exactly why the owner's follow-up `net-start` answered "already active". `check` now re-reads the state after a 2 second pause before failing, and the failure hint says to re-run `check` if `net-start` reports that.

Also added: a **libvirt group** check (missing membership is why libvirt commands need `sudo` until the user logs out and back in), a `net-define` hint for the genuinely-missing case, and `net_state()`, which reads the `Active` field by name via `awk` instead of pattern-matching the whole block.

`tests/cli.sh` caught a side effect of that change immediately: its "every subcommand is documented" check scraped every `case` branch in `lab.sh`, so the new `active`) and `inactive`) branches were demanded of the help text. It now reads the `main()` dispatcher only. **108 checks.**

qemu cannot read your home directory (fixed 2026-09-07)

`up control` failed with `Cannot access storage file '/home/eric/.local/share/bash-mastery-linux/disks/control.qcow2' (as uid:64055, gid:991): Permission denied`, after `virt-install` had already warned that `libvirt-qemu` needed search permission on `/home/eric`, `/home/eric/.local` and `/home/eric/.local/share`.

This is structural, not a mistake by the operator. Under `qemu:///system` the VM process does not run as you: it runs as `libvirt-qemu` on Debian and Ubuntu, and as `qemu` on Rocky. A home directory is mode 0700, so that user cannot traverse into it, and `LAB_HOME` defaults to `~/.local/share`. The lab could never have started a VM on a default Ubuntu install.

Rejected fixes: `chmod 755 ~` (exposes the whole home directory to every local account), and running `qemu` as root via `/etc/libvirt/qemu.conf` (changes the host's security posture for a teaching lab). Chosen fix: **POSIX ACLs granting exactly one user exactly what it needs**, which the owner of the directories can set without `sudo`.

New functions in `lab/lab.sh`:

- `qemu_user()` - resolves `libvirt-qemu`, else `qemu`, else fails.
- `grant_path <perms> <path>...` - One `setfacl -m u:<qemu-user>:<perms>` per existing path; a no-op when `setfacl` or the user is absent.
- `grant_hypervisor_access()` - walks up from `LAB_HOME` to `$HOME` granting **search only** (`x`), then `rx` on `images/` and `seed/`, `r` on the base image, `rw` on `disks/`.

Called from `cmd_image` (both the fresh-download and cached paths) and from `create_vm` immediately after `qemu-img create`, which also grants `rw` on the new overlay - each overlay is created 0600 and needs its own entry. `cmd_check` now reports this too: it looks for the `x` entry on `$HOME` via `getfacl` and warns if it is missing, and warns separately if the `acl` package is not installed. `acl` was added to the apt, dnf and pacman package hints and to the three documented install lines.

Proved with a fake `setfacl` on `PATH` and a fake `$HOME` tree: eight calls, search-only on the four path components, and read/write confined to `disks/`.

Also fixed in the same run: `virt-install` reported `Using --osinfo generic, VM performance may suffer`, because `osvariant()` only tried `rocky9`, `rhel9.0` and `rhel9-unknown`. It now falls back through `rocky9`, `rocky9.0`, `rhel9.6` down to `rhel9.0`, then `rhel9-unknown`, `centos-stream9` and `linux2022`, and queries `virt-install --osinfo list` when `osinfo-query` is missing. A close RHEL 9 profile still yields virtio and a correct clock; `generic` does not.

AppArmor, not permissions: the lab moved out of \$HOME (2026-09-07)

After the ACL fix above, `up control` still failed: `Could not open '/home/eric/.local/share/bash-mastery-linux/images/Rocky-9-GenericCloud-Base.latest.x86_64.qcow2': Permission denied`. Two signals proved the ACLs had in fact worked: libvirt's own accessibility warning about `/home/eric`, `/home/eric/.local` and `/home/eric/.local/share` no longer appeared, and `--osinfo` resolved to `rocky9`.

So the remaining denial is not discretionary access control. On Ubuntu, libvirt confines each `qemu` process with an AppArmor profile whose allowed paths do not include home directories. Correct file permissions cannot help; the kernel refuses the open regardless, and reports it as `EACCES`, which reads exactly like a permission problem.

Rejected fixes: putting the AppArmor profile in complain mode (disables a host security control for a teaching lab) and editing `/etc/apparmor.d/abstractions/libvirt-qemu` (a host-wide change that a learner would have to remember to undo). Chosen fix: **keep the lab where libvirt and AppArmor already expect image files.**

- `LAB_LIBVIRT_HOME=/var/lib/libvirt/images/bash-mastery-linux`.
- `lab_home_default()` returns that path when it exists and is writable by the invoking user, otherwise the old `~/.local/share` path. `LAB_HOME` still overrides everything, so nothing that set it breaks.
- `relocate_hint()` prints the single command that creates it - `sudo install -d -o <user> -g <group> ...` - plus an `mv` of the existing `images/` directory when one is present, so the ~900 MB base image is not downloaded twice.
- `cmd_check` now reports `lab directory is outside your home directory`, or warns and prints the hint. `create_vm` prints the same warning before it builds an overlay that cannot possibly boot.

The ACL machinery from the previous fix is kept: it is still what makes a home-directory lab work on distributions without AppArmor, and it is harmless where it is not needed.

Verified in the sandbox by pointing `LAB_LIBVIRT_HOME` at a writable temporary directory (chose it) and at a missing one (fell back to `$HOME`), and by inspecting `relocate_hint` output.

The VM that creates itself and then powers off (2026-09-07)

With the storage problems gone, `up control` reported `control` defined and booting, then `status` showed `control shut off`. Nothing was broken on the host; the sequence is how `virt-install` works, and `lab.sh` did not account for it.

`virt-install --import --cloud-init` treats the first boot as an *install phase*. During that phase libvirt sets `on_reboot=destroy` on the domain so the generated cloud-init ISO can be detached cleanly, and the Rocky image does reboot once cloud-init has applied the seed. Because we pass `--noautoconsole`, `virt-install` has already returned - it even prints "Domain is still running. Installation may be in progress" - so when that reboot lands, the domain is destroyed and left `shut off`. The final XML has no ISO attached and boots normally, so the only missing step was starting it.

- New `ensure_running()` reads `virsh domstate` and starts a `shut off` domain, resumes a `paused` one, does nothing when it is already `running`, and warns for anything else. It never fails, so it is safe under `set -e`.
- `wait_for_ip()` now sleeps 10s to let cloud-init run, calls `ensure_running`, then polls for up to 180s (was 120s) and re-checks the domain state every 16 seconds, because the power-off can land at any point in that first boot.
- `cmd_status` prints `start it with: lab.sh up <vm>` under a `shut off` VM. `up` on an existing VM skips creation but still waits, so re-running it is now the single recovery command for this and most other boot problems.

Tested with a fake `virsh` across all of `running`, `shut off`, `paused` and `pmsuspended`, including a `virsh start` that exits non-zero: the failure warns and still returns 0.

Running but no address, and a status line that misled (2026-09-07)

`up control` started the powered-off domain correctly, but no DHCP lease ever appeared: `status` showed `control running -`, and `ssh` failed with `no address for control`. Two separate shortcomings in `lab.sh` came out of this, both about the script telling the operator too little:

1. **There was no way to look at the VM.** A running VM with no lease can only be explained from its console - kernel messages, cloud-init output, or a login prompt meaning the guest is fine and the problem is DHCP or the lease lookup. New 13th subcommand `console <vm>` (exec `virsh console`, with a reminder that Enter gives a prompt and `Ctrl+] exits`). `status` now prints `running but no DHCP lease yet: lab.sh console <vm>` in exactly that case, and both `ssh` and `push` print the console command plus `virsh net-dhcp-leases` instead of a bare failure.
2. **status read like an instruction.** Its namespace line said `none - run: sudo lab.sh netns-up`, which on Day 01 looks like a required step; the operator asked whether they had to run it. Namespaces are only for Days 6-10. The line now reads `none - only Days 6-10 need these (sudo lab.sh netns-up)`.

Lesson recorded for the remaining days: any status line that names a command is read as an order. If a command is optional or belongs to another day, the line has to say so on the same line.

Watching the boot from the first byte (2026-09-07)

The first host run reached a state that none of the previous fixes explain: `domstate` says `running`, `domiflist` shows a virtio NIC on the `default` network, the domain XML has a correct virtio disk under `/var/lib/libvirt/images/bash-mastery-linux/disks/` and a proper `isa-serial` console on `/dev/pts/1` - and yet `virsh console` prints nothing and the DHCP lease table is empty.

An empty console proves nothing on its own, which was the real gap: attaching to a VM that has already finished booting shows no output, because the boot messages scrolled past before the connection existed and an idle guest writes nothing new. "Booted fine but no DHCP" and "never executed a kernel" look identical from there.

- `console <vm> --restart` power-cycles the domain and attaches with `virsh start --console`, so firmware, boot loader and kernel output are all captured. The plain form now suggests it when nothing appears.
- **Backing-chain validation in `create_vm`.** A thin overlay whose chain to the base image is broken - for example because the base image was moved after the overlay was created, which is exactly what the relocation fix asked the operator to do - produces a guest that boots to nothing with no host-side error at all. `qemu-img info --backing-chain` is now checked immediately after `qemu-img create`; on failure the overlay is deleted and the operator is told to re-run `image` then `up`.

The empty `<disk device='cdrom'>` entry with no `<source>` seen in the XML is the detached cloud-init ISO from the install phase. It is expected and harmless.

The seed ISO we build ourselves (2026-09-07)

The install-phase problem was patched twice (`ensure_running`, then the console tooling) before the mechanism itself was replaced. `virt-install --cloud-init` is convenient and fragile: it runs the first boot as an install phase with `on_reboot=destroy` so it can detach the seed ISO cleanly, and because `--noautoconsole` makes `virt-install` return early, the guest's post-cloud-init reboot leaves the domain powered off at the worst possible moment - mid-way through growing the filesystem, creating the `lab` user and installing the SSH key.

`create_vm` now builds the NoCloud seed ISO itself and attaches it as a permanent read-only cdrom, with `--boot hd` and no `--cloud-init` flag at all. There is no install phase, so the very first boot is an ordinary boot.

- `seed_iso()` writes `user-data` (from `write_seed`) and a `meta-data` file containing `instance-id` and `local-hostname` into `seed/<vm>/`, then builds the image with `cloud-localds`, Or `genisoimage`, Or `xorriso` - whichever exists. **The filesystem label must be exactly `CIDATA`** or cloud-init will not look at the disk.
- The ISO is granted `r` through `grant_path`, like the base image.

- With no builder present the old `--cloud-init` path still runs, but it warns and prints `iso_pkg_hint()` (`cloud-image-utils` on apt, `cloud-utils` on dnf). `cloud-image-utils` was added to all three install lines.
- `cloud-init` re-reads the seed on every boot now that the ISO stays attached. That is harmless: its modules are idempotent.

Both branches were proved with fake `virt-install / cloud-localds` binaries: the ISO branch emits `device=cdrom,readonly=on` and `--boot hd` and **zero** `--cloud-init` flags; the fallback branch warns, prints the hint, and still passes `user-data=...,disable=on`.

A lock error is not a corruption error. `qemu-img info` on a disk belonging to a running domain fails with `Failed to get shared "write" lock`, which says nothing about the image. Use `--force-share`, or check it while the VM is off.

One command that collects every clue (2026-09-07)

The stuck-VM investigation cost four round trips because each host-side command answers only a fraction of the question, and the most informative file was never asked for. `console <vm> --restart` finally produced the decisive observation: **no output at all, not even firmware**, which rules out the guest OS entirely and points at `qemu` never getting the guest off the ground.

`diagnose <vm>` (14th subcommand) collects the whole picture in one run:

MEASUREMENT	WHAT IT DISTINGUISHES
<code>domstate</code> , <code>dominfo</code>	defined vs running vs paused
<code>cpu-stats --total</code> , read twice three seconds apart	whether the vCPU is executing instructions at all
<code>domblkstat</code> <code>rd_req / rd_bytes</code>	whether the guest read a single block; a guest that reached its boot loader has read thousands
<code>qemu-img info --backing-chain --force-share</code>	whether the overlay still resolves to the base image
seed ISO presence	whether the VM predates the seed-ISO path
<code>/var/log/libvirt/qemu/<vm>.log</code>	the file that names the reason a domain will not run. Needs root; the command is printed when it cannot be read
<code>dmesg</code> AppArmor/OOM lines, <code>free -m</code> , lease table	host-side refusals and memory pressure

The device name comes from `domblklist` rather than being hardcoded to `vda`, and `--force-share` is mandatory: a running domain holds a write lock and `qemu-img` otherwise fails with an error that reads like corruption but is not.

Lesson for every future stuck-guest problem: read the per-domain qemu log first. Serial silence is not evidence, because attaching to an already-booted VM shows nothing either way.

Alive, spinning, and silent (2026-09-07)

The first `diagnose` run on the host overturned the previous conclusion. The guest is not failing to start - it is running hard and saying nothing:

MEASUREMENT	VALUE	MEANING
<code>cpu_time</code>	650.2s, +3.3s over 3s	one vCPU pegged at ~100%, not idle, not stopped
<code>rd_req / rd_bytes</code>	82,236 / 1.44 GB	the guest read a gigabyte and a half; it is well past firmware
backing chain	resolves to the base image	the relocation did not break anything
seed ISO	374 KB, <code>libvirt-qemu:kvm</code> , attached as <code>ide-cd</code>	the new seed path works
qemu log	ordinary command line, <code>char device redirected to /dev/pts/1</code> , no error	qemu itself is not complaining
free memory	2175 MB	no OOM kill

So "qemu never gets the guest executing" was wrong. A guest burning a full core while reading gigabytes and never reaching NetworkManager - hence the empty lease table - has two plausible explanations, and `diagnose` was missing the measurement that separates them:

1. **Software emulation instead of KVM.** TCG pegs one host core, makes boot glacial, and produces exactly this pattern. `info kvm` via the qemu monitor settles it in one line; the tail of the qemu log never shows the `-accel` argument, which is why four round trips missed it.
2. **A guest genuinely stuck in early boot**, spinning on I/O.

Added, therefore:

- `diagnose` NOW RUNS `qemu-monitor-command --hmp 'info kvm'` and `'info status'`, and `domifaddr --source arp` - because the host ARP table sees a guest that configured an address without asking libvirt's dnsmasq, which a lease table never shows.
- `LAB_GRAPHICS` (default `none`). `LAB_GRAPHICS=vnc ./lab/lab.sh up control` gives the guest a screen, bound to `127.0.0.1` only because these guests have no console password. Serial-only is right for a scripted lab, but a guest that boots and never writes to `ttys0` is invisible without it, and there was no way to look at the screen at all. `diagnose` prints the `vncdisplay` when one exists and the `recreate` command when it does not.

Lesson: serial silence is not evidence of a dead guest, and CPU time plus block-read counters are the cheapest proof of life there is.

The missing display device (2026-09-07, resolved)

`LAB_GRAPHICS=vnc ./lab/lab.sh up control` booted in under a minute and reported `control is 192.168.122.122`. The same image, the same seed ISO, the same disk chain and `kvm support: enabled` had previously spun for nineteen minutes with `--graphics none`. **The one variable was the presence of a display device.**

The likely mechanism is the boot loader: RHEL-family cloud images configure GRUB for a graphical terminal, and with no video device at all it can hang before handing over to the kernel - which fits every observation, including the pegged vCPU, the gigabyte of disk reads, and the complete absence of serial output. This is stated as the probable cause rather than a proven one: the decisive experiment (booting headless with a serial-forced kernel command line) was never run, because the working configuration was preferable to a proof.

`LAB_GRAPHICS` now defaults to `vnc`, bound to `127.0.0.1`. The reasoning is not subtle: a guest whose console cannot be looked at is not debuggable, and this project asks a learner to break things on purpose. `LAB_GRAPHICS=none` restores headless behaviour.

The second half of the user's question - "why is it not in the README" - was the fairer one. It was not in the README because it had been invented twenty minutes earlier as a debugging flag, which is exactly the failure mode described in maintenance rule 4: a workaround that lives only in a chat transcript. The working configuration is now the default, and `virsh vncdisplay <vm>` plus `diagnose` are documented in `README.md`, `lab/README.md` and `days/day01/README.md`.

Rule: when a flag turns out to be the difference between working and not working, it stops being a flag. It becomes the default, in the documentation, in the same change.

Day 01 was run on the laptop (2026-09-07)

Day 01 passed - 4 PASS, 1 YOU, exit 0, `\Restart=` proven by killing the process and watching it come back. It was run on the host, not on `\control\`. The screenshots show `\eric@eric-X556UQ .../days/day01$ sudo ./scripts/setup.sh\`, and `\lab-demo.service\` was installed and enabled on the user's own Ubuntu machine. On `\control\`, `\systemctl is-enabled lab-demo\` still answered `\No such file or directory\`.

Nothing was damaged - `\lab-demo\` is a harmless `\sleep\` loop as `\User=nobody\` - but Day 11 rewrites firewall rules, Day 12 hardens sshd and Day 13 relabels filesystems. The same mistake on any of those days ends the project and violates the one hard requirement of this repo: never risk the host OS.

The cause was documentation, not the user. `\### 5. Prove it survives a reboot\` said `\sudo reboot\` without saying which machine, and every step before it read as something you type wherever you happen to be. The give-away was `\Operation inhibited by "eric" ... user session inhibited\` - systemd refusing to reboot a desktop with a graphical session, which no VM would ever say.

Fixed structurally rather than with a warning paragraph:

- `\lab/on-lab-vm.sh\`, a guard library exporting `\require_lab_vm\`. A machine qualifies if `\etc/bash-mastery-linux-lab\` exists or its short hostname is `\control\`, `\node1\` or `\node2\`. Otherwise it prints the three commands that get you onto a VM and exits 1 before anything is touched. `\hostname(1)\` is not installed everywhere, so it falls back to `\etc/hostname\` and then to `\unknown\` - which refuses, because refusing is the safe default. `\LAB_ALLOW_THIS_MACHINE=1\` overrides it and says so on stderr.
- **cloud-init writes the marker** via `\write_files\` in `\write_seed\`, so a lab VM identifies itself even if it is renamed.
- `\setup.sh\` and `\break-and-fix.sh\` call it. `\teardown.sh\` deliberately does not: it removes state, and it must stay usable on a machine where `\setup.sh\` should never have run in the first place. `\explore-boot.sh\` is read-only and needs no guard.
- Every state-changing script in Days 02-20 must source it. This is now maintenance rule 8 in section 10.

Rule: an instruction that does not name the machine it runs on will be run on the wrong machine. Guard the script; do not warn in prose.

Day 02 was written (2026-09-07), and has not been run

Five scripts, following Day 01's shape exactly: a payload, an idempotent `setup.sh`, a read-only tour, a `break-and-fix.sh` with a `--hard` mode, and a `teardown.sh`. `bash -n` clean, `tests/cli.sh` green at 127. **Nothing**

has been executed: the authoring sandbox has no systemd, no `setfacl`, no `visudo` and no Rocky guest, so every claim below is design intent until `node1` says otherwise.

What the day builds, and why each piece is there rather than being asserted in prose:

OBJECT	CHOICE	REASON
group <code>appdata</code>	owns <code>/srv/shared</code> , mode <code>2770</code>	gives the <code>setgid</code> check something real to inherit from
user <code>appsvc</code>	<code>--system</code> , <code>/sbin/nologin</code> , own primary group , <i>not</i> in <code>appdata</code>	if <code>appsvc</code> were an <code>appdata</code> member the ACL would be decoration. Excluding it makes the ACL load-bearing, which is what the day is teaching
<code>/srv/shared</code>	<code>setfacl -m u:appsvc:rwX</code> plus a default ACL	the <code>-m</code> entry and the <code>-d</code> entry are separate; setting one does not set the other, and that gap is a real production bug
<code>/etc/sudoers.d/appsvc</code>	<code>Cmd_Alias</code> , absolute paths, <code>(root)</code> not <code>(ALL)</code> , <code>0440</code>	<code>(root)</code> is what keeps <code>verify.sh</code> 's "cannot become root" check honest, since it greps for <code>(ALL).*ALL</code>
<code>lab-app.service</code>	<code>User=appsvc</code> , <code>ProtectSystem=full</code> + <code>ReadWritePaths=/srv/shared</code>	Day 01 used <code>ProtectSystem=strict</code> , which would block this service's only job. The pairing is the lesson

The service **writes** rather than just sleeping, which is the one structural difference from Day 01's payload. That is deliberate: a permission mistake has to produce a dead service and a `Permission denied` in the journal, otherwise the ACL demonstration proves nothing.

Two deviations from Day 01, both intentional:

- `teardown.sh` **calls** `require_lab_vm`. Day 01's teardown deliberately does not, because it only removes a unit file and a script. This one deletes a user, a group and `/srv/shared` — a plausible path on a real machine — so the guard is worth more than the "usable anywhere" property. The exception in maintenance rule 8 now covers read-only tours only.
- `verify.sh` **gained** `vl_need_root` **and** `vl_need_getfacl_sudo`. No check was added or removed, so the counts in §5 are unchanged. Without root, `sudo -l -U appsvc` fails and two checks reported `FAIL` when the honest answer was `SKIP` — the exact false-failure the `SKIP` tier exists to prevent.

Expected on a healthy run: **5 PASS, 1 YOU, exit 0**.

The likeliest real-world failures, in order: `sudo -u appsvc` behaving differently than expected against a `nologin` account; `useradd --system --create-home` not creating the home directory on Rocky 9; and the exact wording of sudo's refusal, which `break-and-fix.sh` describes but does not parse.

Day 05 was written (2026-09-08), and has not been run

Five scripts, same shape as Days 01-04: `lab-noisy.sh` (payload, no root), `setup.sh`, `explore-logs.sh` (read-only tour, no root), `break-and-fix.sh` and `teardown.sh`. Objects: `lab-noisy.service`, `/usr/local/bin/lab-noisy`, `/var/log/lab-app/app.log`, `/etc/logrotate.d/lab-app`, `/var/log/journal`, `SystemMaxUse=200M`, `chronyd`, timezone UTC. Lab tier - CI lints only.

The organising idea is that **two logging systems are running at once** and the reader has probably never seen them side by side. The payload writes one line per second to a plain file *and* to the journal from the same loop, so every comparison in the day is between two destinations fed by one process.

CHOICE	WHY
Payload holds its log open on fd 3, never reopens	This is the whole of failure 1. A rename cannot reach an open descriptor, so removing <code>copytruncate</code> makes the rotated file keep growing while the tailed file stays empty. Without the held descriptor the failure cannot be demonstrated at all.
<code>size 100k</code> rather than <code>daily</code> in the rule	The reader can trigger a real rotation now instead of waiting until tomorrow.
<code>SystemMaxUse=200M</code> , deliberately small	Small enough that <code>break-and-fix.sh</code> can drop it to 5M and show journald deleting its own history with no error anywhere.
Timezone forced to UTC	Correlation across hosts in Days 06-20. Also sets up the <code>TZ=</code> display demo in the tour: the journal stores UTC regardless.
<code>teardown.sh</code> leaves the journal alone	Persistence is a machine improvement, not this day's mess, and deleting a journal is not undoable. It also produces the best line in the day: the service is gone and <code>journalctl -t lab-noisy</code> still has its history.
Clock moved forward, not backward, in failure 3	Backwards time breaks <code>make</code> , databases and certificate validity. Forward is enough to make <code>--since "5 min ago"</code> miss real entries.

One check was tightened rather than left on the weak list. The generator's `a logrotate rule exists for your own log` WAS `ls /etc/logrotate.d/ | grep -q .`, which passes on any stock machine because the distro ships rules of its own. It now requires `/etc/logrotate.d/lab-app` to name `app.log` and contain `copytruncate`. The description is unchanged, so parity holds at 20/20. **Four weak checks remain** (d04, d07, d16, d03), down from five.

A second `v1_manual` was added - `you watched a rotated log keep growing, and found the open descriptor` - taking the day to 5 PASS, 2 YOU, matching Day 04. The README checklist was updated in the same change.

Packages the VM needs, per the Day 04 lesson: `sudo dnf install -y chrony logrotate`. `setup.sh` checks for `chronyc logrotate logger journalctl timedatectl` before changing anything.

Day 14 was written (2026-09-12)

`days/day14/` ships five scripts, a rewritten README and a six-check verify. The day is Ansible fundamentals: `control` manages `node1`, and the whole day is built around one measurement - the second run reports `changed=0`.

Decisions worth knowing:

- **This is the only day whose `setup.sh` refuses root.** The project lives in `$HOME`, Ansible connects as the lab user with the lab user's key, and `become` is per task. Run it with `sudo` and the project is owned by root, `~/.ssh` is `/root/.ssh`, and every task fails as UNREACHABLE for a reason that has nothing to do with the day. `verify.sh` makes "not root" its first check for the same reason - that is the sixth check the stub did not have.
- **The credential is the prerequisite, and it is explicit.** `control` holds nothing `node1` trusts, so the README adds a second push: `./lab/lab.sh push control ~/.ssh/id_ed25519`. `setup.sh` installs `~/lab/id_ed25519` as `~/ssh/id_ed25519` mode 0600, deletes the pushed copy, and then proves plain `ssh` works before writing any inventory. Both the README and the script say plainly that a private key on a control node is a real-world tradeoff, not a pattern.

- **node1's address is an argument, not a guess.** `control` cannot run `virsh`, so `setup.sh` takes the address as `$1` or `$NODE1`, otherwise reuses the one in the existing inventory, otherwise dies pointing at `./lab/lab.sh status`. A stale DHCP lease is the most common cause of a hung run.
- **The project is** `~/ansible-lab`, holding `ansible.cfg`, `inventory/hosts.ini` and `inventory/hosts.yml` (the same hosts in both formats, so `ansible-inventory --graph` can be compared), `group_vars/`, `host_vars/`, `files/`, `templates/`, `site.yml`, and `not-idempotent.yml`.
- **site.yml notifies a handler on** `rsyslog`, not `nginx` or `chronyd`. `/etc/rsyslog.d/*.conf` is definitely included on Rocky 9, `rsyslog` ships installed, restarting it is harmless, and it calls back to Day 05. Day 13 already owns `nginx` on `node1`, so the two days do not collide.
- **teardown.sh is itself a playbook** with every state inverted, and it deliberately leaves `rsyslog` installed and running because the distribution shipped it. It keeps `~/ansible-lab` unless given `--all`, since Day 15 extends the same project with a role.
- `break-and-fix.sh` writes its broken plays to `/tmp/day14-broken`, never into the project, and finishes by re-applying `site.yml` twice.
- **Fixed after the first hardware run (2026-09-12):** `site.yml` had `copy` owning `/etc/lab-day14/lab-day14.txt` and `lineinfile` adding `owner=` to that same file. Two tasks owning one file is the fighting-tasks bug: `copy` restored its checksum, `lineinfile` re-added its line, and every run reported `changed=2` forever, so `verify.sh`'s second-run check failed on an otherwise-correct lab. The `lineinfile` task now owns `/etc/lab-day14/settings.conf` with `create: true`, and the task comment explains the rule: either `copy / template` owns a file entirely, or `lineinfile` edits a file a package owns. `break-and-fix.sh` case 2 already used `settings.conf`, and its cleanup removing the file is harmless because the next `site.yml` run recreates it.
- **Also fixed after that run:** `setup.sh`'s SSH precheck printed only "ssh failed". It now shows the real `ssh -v` error and classifies it (no route, refused, key rejected including Day 12's `AllowGroups labssh` and a `fail2ban` ban), and it refuses an address belonging to `control` itself. The README no longer presents any address as anything but an example, because the first hardware run used the README's `192.168.122.42` verbatim.

The four failures: `command / shell` instead of a module (works, reports changed forever, and is skipped by `--check` so the dry run predicts nothing), `lineinfile` with no `regexp` (idempotent for one value, appending for the next), handler expectations (no change means no restart; a failed play drops pending handlers), and a missing `become`.

Ran green on hardware (2026-09-13). After the two fixes above, the second run reported `changed=0` on `node1` and `verify.sh` printed **6 passed, 0 failed, 2 for you to judge**. Committed by the owner as `3da954e`. The hardware run also surfaced the memory floor described below.

Packages the control VM needs: `sudo dnf install -y ansible-core`. The managed host needs nothing beyond the Python Rocky already ships.

Day 15 was written (2026-09-13)

`days/day15/` ships five scripts, a rewritten README and an eight-check verify. The day is Ansible roles: Days 11, 12 and 13 become `roles/hardening`, and the role is applied to `node2`, which has never been configured by hand.

Decisions worth knowing:

- **The play is a new `hardening.yml`, not Day 14's `site.yml`.** This is a deviation from the stub verify, which ran `site.yml -l node1,node2`. Leaving `site.yml` alone means Day 14 still passes its own verify without `node2` existing, and the split is defensible on its own terms: the baseline and the hardening policy are different things, applied at different times. The rewritten `verify.sh` uses `hardening.yml` throughout.
- **`ansible-core` ships no collections, so there is no `selinux` or `firewalld` module.** Confirmed on the Day 14 hardware run (`ansible [core 2.14.18]`, no `ansible.posix`, no `community.general`). The role therefore does SELinux with `lineinfile` on `/etc/selinux/config` plus `getenforce` + a `when: -guarded setenforce 1`, and `firewalld` with `firewall-cmd --permanent --list-*` registered, then `--add-*` guarded by `when: item not in ...`, with a `firewall-cmd --reload` handler. Longer than a module call and better teaching material: the pattern is what a module does internally. Do not "improve" this by adding a `collections: key` - it fails on a stock `dnf install ansible-core`.
- **Order in `tasks/main.yml` is a safety property, and it is commented as one.** `ssh.yml` creates the `labssh` group and adds the login account *before* the templated drop-in lands `AllowGroups labssh`. Reversed, `sshd -t` still validates (a group need not exist for the config to parse), the reload succeeds, and the host refuses every subsequent connection including Ansible's. `break-and-fix.sh --hard` describes this and the equivalent "apply the role to control" mistake rather than performing either.
- **`epel-release` is its own task before the package list**, because `fail2ban` and `fail2ban-firewalld` are not in the Rocky repositories and a single package task fails on a clean host at name resolution time.
- **`hardening_open_ports` defaults to `[]`; `8080/tcp` lives only in `group_vars/webservers.yml`.** That is the day's precedence lesson made physical - `node1` opens the port, `node2` does not, same role, and editing `defaults/main.yml` changes neither. `vars/main.yml` holds the paths the handlers depend on, deliberately at high precedence.
- **Vault.** `group_vars/lab/` is the directory form (`vars.yml` readable, `vault.yml` encrypted), the password is a generated 32-char string in `~/.vault-pass-lab` at 0600 *outside* the project, and `ansible.cfg` gains `vault_password_file` so no command needs the flag. `setup.sh` only encrypts when the file does not already start with `$ANSIBLE_VAULT`, because `ansible-vault encrypt` is not idempotent - twice gives you a vault inside a vault. The role reads `hardening_alert_token`, which points at `vault_hardening_alert_token`, so `grep -r vault_` finds every secret.
- `setup.sh` refuses root for Day 14's reasons, takes both node addresses as arguments (falling back to the inventory), refuses any address belonging to `control` itself, and runs the Day 14-style classified `ssh -v` precheck against both nodes before writing anything.
- The five failures in `break-and-fix.sh`: a `--limit` typo (`skipping: no hosts matched`, **exit 0** - a green run that deployed nothing), editing `defaults/` that `group_vars` overrides, a tag that skipped the task the requested one depended on, a handler dropped by a failed play, and a missing vault password (whose error names a file, not the variable people hunt for). Broken plays go to `/tmp/day15-broken`, never into the project, and the script finishes by re-applying `hardening.yml` twice.
- `teardown.sh` is itself a playbook with every state inverted. It targets the `fresh` group (`node2`) by default because `node1`'s hardening was earned by hand on Days 11-13, leaves SELinux enforcing, leaves `firewalld` installed, and leaves the `labssh` group alone - removing it during a cleanup is its own lock-out. `--all` also targets `node1` and deletes the role and playbook.
- `verify.sh` has eight automatic checks: `not-root`, role layout, both nodes ping, syntax, applies with `failed=0` on both, second run `changed=0` on both, `node2 Enforcing`, `node2 passwordauthentication no`,

plus the vault file being ciphertext. Two `you` items: hand-broke a setting and found it with `--check`, and can explain why editing `defaults/` did nothing.

- **Fixed during the first hardware attempt (2026-09-13):** the README's run block listed only `./lab/lab.sh push control ~/.ssh/id_ed25519` and omitted the bare `./lab/lab.sh push control`. With a path argument `push` copies that one file and nothing else, so `~/lab` on control held a key and no `days/`, and `cd ~/lab/days/day15` failed. Day 14's README has both lines; Day 15's now has both plus a paragraph explaining the difference. Worth remembering for Days 16-20: **every day that needs a credential needs two push lines.**
- **Also fixed during that attempt:** `setup.sh` died with "no ansible.cfg here. Run Day 14's setup.sh first" on a rebuilt `control` VM. The Ansible project lives in the VM's home directory, not in the repository, so a `lab.sh down` between days takes it with it - making a previous day's setup a hidden prerequisite. **Day 14 is no longer required:** if `ansible.cfg` is absent, Day 15 writes the whole file (Day 14's config plus `vault_password_file`); if present, it only inserts the vault line. The Day 15 README gained a "Do you need Day 14's setup first?" section. General rule for Days 16-20: **a day may extend a previous day's project, but must not require it**, because VM home directories do not survive a rebuild.

Ran green on hardware (2026-09-13), on all three VMs, after the two fixes above: `verify.sh` printed **9 passed, 0 failed, 2 for you to judge**. The second `hardening.yml` run reported `changed=0` on both nodes, node2 came out `Enforcing` with password auth off having never been touched by hand, and `break-and-fix.sh` reproduced all five failures.

Packages: `sudo dnf install -y ansible-core` on control, nothing on the nodes.

The memory floor was raised across the lab (2026-09-13)

The Day 14 hardware run printed `WARNING Requested memory 1024 MiB is less than the recommended 1536 MiB for OS rocky9 when creating control`. Advisory, not fatal - but it is the same wall Day 12 hit as a bare `killed` from `dnf` when node1 had 768 MB, so the guess was raised rather than explained away.

- `control 1024 -> 1536 MB`, `node1 2048 MB`, `node2 1024 -> 2048 MB`
- `lab/lab.sh`: `vm_mem()` values, the header comment, and the free-memory check thresholds (3800 -> 4200 MB for two VMs, warn text "under 3.2 GB" -> "under 3.7 GB")
- Two-VM days are now **~3.5 GB** and Day 15 is **~5.5 GB**, updated in `README.md`, `docs/curriculum.md`, this file's VM and budget tables, and the memory lines in `days/day12`, `days/day14`, `days/day15`, `days/day16` and `days/day20`
- The rule, written down in `lab/lab.sh` and `README.md`: **nothing in this lab runs below 1536 MB**. Below that, `dnf` is OOM-killed with no explanation.

On an 8 GB laptop Day 15's 5.5 GB is tight. Both `README.md` and the Day 15 README now say to close the browser, or to start `node1` after the role has been applied to `node2` - which is what `--limit` is for.

Day 17 was written (2026-09-13)

`days/day17/` now ships five scripts, a full README and a 19-check verify. The day is a reverse proxy: nginx on `node1` terminating TLS for `www.lab.test` in front of a plain-HTTP backend on `127.0.0.1:8080`.

Decisions worth knowing:

- **The CA is reused if present, issued if not.** `setup.sh` looks for `/etc/lab-tls/{ca.crt,ca.key}` from Day 10 and creates one with Day 10's openssl commands otherwise, so the day is self-contained on a fresh

VM. Same precedent as Day 16: a day may lean on another day's lesson, not on its leftovers.

- **This is the first thing in the repo that installs a trust anchor.** Day 10 deliberately stopped short. Day 17 copies the CA to `/etc/pki/ca-trust/source/anchors/lab-ca.crt` and runs `update-ca-trust`, which is what makes plain `curl https://www.lab.test/` succeed. The README says out loud that Firefox, Java and node keep their own stores.
- **The certificates live under** `/etc/pki/tls`, not next to the CA, because that tree is already labelled `cert_t`. `setup.sh` still runs `restorecon` on both files.
- **The backend is** `python3 -m http.server --bind 127.0.0.1` under `lab-app.service` With `DynamicUser=yes`. Started by systemd it runs unconfined, so no port labelling is needed - the SELinux lesson here is the boolean, not `semanage port`.
- `setsebool -P httpd_can_network_connect on`. `nginx` is `httpd_t` and may not open the upstream socket without it. This is the Day 13 callback the curriculum promised, and break-and-fix turns it off to show the 502.
- **firewalld uses the same daemon-vs-offline split as Day 16**, and 8080 is deliberately left closed - `verify.sh` asserts it is closed.
- `setup.sh` **installs firewalld and starts it**. First hardware run had no firewalld on `node1` at all (Day 11 installs it; the VM had been rebuilt since), so `open_port` took the "no firewalld here" branch, said nothing was wrong, and `verify.sh` then failed the port check. firewalld is now in the package list and the daemon is started and enabled before any port is added. The general rule is now two-part: **a day may not assume another day's daemon is running, nor that its packages are still installed.**
- **Days 14-16 are not prerequisites.** Day 10, Day 11 and Day 13 are the ones it builds on.
- **HTTP/2 is version-dependent and** `setup.sh` **now detects it**. First hardware run failed at step 7 with `unknown directive "http2": http2 on; is nginx >= 1.25.1 and Rocky 9 ships 1.20, which wants listen 443 ssl http2;`. The script parses `nginx -v` and writes whichever form applies, printing which one it chose. Same class of mistake as assuming another day's daemon is running: **do not assume a version, ask the binary.**

`break-and-fix.sh` runs four failures: the boolean off (502 plus `Permission denied`), a certificate for `other.lab.test` (`openssl verify` passes, clients refuse), the backend stopped (502 refused, with prose on why a dropped packet would give 504 instead), and a renewed certificate that `nginx` was never reloaded to read. `--hard` describes two that fail silently: the backend bound to `0.0.0.0`, and opening 8080 "to test something".

Not yet run on hardware. `bash -n clean`, `tests/cli.sh` **352 passed, 0 failed**, exec bits set on all six files.

Day 18 was written (2026-09-14)

`days/day18/` now ships five scripts, a full README and a 22-check verify. The day is one VLAN-aware bridge in a network namespace with five things plugged into it: two access hosts in VLAN 10, one in VLAN 20, a trunk host with `t-eth0.10` and `t-eth0.20`, and a two-leg `active-backup` bond.

- **0 MB, no VM.** Namespaces `sw18`, `h18a`, `h18b`, `h18c`, `h18t`, `h18d`, deliberately named apart from Day 06's `client` / `router` / `resolver` / `auth` so both topologies can be up at the same time. `teardown.sh` removes only Day 18's six.
- **The point of the day** is that `ip link` shows a port as UP whether it is in the right VLAN, the wrong VLAN or none at all. Only `bridge vlan show` can tell you, and `vlan_filtering` is a flag that can be off while

every table below it is perfect.

- **Verify is layer-2 honest:** the bridge has no address and `net.ipv4.ip_forward` is 0 in `sw18`, so the isolation cannot be mistaken for a firewall rule. Two checks assert that explicitly.
- **Bonding is optional.** Some stripped-down kernels have no `bonding` module. `setup.sh` drops `/run/day18/bond.enabled` when it built the bond, and the three bond checks are written `[ ! -f /run/day18/bond.enabled ] || ...` so they pass without asserting on such a kernel instead of failing the day for a kernel the reader did not build.
- **CI executes this day for real.** Day 18 was already in the `verifiable-days` matrix and in `NETNS_DAYS` in `lab/ci-day.sh`; now that `scripts/setup.sh` exists, `ci-day.sh` runs setup and verify instead of printing `SKIPPED` (not yet implemented).
- `break-and-fix.sh` has four failures plus `--hard`: port removed from its only VLAN, right VLAN with the wrong PVID (connectivity to the *wrong* network, which is worse than an outage), trunk made untagged, active bond leg taken down (nothing happens, which is the lesson), and under `--hard` `vlan_filtering 0` with every table left intact.

Two failures on the first real run, both fixed

The first run on the owner's laptop printed **20 passed, 2 failed**. Both were setup bugs, not verify bugs, and both are worth remembering:

1. **VLAN 1 was removed from every port FAILED.** `setup.sh` deleted VLAN 1 from every *port* but not from the bridge device itself. The bridge has its own row in the VLAN tables and needs the `self` flag: `bridge vlan del dev br0 vid 1 self`. Fixed in step 2, and the verify check now strips the port column with `sed` so continuation lines are examined too.
2. **forwarding is off in the switch namespace FAILED.** The check assumed a fresh namespace starts with `net.ipv4.ip_forward=0`. That is usual but not guaranteed - the value can come from a host where Docker, a VPN or an old `sysctl.d` file turned it on. `setup.sh` now sets it explicitly (`echo 0 > /proc/sys/net/ipv4/ip_forward` inside the namespace) and says so, and verify reads the `/proc` file rather than calling `sysctl`, which lives in `/sbin` on some distributions and would look like "forwarding is on" when merely absent. **Set the state the day depends on; do not assume a default.** Same rule as Day 17's `firewalld` lesson.

Re-run `sudo ./scripts/setup.sh` and then `sudo ./verify.sh`: **22 passed, 0 failed, 2 for you to judge**.

Day 19 was written (2026-09-14)

`days/day19/` now ships five scripts, a full README and a 22-check verify. The day is the two detection sensors, side by side: **Suricata** on the wire and **auditd** on the disk. Runs on node1 only, ~2 GB, no second VM.

- **Suricata comes from EPEL**, like Day 12's `fail2ban`. `setup.sh` enables `epel-release` if it has to, then installs `suricata audit jq bind-utils`.
- **The interface is detected, never assumed.** The packaged `af-packet` section says `eth0`; Rocky 9 on libvirt is `enp1s0`. `setup.sh` rewrites only the first `interface:` line inside `af-packet:` using the device from `ip route show default`, and verify asserts the two still agree. An IDS on the wrong interface is green in `systemd` and detects nothing.
- **No ruleset is downloaded.** `suricata-update` is deliberately not run: the day is about rules you wrote. `setup.sh` creates an empty `/var/lib/suricata/rules/suricata.rules` so the packaged config stays valid,

and adds `/etc/suricata/rules/lab.rules` to `rule-files` by absolute path so a future `suricata-update` cannot overwrite it. It also keeps memory small, which matters on a 2 GB VM - the ET open ruleset would not be kind here.

- **Two local rules**, sid 9000001 (ICMP echo request) and sid 9000002 (a `dns.query` content match on a canary name). Verify checks that no lab sid is below 1000000, which is the public-ruleset range.
- **Setup fires both sensors and fails if nothing is recorded.** It pings the gateway, looks up the canary name, reads `/etc/shadow`, writes to `/etc/lab-canary`, then waits up to 30s for sid 9000001 in `eve.json` and for a SYSCALL record under key `shadow_watch`. A detection stack nobody has seen fire is a detection stack you do not know works.
- **auditd rules** in `/etc/audit/rules.d/99-lab.rules`: `/etc/shadow` and `/etc/sudoers` (`-p wa`), `/etc/lab-canary` (`-p rwa`, so reads count), and `-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k root_cmd`. Loaded with `augenrules --load`, never by restarting the daemon - `systemctl restart auditd` is refused by design, and `break-and-fix.sh` shows that refusal on purpose.
- `augenrules --check` is a **verify check**, because "the file says one thing and the kernel says another" is the third appearance of the same lesson after Day 11's `sysctl` files and Day 16's `wg` config.
- `break-and-fix.sh` has four failures plus `--hard`: a rule missing its semicolon (which fails `-T` for the entire ruleset, not just that rule), a valid rule never reloaded, an audit rule in the file but not in the kernel, and `auditctl -D` (perfect files, empty kernel rule set). `--hard` additionally points `af-packet` at `lo` and leaves all five live.
- **CI still only lints this day.** No GitHub runner has an interface worth capturing or a kernel audit subsystem the job can rely on, so Day 19 is not added to the `verifiable-days` matrix. `tests/cli.sh` covers the five scripts statically.

'lab-ids watch' showed nothing, because waiting looks like broken

The two-session demo was run correctly - `lab-ids watch` in one session, `lab-ids trigger` three times in the other - and the watching session printed nothing, so it was Ctrl-C'd within seconds each time. Nothing was broken: on a 2 GB VM `suricata` batches writes to `eve.json` and the alert lands tens of seconds later, well after anyone staring at a blank screen has given up.

The design mistake was mine. An exercise whose success condition is "the user waits long enough", with no feedback while waiting and no known wait time, is not an exercise.

Fixed:

- new `lab-ids prove`: counts alerts in `eve.json`, fires both sensors, polls every 3s for up to 90s, then reports how long the flush actually took and prints the alert. One session, no coordination, no guessing. This is now the recommended path in both the README and `setup.sh`'s closing note.
- `lab-ids watch` keeps the two-session demo but is no longer ambiguous: it prints the last 3 alerts already in the log so you see output immediately, says the wait can be tens of seconds, and runs a background heartbeat that prints `... still watching, Ns, no new alert yet` every 15s. The heartbeat is killed by a trap on `EXIT/INT/TERM`.
- README section rewritten as `### Seeing the flush interval for yourself`, leading with `prove`, with the delay itself called out as the lesson.

Rule for later days: any step that waits on an asynchronous system must show progress while waiting and state the expected wait. Otherwise "working" and "hung" are indistinguishable, and the reader learns to

distrust the lab rather than the system.

The live-watch hint was unusable, and now it is a subcommand

setup.sh ended with a note saying to run `lab-ids trigger` "from another terminal" while watching a `tail -f ... | jq ...` pipeline. Two problems: the two-session workflow was never explained in the README, and the pipeline was printed with escaped quotes, so pasting it appended it to the previous prompt and produced `parse error: Invalid numeric literal at line 2, column 4.`

Fixed:

- new `lab-ids watch` subcommand follows `eve.json` and prints one compact line per alert (`tail -n 0 -F | grep --line-buffered | jq -c --unbuffered`). `--line-buffered` and `--unbuffered` matter: without them the alerts sit in a 4 KB pipe buffer and arrive in silent bursts, which looks exactly like a broken sensor.
- setup.sh's closing note now spells out session A / session B, says not to paste both into one prompt, and gives `./lab/lab.sh ssh node1` for opening the second one.
- README gained a `### Watching an alert arrive, live (two SSH sessions)` section with both sessions written out, the Ctrl-C instruction, and the raw pipeline for anyone who would rather type it than trust the wrapper.
- Usage strings and the scripts table list `watch` alongside `status|alerts|audit|trigger`.

Rule for later days: anything a script tells the user to run must exist in the README too, and any command that does not return on its own must say so and say which terminal it belongs in.

Fourth run: the burst was not the answer - auditd's freq was

Even with 60 reads of the canary and a 30s wait, setup still reported no audit event, with `backlog 0`, `flush = INCREMENTAL_ASYNC` `freq = 50` and 13294 lines already in `audit.log`. verify, run right after, passed all 22 - so the events did land, just later than setup was willing to wait.

`backlog 0` was the tell: the kernel queue was empty, meaning the records had already been handed to auditd and were sitting in *its* buffer, unwritten. Generating more events does not help when the delay is on auditd's side.

Fixed properly this time:

- step 6 now sets `freq = 1` in `/etc/audit/auditd.conf` (original copied to `$BACKUP_DIR/auditd.conf.orig`) and reloads auditd with `systemctl reload auditd`, falling back to `service auditd reload` then `kill -HUP auditd`. With `freq = 1` every record is written as it happens, which is what you want on a machine you are actually watching.
- step 8's audit proof is **no longer fatal**. The rules are loaded and the network half is proven by then, so a slow flush prints the diagnosis plus `sudo ausearch -k lab_canary -ts today` and continues to `done`. verify is the gate, and it checks the audit event anyway.
- teardown restores the packaged `auditd.conf` and reloads auditd.

Lesson for later days: setup should hard-fail only on state it can control. Asynchronous evidence belongs in verify, or in a warning, never in a `die` - a script that aborts because a log has not been flushed teaches the wrong reflex.

Third run on node1: setup failed the audit proof, verify passed it

setup died on `auditd is running and the rule is loaded, but nothing was recorded` - with the diagnosis printing `auditd: active`, `kernel enabled: 1`, `canary rule: -w /etc/lab-canary -p rwa -k lab_canary`, `backlog 0` - and then `verify.sh` immediately reported 22 passed, 0 failed, including `an audit event was actually recorded`. Both were telling the truth.

Two things differed:

1. setup searched `ausearch -k lab_canary -ts recent` (last ten minutes); verify searches with no `-ts`, so it matched an older event from a previous run or from `lab-ids trigger`.
2. The real cause: **auditd buffers**. `/etc/audit/auditd.conf` ships `flush = INCREMENTAL_ASYNC` with `freq = 50`. The kernel hands each record to auditd immediately, but auditd writes `audit.log` only every 50 records. On an idle VM, one read of the canary sits in that buffer, so `ausearch` finds nothing while the event has genuinely been recorded. `backlog 0` confirmed the kernel queue was empty - the records were already with auditd, unwritten.

Fixed in setup step 8: generate a burst of 60 reads of the canary, past `freq`, instead of one; search `-ts today`; keep accessing the file between retries; and add `flush/freq` and `audit.log` line count to the failure diagnosis. README gained the buffering lesson - a log search that says "clean" about a machine that is not.

Lesson for later days: when setup and verify disagree, compare their queries before suspecting the host. Time windows and buffering explain more disagreements than broken configuration does.

Second run on node1: two verify bugs and a wrong audit trigger

After the readiness fix, setup got all the way to step 8, proved sid 9000001, and then died on `auditd is running and the rule is loaded, but nothing was recorded`. verify then reported 20 passed, 2 failed: `the kernel's rules match the files (augenrules --check)` and `the canary watch includes reads, not only writes`. None of the three were the machine's fault.

1. **Permission letters decide what is recordable.** setup proved the audit half by reading `/etc/shadow` and searching for a `shadow_watch` event, but that watch is `-p wa` - writes and attribute changes only. A read of it is never recorded, so the proof could not have passed on a healthy host. setup now reads and writes `$CANARY`, which is watched `-p rwa`, searches `-k lab_canary`, retries for 20s because auditd writes asynchronously, and on failure prints auditd state, kernel enabled, the canary rule and the backlog. The `/etc/shadow` read is kept and its silence is now the lesson.
2. **augenrules --check prefixes its own name:** the output is `/sbin/augenrules: No change`, not `No change`, so the string equality in verify could never be true. Now matched with `grep -q 'No change'`.
3. **auditctl -l prints watches as** `-w /etc/lab-canary -p rwa -k lab_canary`. verify was grepping for `perm=.*r`, which is the `ausearch` field name, not the `auditctl -l` format. Now the permission letters are extracted and checked for `r`.

The verify check `an audit event was actually recorded` moved from `shadow_watch` to `lab_canary` for the same reason as 1. Still 22 checks and 2 manual.

Lesson for later days: a check that has never once passed is not a check. All three of these were written against the wrong output format or the wrong permission model and would have passed review by reading alone.

First run on node1 failed: 'active' is not 'ready'

`setup.sh` step 8 died with `no alert for sid 9000001 - is eth0 really the interface with traffic?` after step 5 had already printed `no eve.json yet`. The interface was right (this node1 names its NIC `eth0`, so the detection from `ip route show default` worked). The cause was timing: `systemctl is-active suricata` returns success as soon as the process exists, but Suricata then parses the config, builds the detection engine from every rule and creates capture threads before it looks at a single packet - tens of seconds on a 2 GB VM. The pings were sent into that gap, so as far as the IDS was concerned they never happened, and `eve.json` cannot report an alert it never had a chance to generate.

Fixed in `setup.sh`:

- new step 5b waits up to 120s for `Engine started` in `/var/log/suricata/suricata.log`, which is the only line that means ready, and bails early if the unit dies while waiting.
- step 8 now sends traffic on a loop (24 rounds, ping + dig, 4s apart, up to ~2 minutes) instead of once, because `eve.json` is flushed on an interval as well - a single ping followed by an immediate grep is a race even on a healthy host.
- on failure it prints a diagnosis instead of guessing at one cause: the configured interface, the real interface, the unit state, the `eve.json` line count, whether the engine started, the rule count, and the last 20 lines of `suricata.log`.
- `lab-ids status` gained an "is the detection engine actually up" section for the same reason.

The general lesson, now in the README: a health check that asks systemd whether a sensor is working gets this wrong on every IDS, agent and log shipper there is. `active` is not `ready`.

- **ShellCheck caught SC2294 on the first pre-commit run.** Day 19's tour and break-and-fix scripts were written with `run() { ...; eval "$@"; }`, which ShellCheck rejects ("eval negates the benefit of arrays"). Days 13-18 all use `run() { printf '$ %s\n' "$1"; bash -c "$1" ...; }` and Day 19 now does too. **Copy the existing helper rather than reinventing it** - the repo's `shellcheck -x -S warning` gate is part of the house style.
- **teardown.sh never stops auditd and never deletes `eve.json`.** Both are deliberate: the audit trail is not a lab artefact, and deleting logs while tidying up is how incidents become unexplainable.

Next: Day 20 - backup, restore and the restore drill, the last day.

Day 20 was written (2026-09-15)

`days/day20/` now ships five scripts, a full README and a 22-check verify. The last day, and the only one whose subject is a proof rather than a configuration: **a backup on another host, on a timer, with a restore that has actually been run and timed.** Two VMs, ~3.5 GB.

- **The repository lives on control, over sftp:** `sftp:restic@<control-ip>:/srv/restic/repo`. A backup on the same disk as the data is not a backup, so the day needs the second VM. node1 pushes; control has no restic, no password, and no key back to node1 — if control is compromised the attacker gets encrypted blobs.
- **setup.sh is role-aware**, the Day 16 pattern: `ROLE=${ROLE:-$(hostname -s)}`, overridable for a differently named host. It runs in four passes, because a public key has to travel between two hosts and no single command on one host can do that: control (create the user and repo), node1 (install, seed `/srv/data`, generate root's key, print the exact command, `exit 0`), control (authorise that key), node1 with control's

address (the real work). Pass 2 exiting 0 while asking for a manual step is the same contract Day 16 used, and it says so in as many words.

- **The restic user has /bin/bash , deliberately.** /sbin/nologin is the reflex for a service account and it breaks sftp: restic runs the remote sftp-server through the login shell, so nologin gives "connection lost" with nothing in any log to explain it. setup prints getent passwd restic | cut -d: -f7 so the learner sees the shell.
- **RESTIC_PASSWORD_FILE , never RESTIC_PASSWORD .** /etc/restic/password is 0600 root:root, generated with openssl rand -base64 24 , and never regenerated if it exists (that would orphan the repository). /etc/restic/env carries the repository and the path to the password, and verify asserts the env file contains no RESTIC_PASSWORD= .
- **The timer is provable within ninety seconds.** restic-backup.timer has OnCalendar=*:0/10 as the real schedule and OnActiveSec=30s so the first run happens while the learner is still watching, plus Persistent=true and AccuracySec=1s . Step 8 polls LastTriggerUsec 30 times at 3s, states the expected wait up front, prints a progress line every fifth attempt, and is **non-fatal** — Day 19's rules applied before they could bite: a step that waits on an asynchronous system must show progress, and must not hard-fail on state it does not control.
- **EnvironmentFile= is the lesson of the service unit.** A timer's service starts with almost no environment, which is why a backup can work by hand and fail on the schedule. Failure 3 in break-and-fix.sh deletes that one line, shows lab-backup run succeeding by hand and the same command exiting non-zero under systemd, then puts it back.
- **lab-backup** (/usr/local/bin/lab-backup , installed from scripts/lab-backup.sh) is both the payload and the ExecStart: status | run | snapshots | drill | check | forget | unlock . run takes the snapshot and applies retention in the same unit of work — --keep-last 3 --keep-daily 7 --keep-weekly 4 --keep-monthly 6 --prune — because a repository with no retention fills the disk it lives on, and a full disk is how a working backup stops being one.
- **drill is the day.** It restores latest into an empty /var/tmp/restore (restoring on top of last week's restore is how a missing file passes a diff), times it, runs diff -r /srv/data /var/tmp/restore/srv/data , compares the mode of the 0600 conf/token , and writes the elapsed seconds to /var/lib/lab-backup/last-restore-seconds . The comparison decides the exit status: a restore that ran is not a restore that worked. Step 9 of setup runs the drill and is fatal, because a mismatch is state the script controls.
- **break-and-fix.sh : four failures, repaired, and a fifth behind --hard .** (1) the password file is gone — the repository is intact and unreadable, which is the same thing as lost, and the only failure in the file that cannot be fixed from that host; (2) the repository host is unreachable, diagnosed by proving the transport with ssh -o BatchMode=yes before suspecting restic; (3) the missing EnvironmentFile= ; (4) a stale lock from a kill -9 'd backup, cleared with restic unlock and never with rm .
- **--hard is Day 19's silent-failure shape again, and worse:** an exclude file swallows /srv/data , so the service exits 0, a new snapshot appears, the timer is green, and every dashboard buildable from systemd says the backup is healthy. The snapshots are empty. Nothing is hidden (systemctl cat , cat /etc/restic/exclude , restic ls latest), the original unit is kept at /tmp/day20-broken/restic-backup.service.prehard , and the fix has to be proven with a drill rather than with a green unit.
- **verify runs on node1, with sudo,** and is 22 checks in four groups: configuration (env file 0600, repository is sftp: , password file not password, 0600 root:root, data still present), repository (readable, ≥1 snapshot, restic check , snapshot contains /srv/data , snapshot not empty, no stale lock), schedule

(payload executable, `keep-daily`, `--prune`, unit exists, `EnvironmentFile=`, `ExecMainStatus` is 0, timer enabled, timer active, `LastTriggerUsec` $\neq$ 0) and the restore (`diff -r`, mode preserved). The stub's seven wordings are kept verbatim, as are both manual items.

- **The "latest snapshot is not empty" check exists because of `--hard`.** `restic snapshots` returning a row proves nothing; `restic ls latest | grep -c '^/srv/data/'` is what catches an excluded source. A check that cannot fail is not a check — Day 19's rule.
- **`teardown.sh` is role-aware too.** On node1 it removes the timer, unit, payload, restore tree and break-and-fix leftovers, and deliberately keeps `/etc/restic` and `/srv/data` unless `--all`; deleting the password makes every snapshot on control unreadable, so it takes an explicit flag. On control, nothing happens without `--all`, which deletes the repository and the account.
- **The README lists what the day leaves unfinished on purpose:** append-only repositories (node1 can delete what it wrote), off-host password custody, `restic check --read-data`, and monitoring the *absence* of backups. It also spells out `set -a; . /etc/restic/env; set +a` for running `restic` by hand, because every command a script mentions has to exist in the README.

`tests/cli.sh` : **397 passed, 0 failed** (was 382). Not yet run on hardware — it needs both VMs and the four-pass key exchange.

Two README gaps fixed after the first review, both fair hits: the "Run it" section had no `lab.sh up / status / push / ssh` block (every earlier day has one, and this day needs `push` to **both** hosts plus two SSH sessions because the passes alternate between them), and the reset snippet under `--hard` was a bare `teardown.sh && setup.sh <control-ip>` with no host named — which reads like "terminate both hosts" and is wrong in any case. It is now explicitly **node1 only**, with the reason it is one pass rather than four (the password file and the authorised key on control both survive `--hard`), and the same wording replaced the corresponding `note` in `break-and-fix.sh`. A new **Tear it down** section covers both hosts in order, node1 first, with the `--all` semantics on each and `./lab/lab.sh down control node1` as the fast path. Also removed an unused `RESTORE=` from `setup.sh` (ShellCheck SC2034 in the owner's pre-commit run; the variable belongs to `lab-backup.sh`, which uses it).

First hardware run found a real bug: SELinux on control silently refuses node1's key. Pass 4 died at step 4 with `failed: the repository host is not reachable as restic`, while control had just said `authorised the key in /srv/restic/.ssh/authorized_keys`. Nothing about the owner, the mode or the key was wrong. `ssh` will only read an `authorized_keys` labelled `ssh_home_t`, and `/srv/restic` is not a home directory as far as the policy is concerned, so the file came out `var_t` and the client got `Permission denied (publickey)` with nothing useful logged on control. This is the price of putting a home under `/srv`, which the day does deliberately. `setup_control()` now installs `polycoreutils-python-utils` if `semanage` is missing, adds the equivalency `semanage fcontext -a -e /home/restic /srv/restic`, runs `restorecon -RF`, also does a direct `chcon -R -t ssh_home_t` as a fallback, and prints whether the resulting label is right. The node1-side failure message was also wrong in a way that wasted the owner's time: it suggested `ssh -v restic@<control>`, which runs as the calling user, has no key on control, and is therefore denied for a completely unrelated reason. It now says `sudo ssh -v -i /root/.ssh/id_ed25519` and names SELinux as the suspect if the key is authorised but still refused.

Second hardware run: setup died silently at step 8 because of `grep -c`. The screen ended with `ok the scheduled run exited 0` and a shell prompt - no step 9, no error, exit status invisible because the owner's next command replaced it. The line was `COUNT="$(restic snapshots --json | grep -c short_id)"`, and `grep -c`

exits 1 when it counts zero, which under `set -euo pipefail` kills the script on the spot. So the repository was empty, the drill was never run by setup, and the owner's manual `lab-backup drill` reported `Fatal: failed to find snapshot`. Now `|| true` with a `COUNT=0` floor, and on zero snapshots setup prints `journalctl -u restic-backup.service -n 20` before taking a backup by hand, so the reason the scheduled run produced nothing is on screen rather than guessed. The `ok the scheduled run exited 0` claim was dishonest too: `ExecMainStatus` is 0 for a unit that has never run at all, so step 8 now checks `ExecMainStartTimestamp` first and says so plainly when the service has not started yet.

Then the lock refused to go, and `unlock` had been lying about it. After the recovery pass the day scored `21 passed, 1 failed`, with only `no stale lock is holding the repository` red. Cause: `restic unlock` removes only locks it can PROVE are stale - same host, pid not running - and the lock `kill -9` leaves in failure 4 is often not provably stale, so it survives. Every caller treated one `restic unlock` as success: `cmd_unlock` printed "removing stale locks" and stopped, `setup_node1` printed "cleared a stale lock" without checking, and break-and-fix said "fix: remove locks that no live process owns" and moved on. Now `lab-backup unlock` counts the locks after unlocking, says `N lock(s) survived - restic will not call them stale`, refuses to escalate while `restic-backup.service` is active, and otherwise runs `restic unlock --remove-all` and explains why that is not the default; setup escalates the same way during a recovery pass; and break-and-fix demonstrates the escalation instead of assuming the first unlock worked. README failure 4 and step 7 say the same thing.

Step 8 `--hard` then exposed a second honesty problem: recovery was not recoverable. After `--hard`, `sudo ./scripts/setup.sh <ip>` left `/etc/restic/exclude` in place and left the empty snapshot as `latest`, so setup's own step 9 drill ended in `DIFFERENT - Only in /srv/data: conf, uploads` and `failed: the restore did not match /srv/data`, with no explanation of why - the script sabotaged by another script could not undo it. `setup_node1` now removes a leftover exclude file, runs `restic unlock` when a lock exists, and after counting snapshots checks whether `latest` actually contains anything under `/srv/data`, taking a fresh backup when it does not. `break-and-fix.sh --hard` now prints the whole repair (four lines: `rm` the exclude, reinstall the `.prehard` unit, `daemon-reload`, `unlock && run && drill`) and says plainly that restoring the unit alone leaves the empty snapshot as `latest`, and that failure 4's lock keeps `no stale lock is holding the repository` red until it is cleared. The README step 8 section carries the same four lines and states that `18 passed, 4 failed` is the correct score for a still-broken day, not a defect.

`#!` was expanded by the wrong shell (2026-09-15). The PID-kill fix in failure 4 was written as `run "restic backup ... & BPID=$!; ..."`. Double quotes mean the OUTER script expands `#!` before `run` ever passes the string to `bash -c`, and with `set -u` and no background job of its own that is `line 105: !: unbound variable` - break-and-fix.sh died halfway through the day. The command is now single-quoted so the inner `bash -c` does the expansion, with only `$DATA` interpolated from outside. Lesson: when you build a command as a string, decide deliberately which shell expands each variable.

Verify was reporting its own lock (2026-09-15). After the orphan was killed, the repository was provably clean - `lab-backup unlock` removed one lock and said `no locks left`, a backup ran, `restic check` printed `no errors were found` - and `verify.sh` still failed only `no stale lock is holding the repository`. Cause: the lock check sat *after* the repository passes an integrity check, and `restic check` takes an EXCLUSIVE lock (`create exclusive lock for repository` is in its own output). Over sftp that lock is still visible when the next command runs, so verify was reporting a lock it had taken itself. The lock check is now the first repository check, before anything that locks. `lab-backup unlock` also prints `restic cat lock <id>` for each lock, naming the host and pid that wrote it. Lesson: a check must not measure a side effect of the checks above it.

The lock had a live owner - the real bug in failure 4 (2026-09-15). Hardware showed 19 passed, 3 failed: the stale lock, the repository passes an integrity check, and the scheduled service has run and exited 0. All three were one cause. `break-and-fix.sh` failure 4 killed the backup with `kill -9 %1` inside `run()`, which runs `bash -c` - job control is off in a non-interactive shell, so the jobspec can fail to match and the backup keeps running as an orphan, refreshing its lock every few minutes. A lock with a living owner is not stale, survives `unlock`, and is re-taken immediately after `--remove-all`; meanwhile `restic check` refuses (it wants the repository to itself) and every timer run exits 1. Fixes: failure 4 now captures `#!` and kills by PID, then checks `pgrep -f 'restic backup'` and `pkill -9` any survivor; the `--remove-all` branch prints `pgrep -af restic` first and says a returning lock means a live owner; `lab-backup unlock` refuses to escalate while a restic process is running on the host and prints it; README gained a `When a lock will not go away` section with the three-red-lines table and the `pgrep / pkill /unlock/run/check` recovery. Lesson: never kill a background job by jobspec in a script, and a lock that returns is never stale.

Verify was racing its own timer (2026-09-15). After the lock check was fixed to count IDs, hardware still failed it, and a hand repair at 15:51 also failed `at least one snapshot exists`. Both were real and neither was a repair problem: the timer is `OnCalendar=*:0/10`, so a scheduled backup starts at every tenth minute and `forget --prune` takes an EXCLUSIVE lock - which makes `restic snapshots --json` fail and leaves a live lock in `restic list locks`. Verify ran seconds after the :50 trigger both times. `verify.sh` now waits for `restic-backup.service` to go inactive before any repository check, and the lock check retries for ten seconds so a lock from a run that has just exited is not called stale. README: `lab-backup unlock` was removed from the step 7 command list (there is nothing to unlock before step 8, and two unlocks in one day read as a contradiction), the step 8 hand repair is now a numbered four-step list explaining why the exclude file is first and the backup last, step 9 explains the waiting line, and setup's closing block labels its commands `step 6` to `step 9` so the script and the README agree. Lesson: a check that talks to a shared resource must account for the schedule the same day installs.

The lock check itself was the bug (2026-09-15). After the unlock escalation shipped, hardware still showed 21 passed, 1 failed with only `no stale lock is holding the repository` red, while `lab-backup unlock` printed `no locks left` - both by hand and through setup. Neither was wrong: `restic list locks` writes a blank line before its list, so `restic list locks | wc -l` returns 1 with zero locks, and the check could never pass. `verify.sh` now counts lock IDs (`grep -cE '^[0-9a-f]{8,}$'`) instead of lines. Every other caller already used `grep -q . / grep -c .`, which is why only verify lied. Lesson repeated for the third time this day: a check that has never once passed on hardware is not a check.

Step 7 on hardware found the bug this whole day is supposed to teach, in my own code. `lab-backup status` said `last run exit : 1`, and the journal said why: `unable to locate cache directory: neither $XDG_CACHE_HOME nor $HOME are defined`, twice, at 08:33 and 08:40. A systemd service has no `HOME`, and restic derives its cache directory from `HOME` or `XDG_CACHE_HOME`, so every scheduled run failed while every run by hand succeeded - the exact "works by hand, not on the timer" failure the README lectures about, shipped live. The unit now carries `Environment=HOME=/root` and `Environment=RESTIC_CACHE_DIR=/var/cache/restic`, and setup creates `/var/cache/restic` 0700 root. Two more real defects from the same screen: `sudo lab-backup` returned `command not found` because `sudo` uses `secure_path` from `/etc/sudoers`, which on Rocky omits `/usr/local/bin`, so setup now also symlinks `/usr/sbin/lab-backup`; and the README's `set -a; . /etc/restic/env; set +a` snippet cannot work as a normal user because that file is 0600 root-only, so it is now `sudo bash -c 'set -a; . /etc/restic/env; set +a; restic snapshots'`. `teardown.sh` removes the symlink and the cache. The verify check `the scheduled service has run and exited 0` also required strengthening: it read

only `ExecMainStatus`, which is 0 for a unit that never ran, so it would have passed on this broken host; it now requires a non-empty `ExecMainStartTimestamp` too.

README restructured after the owner asked "The drill, What to actually look at - are they steps or not?" They were steps, and nothing said so. The day now opens with a **whole day, in order** table: nine numbered steps, each with the host it runs on, plus the optional `--hard` and the teardown. Every section heading carries its number and host (`Step 6 - the drill, on node1`), the four setup invocations are Steps 2-5 rather than "passes", and the table states outright that **nothing runs on control except steps 2 and 4**. Also added the line explaining that `lab-backup` and `explore-backup.sh` correctly refuse before step 5 finishes (`sudo: lab-backup: command not found, no /etc/restic/env - run scripts/setup.sh first`), since the owner hit both while setup was still incomplete and could not tell a missing prerequisite from a broken script.

Day 16 was written (2026-09-13)

`days/day16/` now ships five scripts, a rewritten README and a twelve-check verify. The day is WireGuard: one tunnel between `control` and `node1`, `10.20.0.1` and `10.20.0.2`, `wg-quick@wg0` enabled and proven across a reboot.

Decisions worth knowing:

- **setup.sh runs on BOTH hosts, twice each.** Pass 1 generates the keypair and writes `wg0.conf` with no peer; pass 2 takes the other host's public key and lab address. This is deliberately not automated from `control`: a private key that travelled to the other host is not a private key, and the out-of-band exchange is the lesson. It is the first day whose README tells the reader to `push` the repository to `node1` as well.
- **The host works out which end it is** from `hostname -s` (`control -> 10.20.0.1`, `node1 -> 10.20.0.2`), with `ROLE=` as the override. No address arguments, so nothing drifts when a DHCP lease changes - the lab address is only ever used as the peer `Endpoint`.
- **umask 077 before wg genkey, not chmod afterwards.** The window between a 0644 create and a 0600 chmod is small and a private key only has to leak once. Stated in the script and checked in `verify.sh`.
- **Re-runs use wg synconf, not wg-quick down && up.** Bouncing the interface drops the handshake, and on a management tunnel that is the session you are typing into. `break-and-fix.sh` makes the same point as its fourth failure.
- **No kernel module to build.** WireGuard is mainline since 5.6, so Rocky 9 needs only `wireguard-tools`. `setup.sh` still checks `/sys/module/wireguard` and `modprobe`, because the failure mode on a stripped kernel is otherwise a confusing `wg-quick` error.
- **AllowedIPs is the day.** Taught as a route outbound and a source-address ACL inbound, which is why the two ends are coupled and why narrowing one side makes the *healthy* end look broken. `lab-wg routes` prints the configured value, the routes it became, and `ip route get` next to each other.
- **break-and-fix.sh covers four failures, none of which produce an error message:** a valid-looking peer key belonging to nobody (no handshake, nothing in `journalctl -k`), `AllowedIPs` narrowed to an address the peer does not have, `MTU = 1500` where 1420 belongs (ping passes, `-M do -s 1400` hangs), and a file edited after `wg-quick up` read it. `--hard` describes two lock-outs rather than causing them: `AllowedIPs = 0.0.0.0/0` stealing the default route out from under the SSH session, and `wg-quick down` on the far end over the tunnel itself.
- **verify.sh has twelve automatic checks**, more than any earlier day, because every piece of this day is a separate claim: module loaded, interface addressed, `wg0.conf` and `wg0.key` at 0600, a peer that is not

this host's own key, a handshake that actually completed, a ping inside the tunnel, `AllowedIPs` having become a route, a full-size packet surviving (the MTU check), file and kernel agreeing, the port permanent in `firewalld`, and the unit enabled. Two judgement calls: explaining `AllowedIPs` in each direction, and having rebooted and watched the tunnel return.

- **`wg-quick strip` versus `wg showconf` formatting.** The first hardware run failed one check, `the file` and `the kernel agree`, on a correctly working tunnel: the raw diff compared formatting, and `strip` preserves the file's own key order and column spacing while `showconf` prints the kernel's normalized form. Both `verify.sh` and `break-and-fix.sh` now collapse whitespace, keep only `PublicKey|PresharedKey|AllowedIPs|Endpoint| PersistentKeepalive`, and `sort` both sides. Emulated locally in `/tmp`-style harness: agreeing config passes, `PersistentKeepalive = 99` in the file still fails. General rule: **compare settings, not text.**
- **`firewall-cmd --permanent` versus `firewall-offline-cmd`.** The first hardware run of Day 16 died at step 4 on both hosts with `Firewalld is not running: --permanent` talks to the daemon and does not fall back to editing the XML when it is stopped, and a freshly built lab VM has `firewalld` installed and stopped (Day 11 starts it, and only on `node1`). `setup.sh` now branches on `systemctl is-active firewalld`, uses `firewall-offline-cmd` when it is down, then starts and enables the daemon so the policy is actually loaded. `teardown.sh` has the same split and `verify.sh` accepts either query. General rule: **a day may not assume another day's daemon is running.**
- **Day 14 and Day 15 are not prerequisites** and the README says so. Today touches nothing they built - no `ansible.cfg`, no project directory.

Run on hardware (2026-09-13): 12 passed, 0 failed, 2 for you to judge on both `control` and `node1`. One check needed fixing first - see the note on comparing settings rather than text, above. `bash -n` clean, `exec` bits set on all six files.

Day 13 was written (2026-09-12)

`days/day13/` now ships five scripts plus a policy source file, a rewritten README and a seven-check verify. The day is SELinux: serving `/srv/www` with `nginx` while the machine stays enforcing.

Decisions worth knowing:

- **Port 8080, not 80.** Rocky's `nginx.conf` already has a default server on 80, and a second `default_server` is an `nginx` error, not an SELinux one. `8080/tcp` is already `http_cache_port_t`, so the bind succeeds and the port lesson stays a deliberate exercise rather than an accident.
- **`setup.sh` refuses to run unless SELinux is enforcing.** It will flip Permissive to Enforcing, but a Disabled system needs an `autorelabel` and a reboot, so it dies with those instructions instead of pretending.
- **Nothing in the day ever runs `setenforce 0`.** Every fix is an `fcontext` rule plus `restorecon`, a `semanage port` entry, a `-P` boolean, or a module whose source was printed first. `--hard` describes `setenforce 0` and a full filesystem relabel without performing either.
- **The module ships as readable source.** `scripts/lab_selinux.te` grants `httpd_t` read on `var_log_t` and nothing else, with the permissions named individually, because the teaching point is that `audit2allow` will happily write `:file *`. `setup.sh` compiles it with `checkmodule / semodule_package`, printing the rules before `semodule -i`.

- `teardown.sh` removes the `fcontext` rule before the directory, so no local rule is left describing a path that no longer exists, and it leaves SELinux enforcing because Days 14-20 assume a labelling machine.
- Verify needs root (`semanage` , `semodule` , `restorecon` read non-world-readable policy) and checks the permanent forms: `semanage fcontext -l -C` , a silent `restorecon -nvR` , and `semanage boolean -l -C` .

The four failures in `break-and-fix.sh` : a `chcon` that a relabel reverts, an `httpd` type that is still not readable content, an unlabelled port that refuses a valid bind, and a boolean that was off when no module was needed.

Day 13 **passed on real hardware: 7 PASS, 0 FAIL, 2 YOU** (2026-09-12), on `node1` with SELinux enforcing throughout. CI still only lints it - a GitHub runner has no policy store.

Packages the VM needs: `sudo dnf install -y nginx policycoreutils policycoreutils-python-utils checkpolicy setools-console audit curl` .

Day 12 was written (2026-09-11)

`days/day12/` now ships five scripts, a 233-line README and a five-check verify. The day is SSH hardening: keys only, `AllowGroups labssh` , a fail2ban jail with a two-minute ban, and ProxyJump.

Decisions worth knowing:

- **One VM, not two.** The curriculum lists this day as `control + node1` because a bastion needs two hosts. All five automatic checks run on `node1` alone, and the README gives a single-host ProxyJump exercise (`ssh -J lab@node1 lab@127.0.0.1`) for anyone without the 3 GB.
- **setup.sh refuses to run if the login user has no `authorized_keys`** . Disabling passwords on an account with no key is the way people lose a cloud VM, and a teaching script must not be able to do it.
- **Nothing in this day ever restarts sshd.** Every change is validated with `sshd -t` and loaded with `systemctl reload sshd` , which keeps the current session alive even when the new policy would refuse it.
- **fail2ban comes from EPEL** on Rocky 9, so setup fails with the two `dnf` lines rather than a bare "command not found".
- **`v1_need_root` and absolute `/usr/sbin/sshd`** in `verify.sh`: `sshd -T` reads host keys and `/usr/sbin` is not on a normal PATH. Same root- required lesson as Day 11.
- The payload is `/usr/local/bin/lab-ssh` , and every reference to it in the README and the scripts uses the **full path**, because `sudo` on Rocky uses a `secure_path` that excludes `/usr/local/bin` .

The five failures in `break-and-fix.sh` : a later drop-in that never wins (00 sorts before 70, first value wins), an allow list that excludes you, self-ban, a `Match` block where `sshd -T` is green and `sshd -T -C` is not, and the unsurvivable one, described and not executed.

Day 12 was first exercised end to end in the authoring sandbox against mock `sshd` , `fail2ban-client` and `systemctl` . Real Rocky 9 execution then exposed three portability issues, all now handled by the repository:

- `node1` was raised from 768 MB to 2048 MB because `dnf` could be killed by the OOM killer while installing EPEL and fail2ban. All memory tables and launch notes were updated in the same change.
- The managed policy is `00-lab-hardening.conf` , not `60-` , so it is read before Rocky's and cloud-init's `50-*` drop-ins. Setup removes the legacy `60-` filename and teardown cleans up both names.
- With `set -o pipefail` , `sshd -T | grep -q` could report a false failure after `grep` found its match and `sshd` received `SIGPIPE` . Day 12's checks now consume the full output (setup captures `sshd -T` once). Rocky/OpenSSH may canonicalize `PermitRootLogin prohibit-password` as the equivalent `without-`

`password` ; setup and verify accept `no` , `prohibit-password` , or `without-password` as policies that disable root password login.

These fixes were applied to `setup.sh` , `verify.sh` , the related diagnostic and cleanup scripts, the Day 12 README, and this handoff. **Day 12 then passed on real hardware: 5 PASS, 0 FAIL, 2 YOU** (2026-09-12). `sshd -T` on that VM prints `permitrootlogin without-password` , confirming the alias handling was necessary, and `lab-ssh root` correctly reports `REFUSED - not in any allowed group` because root is not in `labssh` . The executable bits on all five Day 12 scripts and `verify.sh` were also restored after archive merging, and `./tests/cli.sh` is green. The remaining two Day 12 items are the manual ones: the ProxyJump exercise must be run from the **laptop**, not from a shell on `node1` , and with the address `./lab/lab.sh status` prints rather than a remembered one.

Day 11 was written (2026-09-10)

Day 11 is a VM day - `node1` , Rocky 9 - and the first VM day since Day 05, because Days 06 to 10 needed no virtual machine at all. Every script sources `lab/on-lab-vm.sh` and calls `require_lab_vm` , per Rule 8. It cannot be run in the authoring sandbox or in CI: no firewalld, no nftables, no systemd units to own. CI lints it and nothing more, which the README says plainly.

The day builds both halves of reachability, because a firewall with nothing behind it teaches nothing.

`setup.sh` starts and enables firewalld, forces the default zone to `public` , installs `/usr/local/bin/lab-web` (a one-line `python3 -m http.server 8080 --bind 0.0.0.0`) as `lab-web.service` , opens 8080/tcp permanently AND reloads, adds one rich rule (9090/tcp from 127.0.0.0/8 only), and then prints the `nft` chains its own commands produced.

The spine of the day is that "is the port open" is three questions answered by three programs: `ss` (is anything listening), `firewall-cmd` (what the policy intends, runtime and permanent separately), and `nft` (what the kernel will actually do). The payload `lab-fw` prints all three side by side and names the mismatch; with no argument it ends in a "where the two disagree" section that lists open ports with no listener and listeners the firewall blocks.

`break-and-fix.sh` has three cases plus two behind `--hard` : a runtime-only change that vanishes at the next reload, an open port with the service stopped, the `trusted` default zone (a healthy firewalld enforcing nothing

- * which is exactly what verify's fifth check exists to catch), then `--hard` adds a second nftables table hooked at priority -300 that drops the packet while `firewall-cmd --query-port` keeps saying yes, and finally the SSH lockout, which is DESCRIBED AND NOT EXECUTED because it would end the reader's session with no console to recover from. `--timeout=120` is taught there as the habit that makes remote firewall work safe.

`restore()` runs on `trap ... EXIT INT TERM` , re-adds the port, resets the default zone, deletes the stray nft table, restarts the service, and then CHECKS the result with `--query-port` and a real curl probe rather than assuming it - the Day 09 lesson applied here.

`verify.sh` is the stub's, unchanged: five automatic checks and one manual. The README was written to it, parity verified.

Day 10 was written (2026-09-10) and RAN GREEN in the authoring sandbox

Day 10 is the first day since Day 04 that could be executed where it was written, because it needs only `openssl` - no namespaces, no VM, no `ip` . Setup, verify, the tour, `break-and-fix.sh --hard` and teardown

were all run end to end here: **7 passed, 0 failed, 2 YOU**. The header row says "Host: openssl only, no namespaces", changed from the stub's "Host: network namespaces", because the day does not use them.

Everything lives in `/etc/lab-tls` (mode 0700), plus `/run/lab-tls` for the pidfile and the server log, and `/usr/local/bin/lab-tls` for the payload. The CA is self-signed with `basicConstraints=critical,CA:TRUE` and `keyUsage=critical,keyCertSign,cRLSign`, valid 3650 days. Three leaves are issued by one `issue()` function that differs only in SAN and dates: `server.crt` (SAN `www.lab.test`, 365 days), `wrongname.crt` (SAN `other.lab.test`), and `expired.crt` (`-not_before` 30 days ago, `-not_after` yesterday, with a `-days -1` fallback for older openssl). Every leaf gets `basicConstraints=CA:FALSE`, `keyUsage`, `serverAuth`, and a SAN list including `localhost` and `IP:127.0.0.1` so the server can be reached on loopback.

The teaching point the scripts are built around: `openssl verify` accepts `wrongname.crt` because it checks the chain and the dates and is never told what hostname you wanted. Hostname matching is the client's job (`-verify_hostname`), which is why chain, dates and hostname are reported separately by `lab-tls` and by every case in `break-and-fix.sh`.

`s_server` runs as `nohup openssl s_server -accept 4433 -naccept 200 -www` with its pid in `/run/lab-tls/s_server.pid`. `break-and-fix.sh` swaps the served certificate with a `serve()` helper rather than editing files, and a `trap restore EXIT INT TERM` puts the good pair back. Case 4 (cert with the wrong key) is the only case where the SERVER fails to start; it was verified to print `key values mismatch` from the real openssl log.

Verify has seven automatic checks, one of them negative: a handshake with no `-CAfile` must be REFUSED. Teardown stops the pidfile server, then kills any stray `s_server` on 4433 by `pgrep`, then proves the port is free with `ss`

- the Day 08 stray-daemon lesson applied to a port instead of a namespace. Nothing is ever added to a system trust store; Day 17 is the day that does that, and it reuses this CA.

Day 09 was written (2026-09-10) and has not been run locally

Day 09 adds no topology. It reuses Day 06's four namespaces and installs one observer, `/usr/local/bin/lab-trace`, plus a capture directory at `/var/log/lab-trace`. `setup.sh` rebuilds Day 06's topology if a namespace is missing (same idiom as Days 07 and 08), then takes one real capture at `router:veth-rc1` and reads it back, so the day proves its own tooling before teaching with it.

The interface to watch is `veth-rc1`, the router's end of the client's veth pair. Capturing in the middle is the whole point: it is the only place that can tell "never sent" apart from "never arrived".

`break-and-fix.sh` has five cases. Three are loud (interface down, no route, far end with no route back) and two are behind `--hard`: an MTU of 1280 that passes ping and refuses `-M do -s 1400`, and a permanent bogus ARP entry for 10.10.0.1. All are undone by a `trap restore EXIT INT TERM`, so an interrupt cannot leave the network broken.

Every capture uses `tcpdump -c N -w FILE` started one second before the traffic and killed afterwards if `-c` did not end it. That pattern lives in one function, `capture_probe`, rather than being copied per case.

Day 08's DNS is optional for Day 09. Setup reports whether the resolver is still on `:53` and continues either way, so CI does not depend on Day 08 having run.

Also fixed in this pass: Day 08's `teardown.sh` no longer asserts "Nothing, after a two second wait" without reading the command's output. It captures the answer and branches - empty means the timeout paragraph,

non-empty means a paragraph explaining that a name still resolving after its server is gone is the most misleading state in DNS, with two commands to find who answered.

Day 08 was written (2026-09-09) and has not been run locally

Five scripts: `lab-dnsq.sh` (payload), `setup.sh`, `explore-dns-server.sh` (read-only tour, twelve sections), `break-and-fix.sh`, `teardown.sh`. All five need root, like Days 06 and 07. No `on-lab-vm.sh` guard, same reason: namespaces cannot reach the host's own stack. CI executes this day for real.

The authoring sandbox has neither `ip` nor `unbound`, so nothing here was executed. Every config was written against the unbound documentation and is validated at runtime by `unbound-checkconf` before the daemon reads it, which is the strongest guarantee available without running it. CI is the first real run.

DECISION	WHY
<code>unbound</code> for BOTH halves, not bind for one	<code>ci.yml</code> already installs <code>unbound</code> and nothing else DNS-serving. One binary doing both jobs is also the better lesson: <code>local-zone: "lab.test." static</code> is the entire difference between authoritative and recursive
Configs under <code>/etc/unbound/lab/</code> , started with explicit <code>-c</code>	Never touches the distribution's own <code>unbound.conf</code> or its service. A reader with unbound already running on port 53 is unaffected, because both instances live in namespaces
<code>local-zone: "." refuse</code> on the auth server	Makes it non-recursive, which produces <code>REFUSED</code> for outside names. That gives the day three distinguishable rejections - <code>NXDOMAIN</code> , <code>SERVFAIL</code> , <code>REFUSED</code> - instead of two
<code>private-domain: "lab.test."</code> on the resolver	Without it unbound strips RFC1918 answers as DNS-rebinding protection and every reply comes back <code>NOERROR</code> with zero answers, nothing logged. The single most confusing failure available in this stack, so it is called out in the README's Notes
<code>www.lab.test</code> TTL is 30 seconds	A full cache expiry is watchable inside a minute. <code>--hard</code> then sets the same record to 86400 to show a correct, completed migration serving the old address for a day
<code>setup.sh</code> kills Day 07's nameserver	It holds <code>10.10.1.2:53</code> . Otherwise unbound fails with "address already in use" and the reader debugs today's config instead of yesterday's leftovers
Payload is a query tool, not a daemon	unbound IS the daemon today. <code>lab-dnsq</code> asks both servers and prints status/flags/TTL/answer aligned, so "run it twice" makes the cache visible with no tooling
Day 06's topology rebuilt, not required	The rule earned by Day 07's CI failure - see below

The reader's first run found one bug: unbound ships a built-in `local-zone` for the reserved TLD `.test` (RFC 6761) that answers `NXDOMAIN` before any `stub-zone` is consulted, so the resolver denied every name while the authoritative server answered correctly. The tell was the SOA in the reply naming `localhost.`. Fixed with `local-zone: "lab.test." nodefault` and `domain-insecure` on the resolver. Same class of default as `private-domain`; Days 10, 12 and 17 inherit both.

Also fixed then: `lab-dnsq` keyed answer/TTL extraction on the query type, so a CNAME reply printed `-` for both, and the README wrongly claimed a CNAME returns two records - a static local-zone does not chase the alias.

Verify checks were all five tightened from the generator's stubs, which had `grep -q` on three of them (any answer at all would pass). They now match exact addresses and the SOA's own mname/rname. `v1_need dig ip ss` gained `ss`, which check one needs. A second `v1_manual` was added - naming what `REFUSED`,

SERVFAIL and NXDOMAIN each mean - and the README's manual list was updated in the same change, because parity compares descriptions.

Day 07 was written (2026-09-09), and its payload was run, but the day was not

Five scripts: `lab-nameserver.sh` (payload), `setup.sh`, `explore-dns.sh` (read-only tour, twelve sections), `break-and-fix.sh`, `teardown.sh`. All five need root, like Day 06. No `on-lab-vm.sh` guard, for the Day 06 reasons - everything it writes lives under `/etc/netns/<ns>/` and is invisible outside the namespace it belongs to. No `ci.yml` override needed.

Objects: `/usr/local/bin/lab-nameserver`, `/etc/netns/client/{resolv.conf, nsswitch.conf, hosts}`, `/etc/netns/resolver/resolv.conf`, `/run/lab-nameserver.pid`, `/var/log/lab-nameserver.log`, backups in `/root/day07-backup`. Zone served: `lab.test`, `www.lab.test`, `auth.lab.test` -> 10.10.2.2, `client.lab.test` -> 10.10.0.2. The hosts file says `www.lab.test` is 10.10.0.99, so `getent` and `dig` disagree on purpose.

The payload WAS executed in the authoring sandbox and works. `sudo ./lab-nameserver.sh serve 127.0.0.1`, then `dig @127.0.0.1`: A records returned with the right TTL and address, NXDOMAIN for an unknown name, and every query logged with the asker's address. What could NOT be run is the day itself, because the sandbox still has no `ip` binary and no network to install one - so `setup.sh`, `explore-dns.sh`, `break-and-fix.sh`, `teardown.sh` and `verify.sh` are linted only, and CI is their first real run.

CHOICE	WHY
A hand-written ~60-line Python nameserver instead of <code>dnsmasq</code> or <code>unbound</code>	Day 08 is "Running DNS: authoritative and recursive" and owns <code>unbound</code> . If Day 07 configured a real server there would be nothing left for Day 08 to teach. The payload also needs no package that a CI runner might not have: <code>python3</code> is everywhere, and a DNS answer is a byte layout, so it cannot be faked with <code>printf</code> and netcat.
Day 07 does not invent a network; it calls Day 06's idempotent <code>setup.sh</code> when the namespaces are absent, then pings across it before relying on it	First real inter-day dependency, and the first version got it wrong: it <i>died</i> telling the reader to run Day 06 first. Namespaces do not survive a reboot, and every CI job is a fresh runner, so a missing topology is the NORMAL case. CI failed on exactly that (<code>no 'client' namespace</code> , exit 1) while Days 06, 08, 09, 10 and 18 passed, because they build their own floor. A prerequisite that can be rebuilt in one second must be rebuilt, not reported.
Per-namespace config via <code>/etc/netns/<ns>/</code> rather than editing <code>/etc/resolv.conf</code>	This is the day's best single fact: <code>ip netns exec bind-mounts /etc/netns/NAME/foo</code> over <code>/etc/foo</code> for the life of that one command. It is also what makes the day safe on a reader's laptop and what makes failure 5 possible.
<code>setup.sh</code> fails if <code>dig</code> and <code>getent</code> ever agree about <code>www.lab.test</code>	If they agree, the hosts entry is not being read and the entire day teaches nothing. Better to stop at setup than to have the reader work through a tour whose premise is silently false.
Four of the five failures print no error at all	The thesis of the day: DNS failures are mostly not errors, they are correct answers from the wrong layer. Failure 1 (drop <code>files</code> from <code>nsswitch</code>) is the sharpest - <code>getent</code> starts <i>agreeing</i> with <code>dig</code> , and a pinned address silently stops applying.
<code>--hard</code> = trailing dot in <code>/etc/hosts</code> , and the right file in the wrong <code>/etc/netns</code> directory	Both pass a configuration review. <code>/etc/hosts</code> matches literal strings, so <code>www.lab.test.</code> never matches <code>www.lab.test</code> ; and <code>grep -r</code> finds a misfiled hosts entry immediately, which is exactly why people believe it is being used.
Failure 3 uses SIGSTOP, not SIGTERM	Distinguishes timeout (socket open, nobody reading) from connection-refused (nothing listening) from NXDOMAIN (a server said no). All three get reported as "DNS is broken". <code>teardown.sh</code> sends SIGCONT before SIGTERM for this reason - a stopped process cannot act on SIGTERM.

`verify.sh` **was rewritten, not extended.** The stub had two checks that could pass without the day being done: `grep -qE "^hosts:" /etc/nsswitch.conf` read the *host's* file, which is `files dns` on nearly every Linux machine; and the nameserver check ended in `|| ip netns exec client cat /etc/resolv.conf`, which succeeds whenever the file exists. Both now read from inside the namespace and match on the exact address. This clears one of the four known weak checks (the d07 one). A fifth check was added, `the nameserver answers from the resolver namespace`, taking the day to 5 PASS + 2 YOU and matching Days 04-06. `vl_need ip getent dig + vl_need_root`.

README is 176 lines and follows **Day 06's** host-day headings, not Day 05's VM sequence. It carries the layer diagram, a five-failures table, and states plainly that `dig` is a DNS client rather than a resolver - it reads `resolv.conf` only to find a socket, and never reads `nsswitch.conf` or `/etc/hosts`.

Day 06 was written (2026-09-08), and has not been run anywhere

Five scripts: `lab-netcheck.sh` (payload), `setup.sh`, `explore-net.sh` (read-only tour), `break-and-fix.sh`, `teardown.sh`. All five need root, which is a first for this repo - `ip netns exec` is privileged and there is no unprivileged substitute. Objects: namespaces `client router resolver auth`, veth pairs `veth-cl/veth-rc1`, `veth-rs/veth-rrs`, `veth-au/veth-rau`, three /24s on 10.10.0/1/2, `net.ipv4.ip_forward=1` in the router namespace only.

This is the first day whose scripts do NOT source `lab/on-lab-vm.sh`, and that is deliberate, not an oversight of Rule 8. A network namespace is a separate copy of the kernel's network stack; nothing here adds an address, a route or a sysctl to the namespace the host actually uses. That isolation is the same fact that makes Days 06-10 cost 0 MB and makes them CI-executable. Rule 8 exists to stop a day rewriting the reader's own machine, and this day cannot. No `LAB_ALLOW_THIS_MACHINE` override is needed in `ci.yml` either - there is no guard to override, unlike Day 04.

CHOICE	WHY
<code>setup.sh</code> builds the topology itself instead of calling <code>lab.sh netns-up</code>	<code>ci-day.sh</code> prefers <code>scripts/setup.sh</code> when it exists, so adding one silently changes what CI runs for Day 06. Building it here in seven narrated steps is the day's actual content; <code>lab.sh netns-up</code> remains the one-command form for Days 07-10 and 18. Same names, same addresses, so a later day cannot tell which was used.
Idempotent via <code>addr replace / route replace</code> , and repairing rather than dying on a second run	<code>lab.sh netns-up</code> dies if a namespace already exists. A reader who ran that first and then <code>setup.sh</code> would hit a dead end on line 1. <code>setup.sh</code> also clears veth ends stranded in the root namespace by a half-finished run.
No <code>set -e</code> in <code>lab-netcheck.sh</code>	A reachability probe whose job is to report failures must not exit on the first one. Its exit status still reflects the network.
Five failures chosen to produce four <i>distinguishable</i> symptoms	The skill being taught is reading the symptom, not the fix. Instant "unreachable" = local routing; timeout with a reachable next hop = forwarding or a filter; timeout with requests arriving = return path; timeout with everything apparently correct = <code>ip route get</code> . The README carries this as a table.
<code>--hard</code> = link down with the address still present, and <code>/16</code> where <code>/24</code> belonged	Both look like working configuration. The first withdraws the connected route while <code>ip addr</code> still shows the address; the second makes the client ARP for hosts two networks away. Neither would fail a config review.
Second <code>v1_manual</code> added: you saw a ping fail in one direction only, and proved which one	Ties failure 3 to the checklist and makes the point that a ping proves a round trip, so a failed ping never says which direction broke. Takes the day to 5 PASS, 2 YOU, matching Days 04 and 05.
<code>10.10.0.2</code> tightened to <code>10.10.0.2/24</code> in check 2	The loose form also matched <code>10.10.0.20</code> and, worse, the <code>/16</code> of failure 5 - so break-and-fix could leave the day broken and still green.

The README departs from the Days 01-05 runbook shape in one visible way: the six step headings are not the VM sequence (`bring up node1`, `copy the repo`, `prove it across a reboot`). There is no VM to bring up and nothing survives a reboot by design, so step 1 is "No VM today" and step 5 is the check. Days 07, 08, 09 and 18 should follow **Day 06's** headings, not Day 05's.

Day 04 was written (2026-09-08), and RAN GREEN on node1

First day after Day 01 to be executed on a real Rocky VM. Final result: **5 PASS, 0 FAIL, 2 YOU, exit 0**, with the ghost file found via `sudo lsof +L1` showing `NLINK 0` on `/srv/data/ghost.bin`. `setup.sh`, the live `lvextend + resize2fs` grow, and `break-and-fix.sh --hard` all completed for real.

Three bugs the run exposed, all fixed:

1. **No `v1_need_root` in `verify.sh`.** `vgs` and `lvs` are root-only while `/proc/mounts` is world-readable, so a non-root run printed FAIL for the volume group and PASS for the mount served *by* that volume group - a convincing false red. The README had *documented* the hazard instead of fixing it. Standing rule now: **if a day's checks read privileged state, call `v1_need_root`; never substitute README prose for a missing guard.**
2. **Bare `lab-writer` in the README.** Installed to `/usr/local/bin`, which `sudo 's` `secure_path` omits on RHEL-family systems. Now called by path.
3. **No package list.** The Rocky 9 cloud image ships neither `lvm2`, `e2fsprogs` nor `lsof`, and `lab.sh` cloud-init installs nothing. Every day must now name the packages it needs, and the dependency check must cover every tool the whole day uses, not just `setup.sh`'s own.

Also confirmed by the run: `./lab/lab.sh down node1` really does take the `add-disk` disk with it, and a rebuilt VM has no packages - `pvs / vgs / lvs` printing nothing is a blank machine, not a broken one. Documented in the day's step 3.

Day 04 as originally written

Five scripts, same shape as Days 01-03: `lab-writer.sh` (payload, no root), `setup.sh`, `explore-storage.sh`, `break-and-fix.sh`, `teardown.sh`. Objects: PV on `/dev/vdb` or a loop device, VG `labvg`, LV `labdata` (512 MB), ext4, mount point `/srv/data`, unit `srv-data.mount`, payload installed as `/usr/local/bin/lab-writer`.

Why each choice, so the next person does not undo one by accident:

CHOICE	REASON
LV starts at 512 MB, not the whole disk	the day is about growing it online; a full-size volume has nothing to teach
ext4, not xfs	both grow mounted, only ext4 can shrink. <code>--hard</code> step 5 needs a shrink that is <i>refused for a reason</i> , not one that is impossible
<code>.mount</code> unit, not <code>/etc/fstab</code>	so the filename-must-match-the-path rule can be broken on purpose in <code>break-and-fix.sh</code> step 2
real disk preferred, loop device as fallback	a loop device is real LVM but does not survive a reboot, so step 5 of the README needs the real one. <code>setup.sh</code> prints which it chose and why
refuses any device with a <code>blkid</code> signature, partitions, or a mount	this is the most destructive script in the repo; <code>pvccreate</code> on the wrong device is unrecoverable
never reformats an existing <code>labdata</code>	a second <code>setup.sh</code> run must not destroy the reader's data

Expected on a healthy run: **5 PASS, 2 YOU, exit 0.**

Day 04 is the first day CI actually executes. `lab/ci-day.sh 04` finds `scripts/setup.sh` and runs it as root on an Ubuntu runner, which is not a lab VM by hostname, so `require_lab_vm` would refuse. Resolved by setting `LAB_ALLOW_THIS_MACHINE=1` on the day-04 matrix step only, in `ci.yml`, with a comment explaining it — a runner is destroyed after every job, which is the only thing that guard is really asking about. Rule 8 is unchanged and the other six CI days do not need this: days 06-09 and 18 work inside network namespaces, and day 10 only writes files under `ca/`.

The likeliest real-world failures, in order: `lsyf` not being installed on a minimal Rocky 9 image (`break-and-fix.sh` step 3 falls back to `/proc/PID/fd` and says so); `resize2fs` on a loop device inside CI behaving differently from one on virtio; and the ordering of `udev` settling after `lvcreate`, which can make `mkfs` race on a slow machine.

First real execution of any `verify.sh` (2026-09-08)

The owner ran three days on their **Ubuntu laptop**, not on a VM. This is the first execution evidence in the project, and it is all off-target on purpose:

DAY	RESULT ON THE LAPTOP
01	1 PASS, 3 FAIL, 1 YOU — the PASS is <code>the machine boots with no failed units</code> , which is about the laptop
02	5 SKIP, 1 YOU — <code>v1_need_root</code> caught it, the only day that degrades honestly
03	1 PASS, 3 FAIL, 2 YOU — the PASS is <code>cgroups v2 is the unified hierarchy</code> , true of any modern Linux

Two lessons were taken from this. A lab-VM guard in `verify-lib.sh` was considered and **declined** by the owner in favour of documentation, so each day's README now carries a bolded "Run it on `\<vm\>`", not on

your laptop" paragraph that names the checks which would falsely `PASS` off the VM. And a sentence in Day 03's README claiming "if every check reports SKIP, systemctl is missing" was factually wrong and has been removed — the laptop has systemctl.

Sandbox limitations worth knowing

No shellcheck, bats, `ip`, `nft`, KVM, libvirt or network access. Verification is therefore `bash -n`, targeted `grep`, running the error paths, and reading output carefully. Also: file writes have silently succeeded while creating nothing — always confirm with `ls -l` before trusting a write.

9. Open items

In the order they should probably be done.

1. **Finish the first real host run.** `check` has now been run on the owner's Ubuntu laptop (2026-09-06): host, KVM, memory and disk sections all pass; it correctly reported 5 blocking problems because libvirt, virtinst and qemu were not installed. Nothing past `check` has run for real yet: `image`, `up control`, `push control`, `ssh control` are still untested against real KVM, as are all five Day 01 scripts against real systemd.
2. **Write the day scripts.** Days 01 through 19 are done (5 scripts each). Day 20 ships an empty `scripts/` directory. They are written one day at a time, each run on the real lab before the next is started — writing them in bulk would produce plausible code that has never met a Rocky VM. Delivery convention agreed with the owner: **Day 01 shipped as the complete repository; every day after that ships only the changed files**, which will usually be `days/dayNN/scripts/*`, `days/dayNN/README.md`, `days/dayNN/verify.sh`, and this handoff.
3. **Add `scripts/setup.sh` for the CI days** (04, 06, 07, 08, 09, 10; 18 is done, and is now a real CI job rather than a `SKIPPED` one). Each one converts a `SKIPPED` CI job into a real one.
4. **Git.** Not initialised. No remote, no first commit. The owner is doing this themselves.
5. **Pre-commit: shipped.** `.pre-commit-config.yaml` has whitespace and shebang hooks, `shellcheck -x -S warning`, a 256 KB large-file guard (VM images must never enter git), and `tests/cli.sh` as a local hook. **Never executed** — `pre-commit` is not installed in the authoring sandbox and installing it needs network. The YAML is tab-free and was structurally checked, nothing more. Confirm with `pre-commit run --all-files` on first use.
6. Optional: a `lab.sh snapshot / revert` pair using `virsh snapshot-create-as`. Days 11–13 and 19 involve changes that can lock a host out, and a one-command revert would make experimentation cheaper.
7. Optional: extend past 20 days. Candidates never written up: Kerberos and FreeIPA, NFS with SELinux labels, systemd-nspawn, kernel tunables and sysctl, performance triage with `perf` and `bpftrace`.

10. Maintenance rules

When changing this repository, keep these in sync:

IF YOU CHANGE	ALSO UPDATE
A day's content	its <code>README.md</code> and <code>verify.sh</code> , together
A day's scripts	the Scripts table and runbook on its page; §8 and §11 here
A day's tier	the CI matrix in <code>ci.yml</code> , §4 and §5 here, and the README tier table
The number of days	both tier tables, the memory tables, <code>docs/curriculum.md</code> , §4 here
A check	the count in §5 here (<code>102 automatic</code> , <code>31 judgement</code>)
<code>lab.sh</code> subcommands	§3 here, the README lab section, <code>lab/README.md</code>
VM names or memory	§3 and §4 here, both memory tables, <code>vm_mem()</code> in <code>lab.sh</code>
The namespace topology	§3 here, the README diagram, <code>NS_LIST</code> and <code>cmd_netns_up</code>
A <code>lab.sh</code> subcommand	the header help block too, or <code>tests/cli.sh</code> fails
A rule in §2 or §5	<code>CONTRIBUTING.md</code> restates all three headline rules
The day-script conventions	<code>CONTRIBUTING.md</code> "Writing day scripts"
Anything at all	run <code>./tests/cli.sh</code> , then the Last updated date at the top of this file

Rules that exist for a reason

1. **Never claim something works that has not been executed.** The distinction between "lint-clean" and "verified" is the whole point of §5.
2. **Never add a simulation or offline mode**, however convenient. If a thing cannot be verified for real, say so plainly instead.
3. **Never describe network namespaces as fake.** See §2.
4. **A day page and its `verify.sh` must never disagree.** See §7.
5. **A green CI run does not mean a day works.** It means lint passed and, for 7 of 20 days, that a runner executed them.
6. **Confirm file writes with `ls -l`.** They have silently no-opped.
7. **Do not put lab state in the repo.** Everything lives under `$LAB_HOME` ; `.gitignore` covers qcow2 images, pcaps, private keys and vault passwords.
8. **Every script that changes system state must `source lab/on-lab-vm.sh` and call `require_lab_vm`.** Day 01 was run on the user's laptop because no script asked where it was. Read-only tours are the one deliberate exception. A teardown may skip the guard only when everything it removes is unambiguously ours, as in Day 01; Day 02's teardown deletes a user and `/srv/shared` , so it is guarded.
9. **Name the machine in every instruction.** "Run `sudo reboot` " is a bug. "Reboot the VM: check `hostname` prints `control` , then `sudo reboot` " is not.

11. File inventory

README.md	9.4 KB	overview, quickstart, tiers, memory budget
docs/curriculum.md	9.5 KB	all 20 days with reasoning
docs/HANDOFF.md		this file
lab/lab.sh	36.2 KB	1043 lines, 42 functions, 14 subcommands
lab/verify-lib.sh	2.7 KB	100 lines, 6 public functions
lab/on-lab-vm.sh	2.0 KB	65 lines, refuses to run off a lab VM
lab/ci-day.sh	1.9 KB	57 lines
lab/README.md	3.3 KB	hardware, install, manual fallbacks
days/dayNN/README.md	20 files	
days/dayNN/verify.sh	20 files,	102 auto + 31 judgement checks
days/day01/scripts/	5 files	470 lines: lab-demo.sh, setup.sh, explore-boot.sh, break-and-fix.sh, teardown.sh
days/day02/scripts/	5 files	~34 KB: lab-app.sh, setup.sh, explore-perms.sh, break-and-fix.sh, teardown.sh
days/day03-20/scripts/	18 empty	directories
.github/workflows/ci.yml	2.5 KB	79 lines, 3 jobs
CONTRIBUTING.md	7.7 KB	208 lines, 9 sections
LICENSE	1.1 KB	MIT, holder: ericvalijani
tests/cli.sh	4.2 KB	127 checks, no VM or root needed
.pre-commit-config.yaml	1.3 KB	never executed, see §9 item 5
.gitignore	413 B	33 lines

50 shell scripts, all `bash -n` clean. 20 days. Day 01 written and run for real on the lab; **Day 04 written and run end-to-end on node1 (2026-09-08)**; Days 02, 03 and 05 written, never executed on a Rocky VM. Days 06, 07 and 08 written and never executed locally - they need no VM, but the authoring sandbox has no `ip` and no `unbound`, so CI is their first real run (Day 07's nameserver payload alone WAS run and works). Day 06 is green in CI; Day 07's first CI run failed on a missing prerequisite and was fixed by rebuilding it. Every day 01-20 is now written. Days 12 through 17 run green on hardware (Day 17: 19 passed, 0 failed, 2 to judge), Day 18 runs green on the owner's laptop with no VM (22 passed, 0 failed, 2 to judge), Day 19 runs green on node1 (22 passed, 0 failed, 2 to judge) and Day 20 is written and not yet run. `tests/cli.sh` : 397 passed, 0 failed.

LICENSE

MIT License

Copyright (c) 2026 ericvalijani

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.